

Développement front-end avancé



Références d'enquêtes annuelles :

<https://survey.stackoverflow.co/>

<https://stateofjs.com/en-US>

Partie 1: Rappel HTML, CSS et JavaScript



Le Web est construit avec HTML, CSS et JavaScript.

HTML



Structure et contenu d'une page Web

CSS



Mettre en forme une page web (Design)

JavaScript



- **Structure dynamique** d'une page web (**DOM**)
- **Interactivité** d'une page web (**événements**)
- **Requêtes asynchrones** pour récupérer des données (**XMLHttpRequest-XHR**)

JavaScript – Structure dynamique (DOM)

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>HeH - DST</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Bachelier en Informatique</h1>
```

```
  <div>
```

```
    <p>L'équipe pédagogique</p>
```

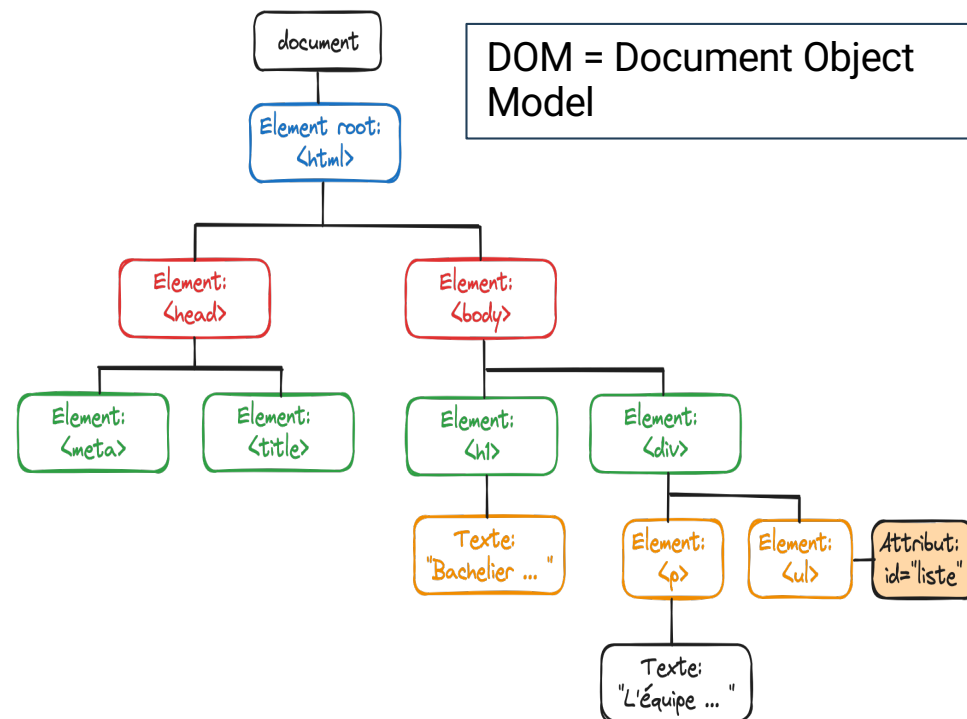
```
    <ul id="liste">
```

```
  </ul>
```

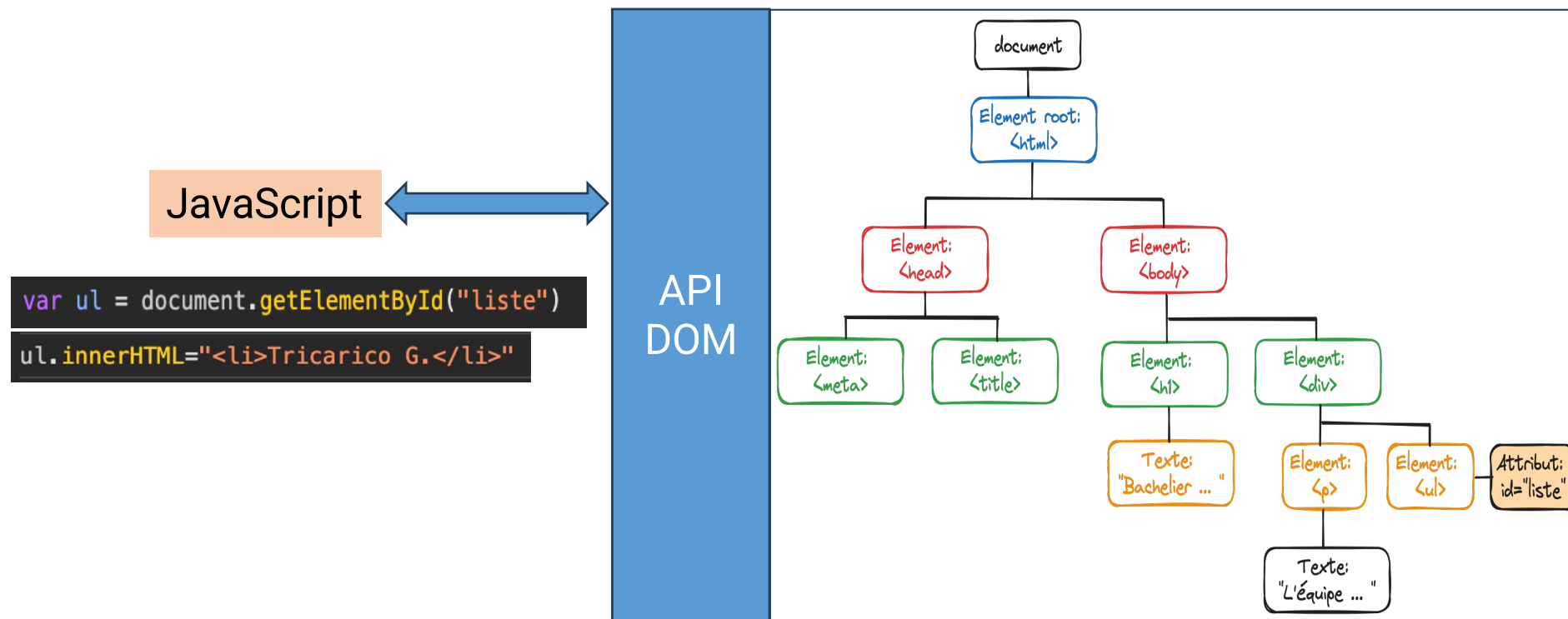
```
  </div>
```

```
</body>
```

```
</html>
```



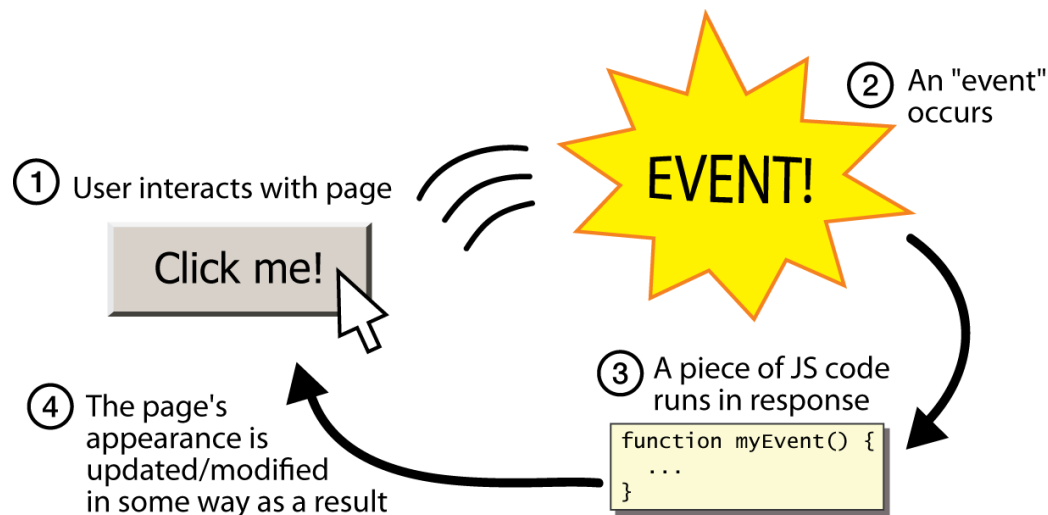
JavaScript – Structure dynamique (DOM)



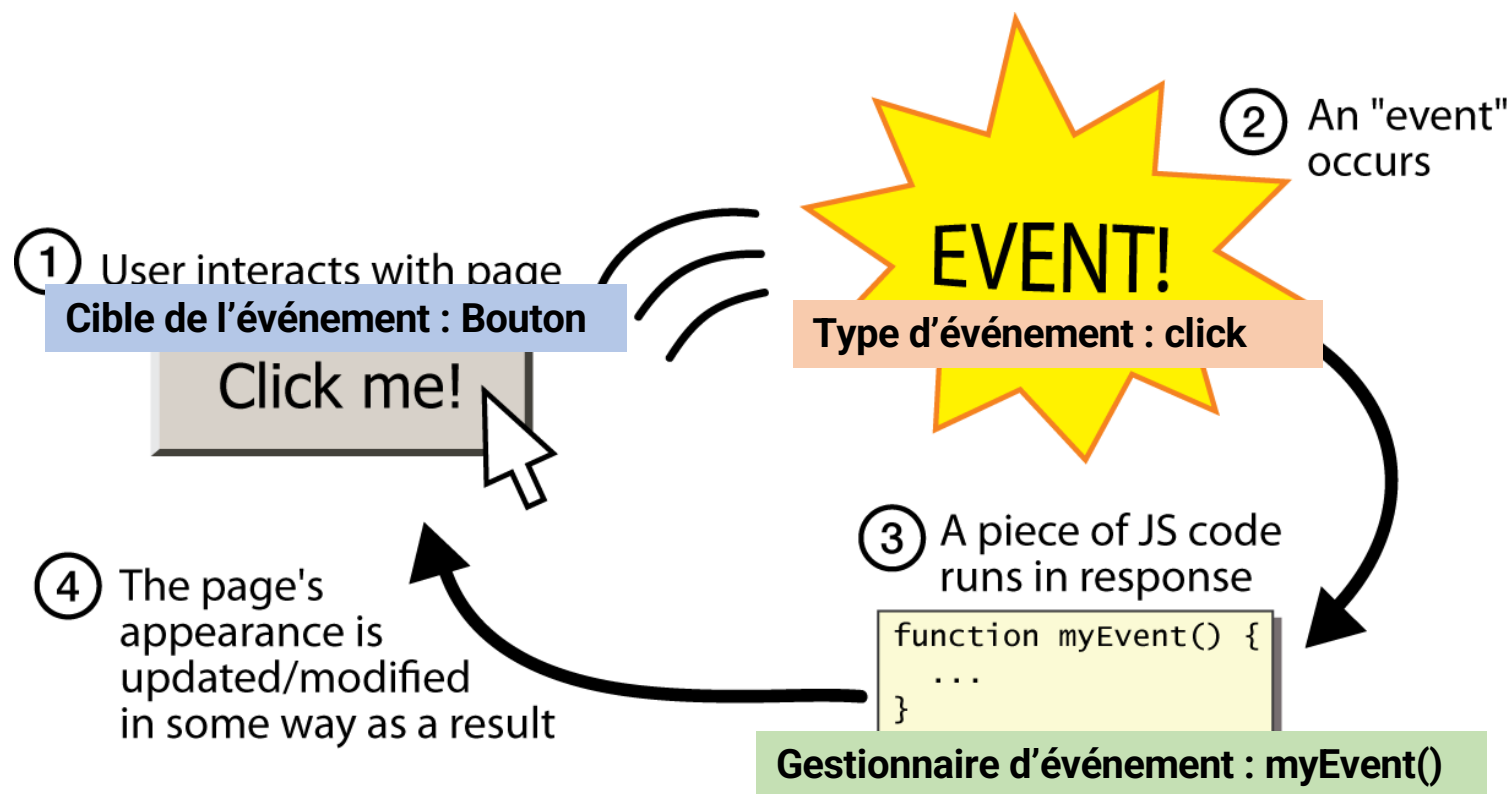
La méthode **getElementById()** de Document renvoie un objet Element représentant l'élément dont la propriété id correspond à la chaîne de caractères spécifiée.

La propriété **Element.innerHTML** de Element récupère ou définit la syntaxe HTML décrivant les descendants de l'élément.

- Toutes les applications dotées d'une interface utilisateur graphique peuvent enregistrer une ou plusieurs fonctions à invoquer lorsque des événements spécifiques se produisent
- Les événements peuvent se produire sur n'importe quel **élément** d'un document HTML.



- **Type d'événement (*event type*)**
C'est une **chaîne de caractères** qui indique le type d'événement (click, load, ...)
- **Cible de l'événement (*event target*)**
C'est **l'objet** sur lequel l'événement s'est produit ou avec lequel l'événement est associé (bouton, champ texte, ...)
- **Gestionnaire d'événement ou écouteur d'événement (event handler, or *event listener*)**
Cette **fonction** gère ou répond à un événement.
- **Objet événement (*event object*)**
Cet objet est associé à un événement particulier et contient des détails sur cet événement. Les objets d'événement sont transmis en tant qu'argument à la fonction de gestion d'événement.



Il existe trois méthodes de base pour enregistrer des gestionnaires d'événements.

1. Spécifier les événements avec HTML

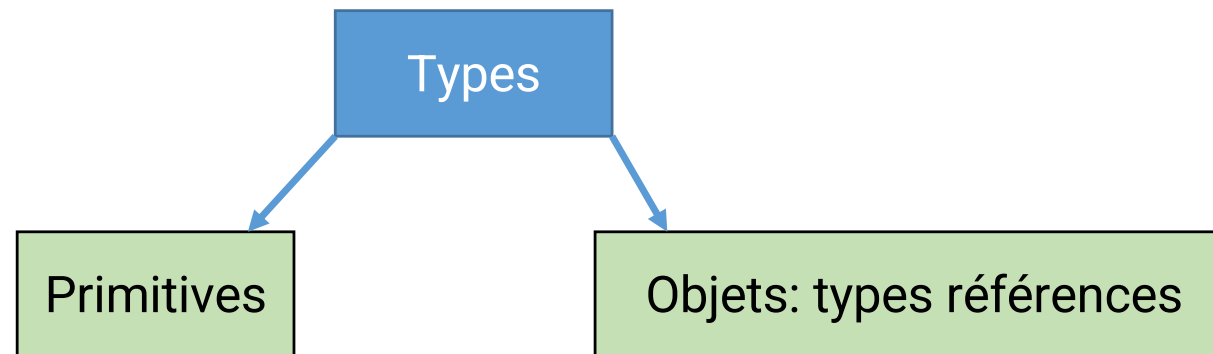
```
<button id=« mybutton » onclick=«handleClick() »>Click me</button>
```

2. Spécifier les événements avec JavaScript

```
<script>  
  let b = document.getElementById("mybutton");  
  b.onclick = function() { console.log("Thanks for clicking me!"); };  
</script>
```

3. Spécifier les événements avec « Event Listener » => Basé sur le pattern observer

```
b.addEventListener("click", function() { console.log("Thanks again!"); });
```



- boolean
- number
- string
- null
- undefined

- functions
- arrays
- class
- object

Elles se composent du mot-clé ***function***, suivi des éléments suivants :

- Un **identifiant** qui nomme la fonction. **Obligatoire** => **le nom de la fonction devient une variable dont la valeur est la fonction elle-même.**
- Une **paire de parenthèses** autour d'une liste de zéro ou plusieurs **paramètres**. => ils se comportent comme des variables locales dans le corps de la fonction.
- Une **paire d'accolades** contenant les instructions JavaScript. Ces instructions constituent le **corps de la fonction** : elles sont exécutées chaque fois que la fonction est invoquée.

Les déclarations de fonctions sont évaluées avant que le reste du code ne soit interprété.

Remarquez la différence entre *paramètres* et *arguments* :

- Les paramètres d'une fonction sont les noms listés dans la définition de la fonction.
- Les arguments d'une fonction sont les valeurs réelles passées à la fonction.
- Les paramètres sont initialisés avec les valeurs des arguments fournis.

The diagram shows a code snippet on a dark background. The function definition is `function sum(param1, param2){` followed by `return param1 + param2;` and a closing brace. An orange bracket labeled "Parameters" points to `param1` and `param2`. Below the function, the function is called with `sum(5, 6);`. A blue bracket labeled "Arguments" points to the values `5` and `6`.

```
function sum(param1, param2){  
  return param1 + param2;  
}  
  
sum(5, 6);
```

<https://developer.mozilla.org/fr/docs/Glossary/Parameter>

```
let pt1 = {  
  x : 2,  
  y : 3  
}  
let pt2 = {  
  x : 4,  
  y : 5  
}  
  
function distanceEntre2Points(point1,point2){  
  let deltaX = point2.x - point1.x;  
  let deltaY = point2.y - point1.y;  
  return Math.sqrt(deltaX*deltaX + deltaY*deltaY);  
}  
  
//Invoquer la fonction => mettre les parenthèses  
let resultat = distanceEntre2Points(pt1,pt2);  
console.log('distanceEntre2Points(pt1,pt2) -> ' + resultat);  
//Affecter la référence de la fonction "distanceEntre2Points" à la variable "distance"  
let distance = distanceEntre2Points;  
console.log('distance(pt1,pt2) -> ' + distance(pt1,pt2));
```

Les **expressions de fonction** ressemblent beaucoup aux déclarations de fonction, mais elles apparaissent dans le contexte d'une expression et leur nom est facultatif.

```
const carre = function carreNombre (nombre){  
  return nombre*nombre;  
}  
  
console.log('Fonction nommée -> ' + carre(2));  
  
// le nom de la fonction n'est pas utile  
// => on peut le supprimer => fonction anonyme  
  
const carreAnonyme = function (nombre){  
  return nombre*nombre;  
}  
  
console.log('Fonction anonyme -> ' + carreAnonyme(2));
```

Bonne pratique : Utiliser **const** avec les **expressions de fonction** afin de ne pas écraser accidentellement vos fonctions en leur assignant de nouvelles valeurs.

La plupart des fonctions définies comme des **expressions n'ont pas besoin de noms**, ce qui rend leur définition plus compacte.

```
//Le nom de la fonction est utile pour les appels récursifs  
const facto = function fact(x) {  
  if (x <= 1) return 1;  
  else return x*fact(x-1); };  
  
console.log('La factorielle de 5 -> ' + facto(5));
```

La **déclaration d'une fonction** peut être faite après l'appel :

```
console.log(carre(5));  
/* ... */  
function carre(n) { return n*n }
```

Déclaration de fonction

≠

≠

```
console.log(carre2);  
// ReferenceError: carré2 is not defined  
console.log(carre2(5));  
// TypeError: carre2 is not a function  
  
let carre2 = function (n) {  
  return n * n;  
}
```

Expression de fonction

En **ES6**, vous pouvez définir des fonctions à l'aide d'une **syntaxe** particulièrement **compacte** appelée "**fonctions fléchées** ».

```
const somme = function (nbre1,nbre2){  
  return nbre1+nbre2;  
}  
  
//Fonction fléchée avec deux paramètres  
const sommeFl = (nbre1,nbre2) => {return nbre1+nbre2;}  
  
// Fonction fléchée simplifiée  
const sommeFl2 = (nbre1,nbre2) => nbre1+nbre2;  
  
console.log(somme(5,2));  
console.log(sommeFl(5,2));  
console.log(sommeFl2(5,2));
```

```
const carre = function(nbre) {  
  return nbre*nbre;  
}  
  
//fonction fléchée avec un paramètre  
const carreFl = nbre => nbre*nbre;  
  
console.log(carreFl(2));
```

```
const afficheBonjour = function() {  
  return console.log('Bonjour')  
}  
//fonction fléchée sans paramètre  
const afficheBonjourFl = () => console.log('Bonjour');  
  
afficheBonjour();  
afficheBonjourFl();
```

La syntaxe concise des fonctions fléchées est lorsque vous avez besoin de passer une fonction à une autre, ce qui est courant avec des méthodes de tableau comme *map()*, *filter()* et *reduce()*.

Partie 2: Introduction à React



Introduction - React



Lorsque vous utilisez une **bibliothèque**, vous importez son code et appelez ses fonctions.

Cette bibliothèque **JavaScript** (frontend) va permettre :

- De construire des interfaces utilisateurs **réactives (React)**
- Des composants réutilisables

1. **Interface réactive** (d'où le nom React)
=> **DOM virtuel** (cf *reactSansJSX_DOMVirtuel.zip*)
2. **Composants**
=> Éléments de décomposition d'une page web => blocs de construction.
3. **Déclarative** (la philosophie de React étant que vous déclarez votre ou vos états finaux souhaités et que vous laissez React déterminer comment y parvenir)
=> **JSX**

React - DOM virtuel

Voir le code : DOMVirtuel_JavaScript_avecEtSansReact.zip

index.html

```
<body>
  <div id="Conteneur1"></div>
  <div id="Conteneur2"></div>
  <script
    src="https://unpkg.com/react@16/umd/react.development.js"
    crossorigin
  ></script>
  <script
    src="https://unpkg.com/react-dom@16/umd/react-dom.development.js"
    crossorigin
  ></script>

  <!-- Chargement du JavaScript -->
  <script src="script.js"></script>
```

Importe la bibliothèque React

Importe la bibliothèque ReactDOM

React - DOM virtuel

```
const e = React.createElement;
const domContainer1 = document.getElementById("Conteneur1");
const domContainer2 = document.getElementById("#Conteneur2");

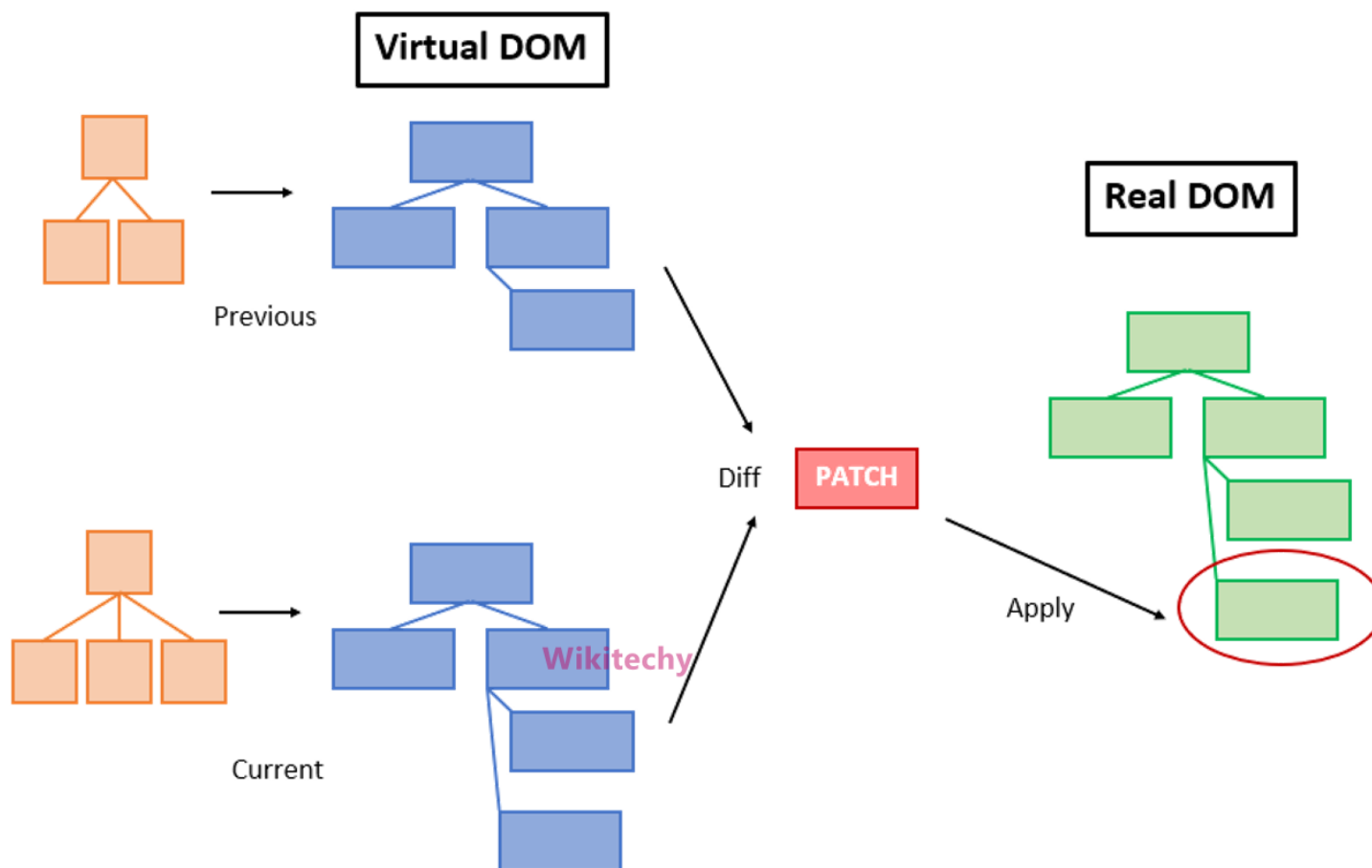
const render = () => {
  //Utilisation de Javascript sans React
  domContainer1.innerHTML = `
    <div>
      Hello HTML
      <input />
      <pre>${new Date().toLocaleString()}</pre>
    </div>
  `;

  -----

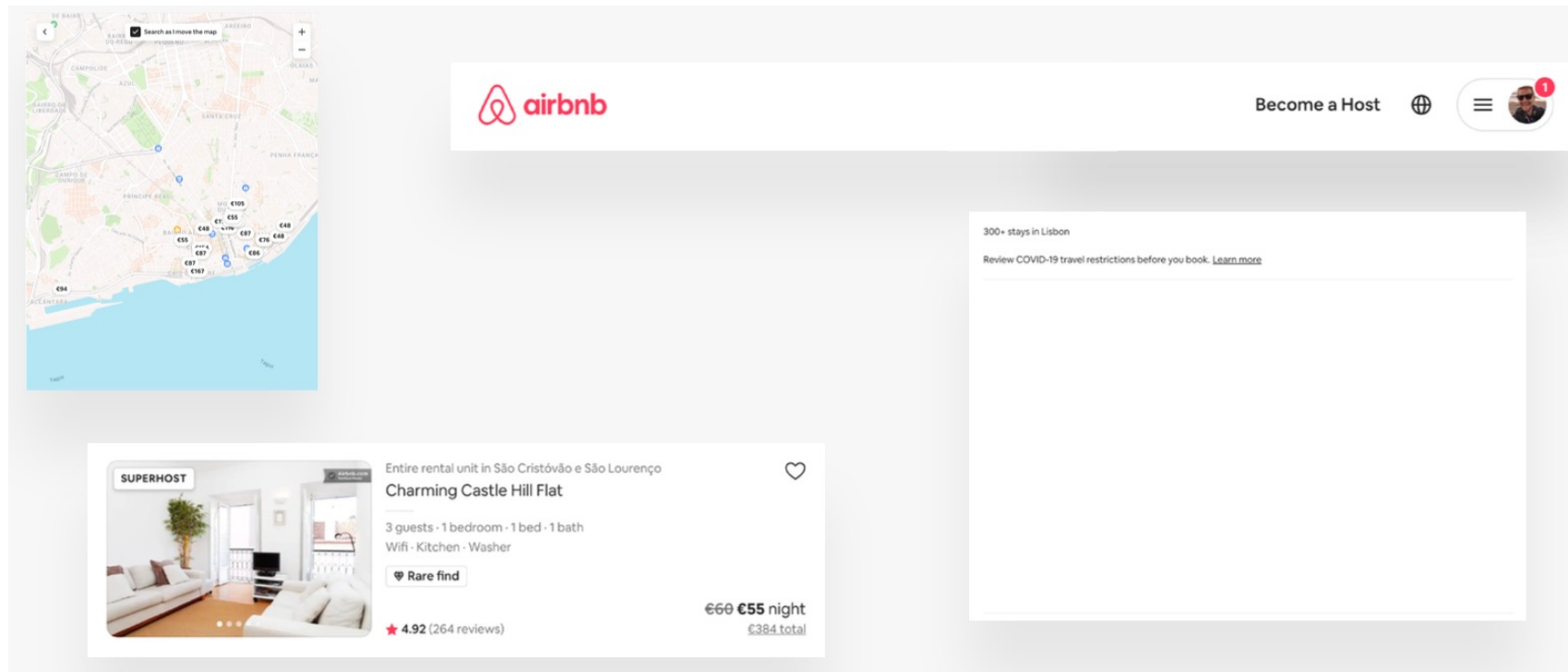
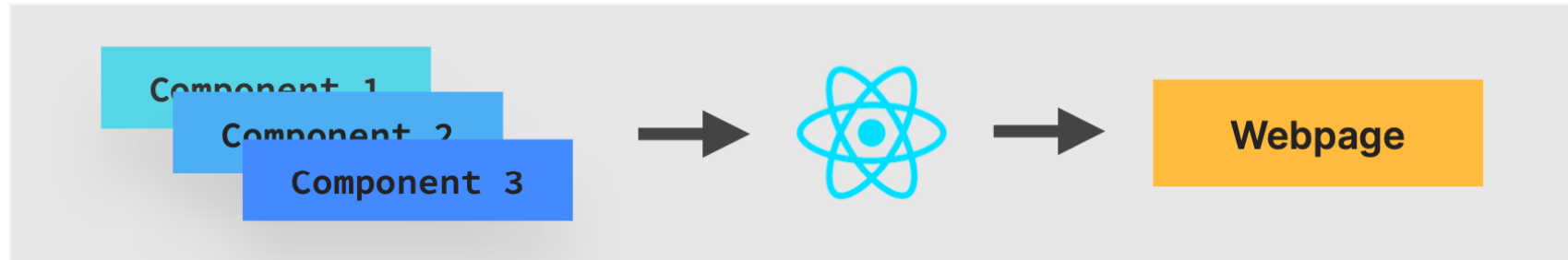
  //Utilisation de JavaScript avec React
  const elementsReact = React.createElement(
    "div",
    null,
    "Hello React ",
    React.createElement("input", null),
    React.createElement("pre", null, new Date().toLocaleString(
  );
  ReactDOM.render(elementsReact, Conteneur2);
};

setInterval(render, 1000);
```

React - DOM virtuel



React – Composants



React – Composants


NavBar

Lisbon
Oct 9 – 16, 2023
Add guests

Search

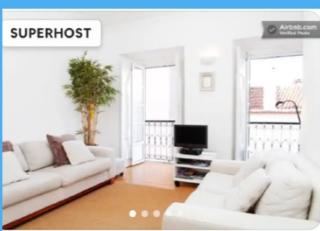
Become a Host




Price
Type of place
Free cancellation
Beachfront
Wifi
Kitchen
Washer
Pool
Self check-in
Dedicated workspace
Free parking
Iron
Filters

300+ stays in Lisbon
Review COVID-19 travel restrictions before you book. [Learn more](#)

Results



SUPERHOST

Entire rental unit in São Cristóvão e São Lourenço

Charming Castle Hill Flat

3 guests · 1 bedroom · 1 bed · 1 bath

Wifi · Kitchen · Washer

 Rare find

★ 4.92 (264 reviews)

€60 ~~€55~~ night

€384 total

Listing



SUPERHOST

Entire rental unit in São Vicente de Fora

 **Feel Alfama at BCA | Cascao 18**

2 guests · 1 bedroom · 1 bed · 1 bath

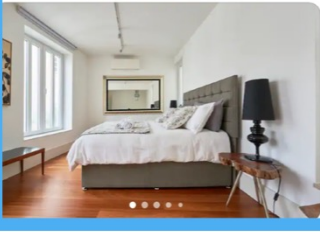
Wifi · Kitchen · Washer · Air conditioning

★ 4.92 (252 reviews)

€48 night

€333 total

Listing



Entire rental unit in Alfama

Duplex & Terrace with a river view

4 guests · 1 bedroom · 2 beds · 1.5 baths

Wifi · Kitchen · Washer · Air conditioning

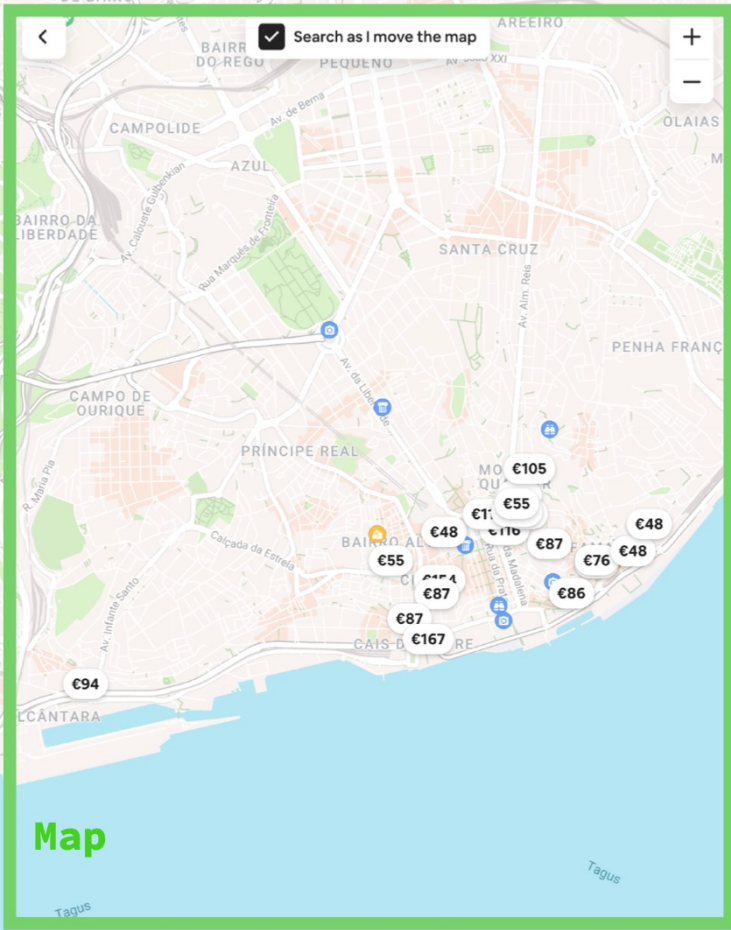
★ 4.98 (44 reviews)

€93 ~~€86~~ night

€597 total

Listing

☒ Search as I move the map



Map

React – JSX (déclarative)

JSX (JavaScript XML) : JSX est une extension de syntaxe pour JavaScript qui vous permet d'écrire du balisage **similaire** au **HTML** au sein d'un fichier **JavaScript**.

Nous décrivons l'aspect et le fonctionnement des composants à l'aide d'une syntaxe déclarative appelée JSX

```
1 const elementsReact = (  
2   <div>  
3     Hello React  
4     <input />  
5     <pre>{new Date().toLocaleString()}</pre>  
6   </div>  
7 );
```

JSX => syntaxe déclarative

JavaScript => syntaxe impérative

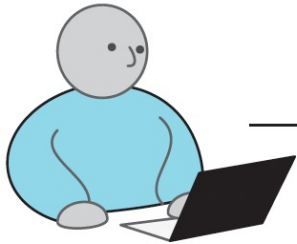
```
1 const elementsReact = /*#__PURE__*/ React.createElement(  
2   "div",  
3   null,  
4   "Hello React",  
5   /*#__PURE__*/ React.createElement("input", null),  
6   /*#__PURE__*/ React.createElement("pre", null, new Date().toLocaleString())  
7 );
```


React – JSX (déclarative)

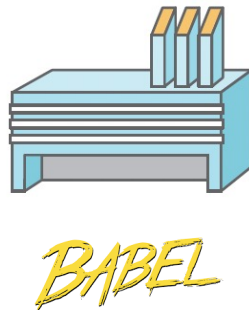
```
1 const elementsReact = (  
2   <div>  
3     Hello React  
4     <input />  
5     <pre>{new Date().toLocaleString()}</pre>  
6   </div>  
7 );
```

```
1 const elementsReact = /*#__PURE__*/ React.createElement(  
2   "div",  
3   null,  
4   "Hello React",  
5   /*#__PURE__*/ React.createElement("input", null),  
6   /*#__PURE__*/ React.createElement("pre", null, new Date().toLocaleString())  
7 );
```

1. JSX



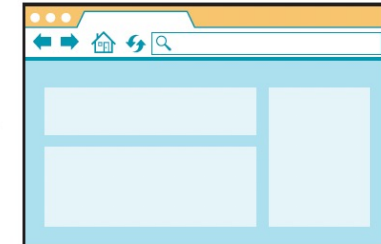
2. Transpiler



3. JavaScript

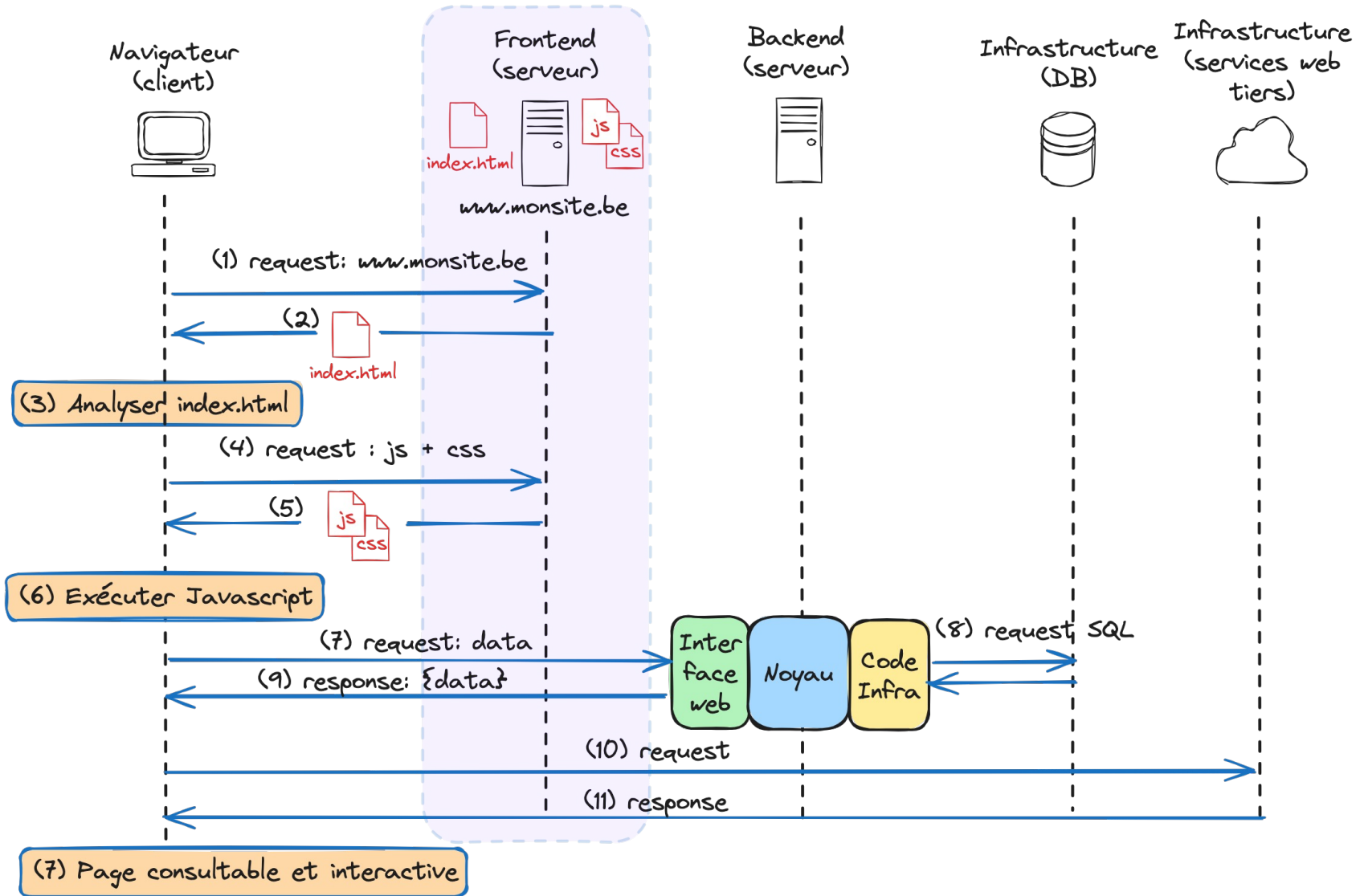


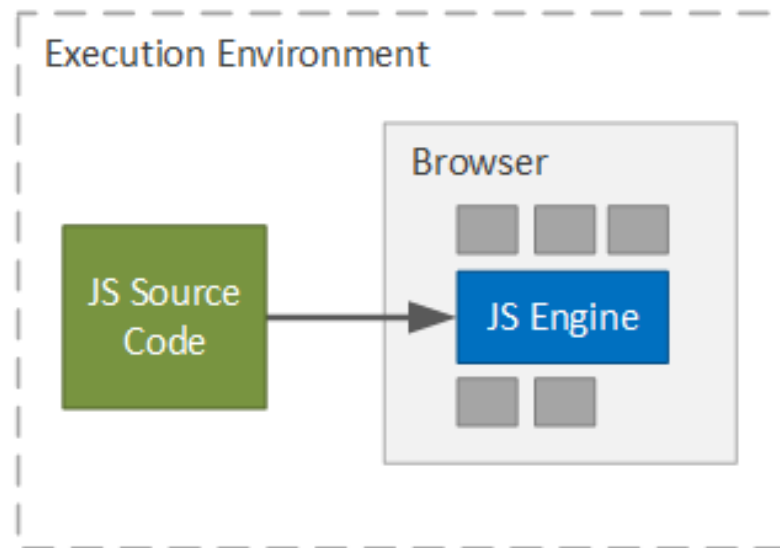
4. Browser



<https://babeljs.io/>

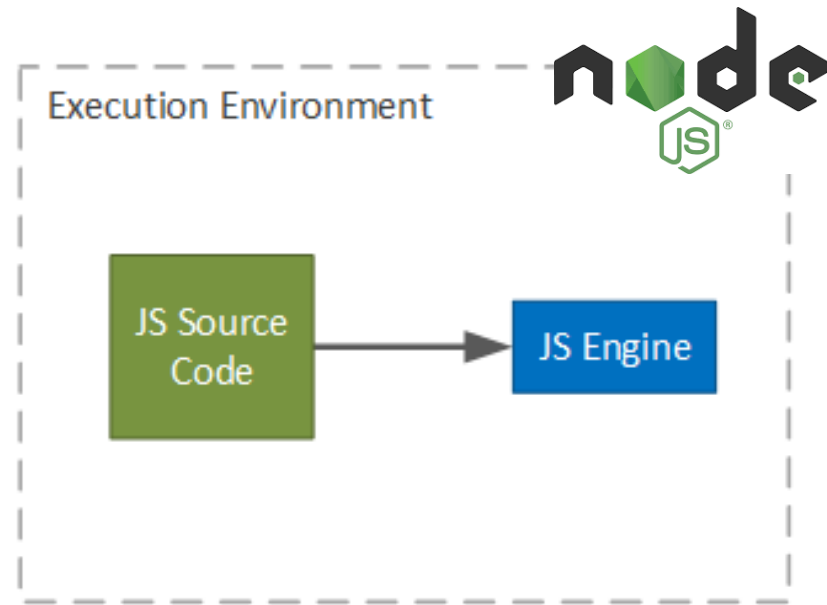
React – Single Page applications (SPA)





JAVASCRIPT IN A BROWSER

```
<script src=« ./app.js »></script>
```



STANDALONE JAVASCRIPT

```
$ node app.js
```



V8 est un **moteur JavaScript (JS) open-source** développé par Google.



« **Vite** est un **serveur de développement local** écrit par Evan You, le créateur de Vue.js, et utilisé par défaut par Vue et pour les modèles de projets React. Il prend en charge TypeScript et JSX. » (Wikipédia)

Vite nécessite l'installation de *Node.js* et *npm* (gestionnaire de paquets)

<https://nodejs.org/en>

```
node --version  
v21.6.1
```

```
npm --version  
10.2.4
```

Mettre à jour Node.js : <https://www.npmjs.com/package/n>

Introduction – Créer un projet React

```
npm create vite@latest ex1 -- --template react
Need to install the following packages:
create-vite@5.2.0
Ok to proceed? (y) y

Scaffolding project in /Users/giannitricarico/Library/Developer/
Done. Now run:

  cd ex1
  npm install
  npm run dev
```

npm run dev ➡ Lancer le serveur de développement

```
> ex1@0.0.0 dev
> vite

VITE v5.1.0 ready in 696 ms

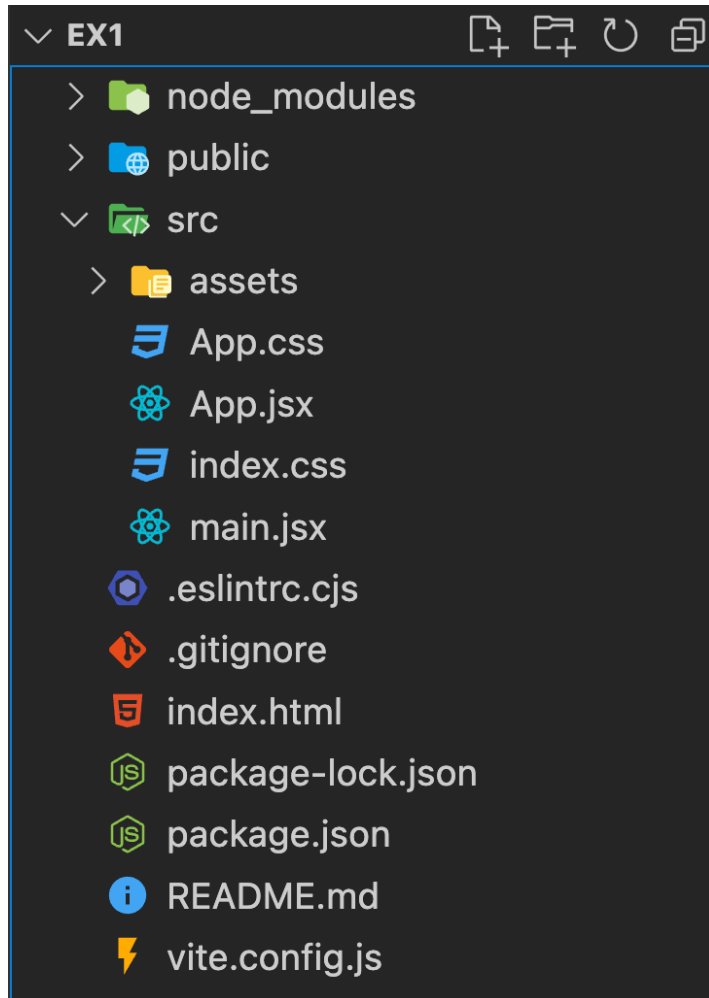
➔ Local:   http://localhost:5173/
➔ Network: use --host to expose
➔ press h + enter to show help
```

The supported template presets are:

JavaScript	TypeScript
vanilla	vanilla-ts
vue	vue-ts
react	react-ts
preact	preact-ts
lit	lit-ts
svelte	svelte-ts
solid	solid-ts
qwik	qwik-ts

<https://vitejs.dev/guide/>

Introduction – Créer un projet React



package.json : Ce fichier contient une liste de toutes les dépendances tierces.

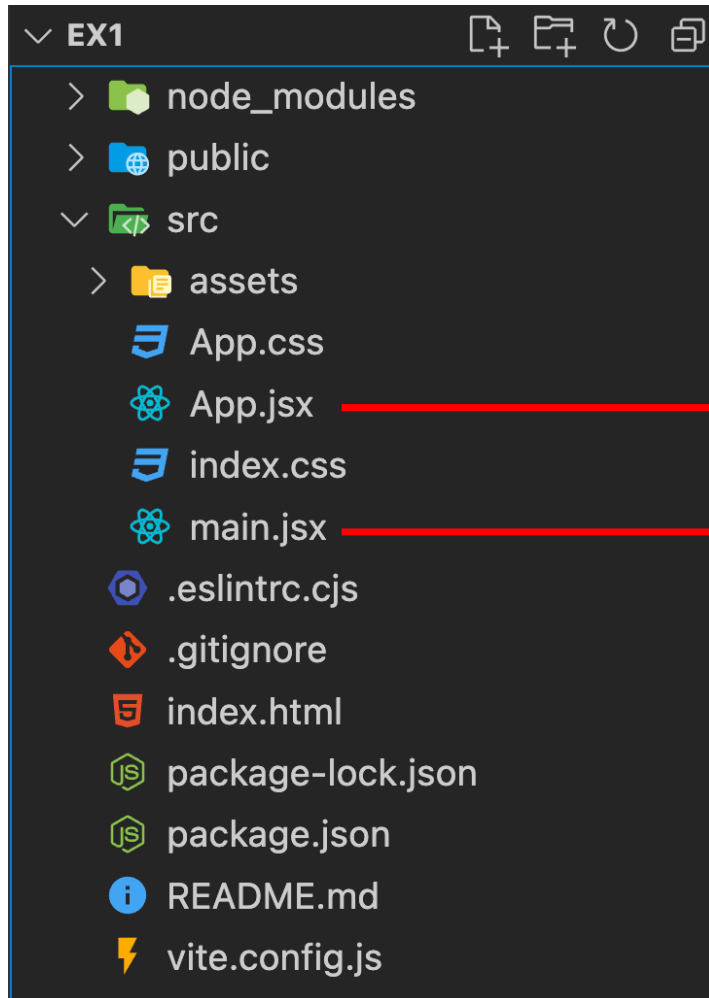
node_modules/ : Ce dossier contient tous les paquets node qui ont été installés. Comme nous avons utilisé Vite pour créer notre application React, plusieurs modules node (par exemple React) sont déjà installés pour nous.

vite.config.js : Un fichier pour configurer Vite.

public/ : Ce dossier contient des ressources statiques pour le projet, comme une favicon qui est utilisée pour la vignette de l'onglet du navigateur lors du démarrage du serveur de développement ou de la construction du projet pour la production.

index.html : Le HTML qui est affiché dans le navigateur au démarrage du projet.

Introduction – Créer un projet React



src/App.jsx qui est utilisé pour implémenter les composants React => **composant root**.

src/main.jsx est le point d'entrée d'une application React.

Toutes les commandes spécifiques à votre projet se trouvent dans votre fichier package.json

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "lint": "eslint . --ext js,jsx",  
  "preview": "vite preview"  
}
```

Ces scripts sont exécutés avec la commande : `npm run <script>`

#Exemples

Runs the application locally for the browser (start dev server)

`npm run dev`

Builds the application (production)

`npm run build`

locally preview production build

`npm run preview`

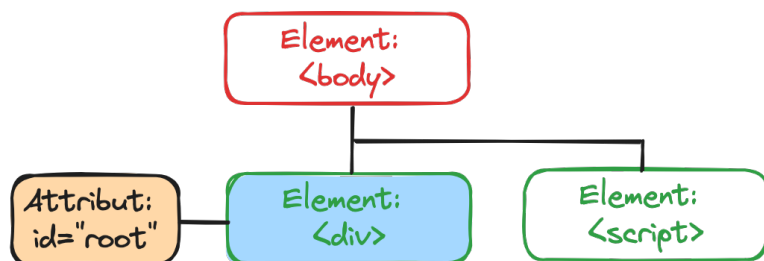
Introduction – Principe de React

index.html

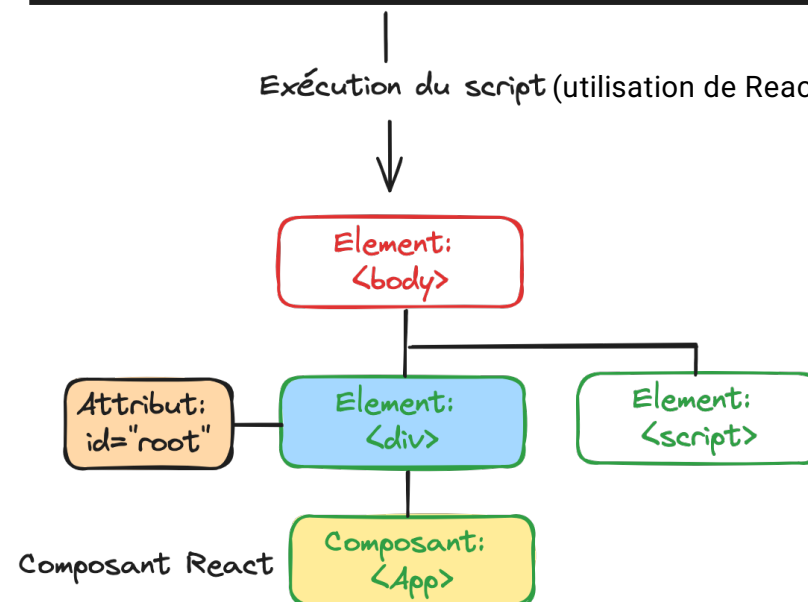
```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

main.jsx

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

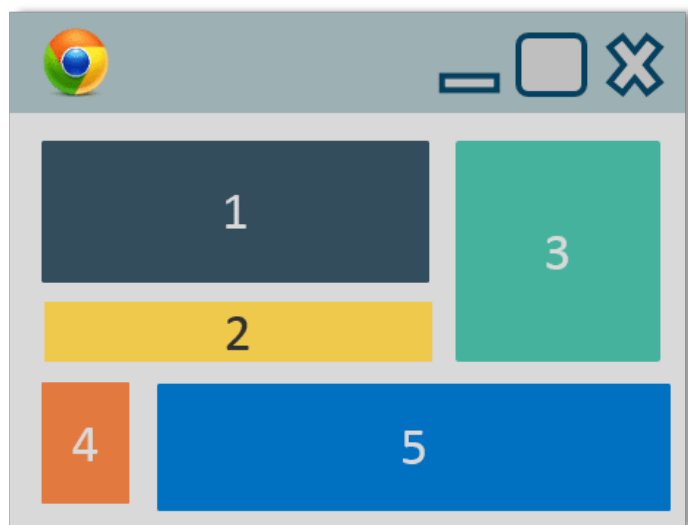


Exécution du script (utilisation de ReactDOM)

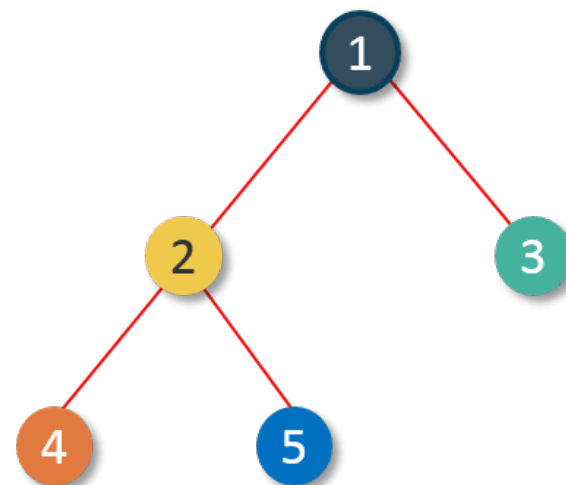


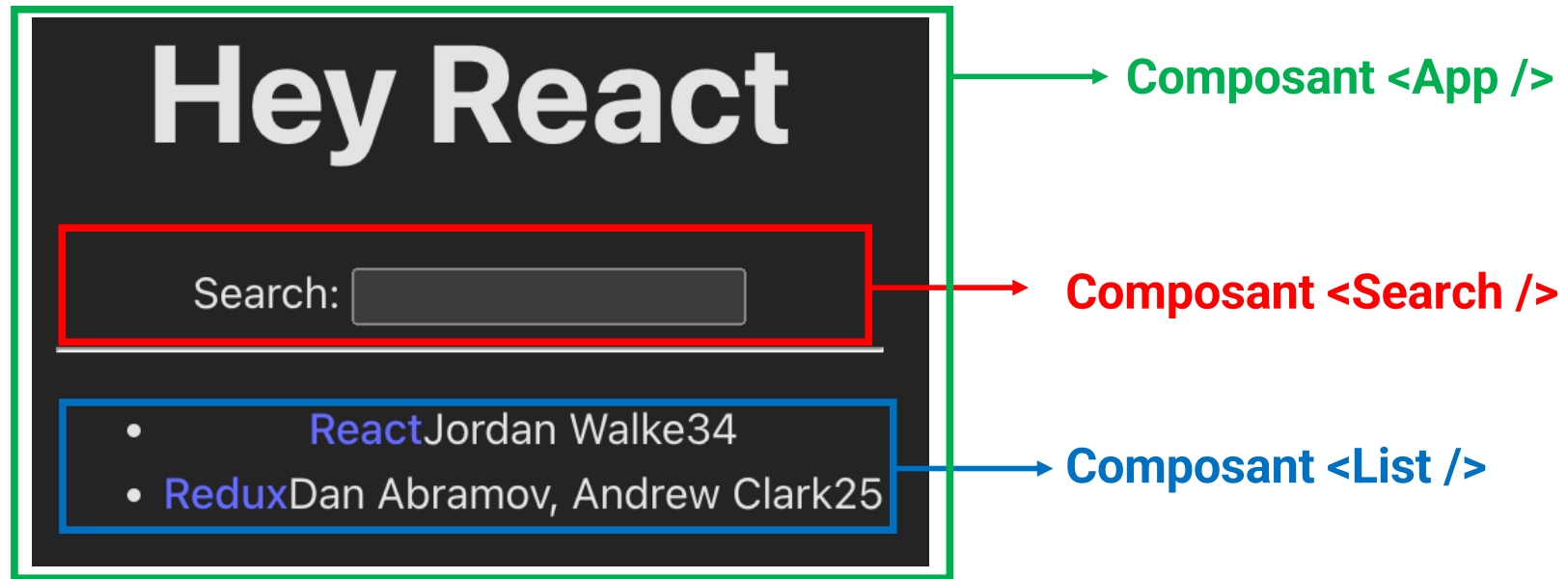
Partie 3: Composants et props

Browser



UI Tree





2 types de composant :

- Composants **intégrés** => <p>, <div>, ...
- Composant **personnalisé** en React => utilisé comme élément HTML dans du code JSX

Les composants personnalisés sont des fonctions JavaScript qui :

- **renvoient** une valeur pouvant être **rendue par React** (typiquement du code JSX)
- portent des noms en **majuscules** (sauf les **composants intégrés** de React comme <p>, <div>,...)
- sont **exportées** (via *export default*)

```
1  import "./App.css";
2
3  function App() {
4      return (
5          <div>
6              <h1>Hello React</h1>
7          </div>
8      );
9  }
10
11 export default App;
```

Composant de fonction « App »

Retourne JSX => Rendu par le navigateur

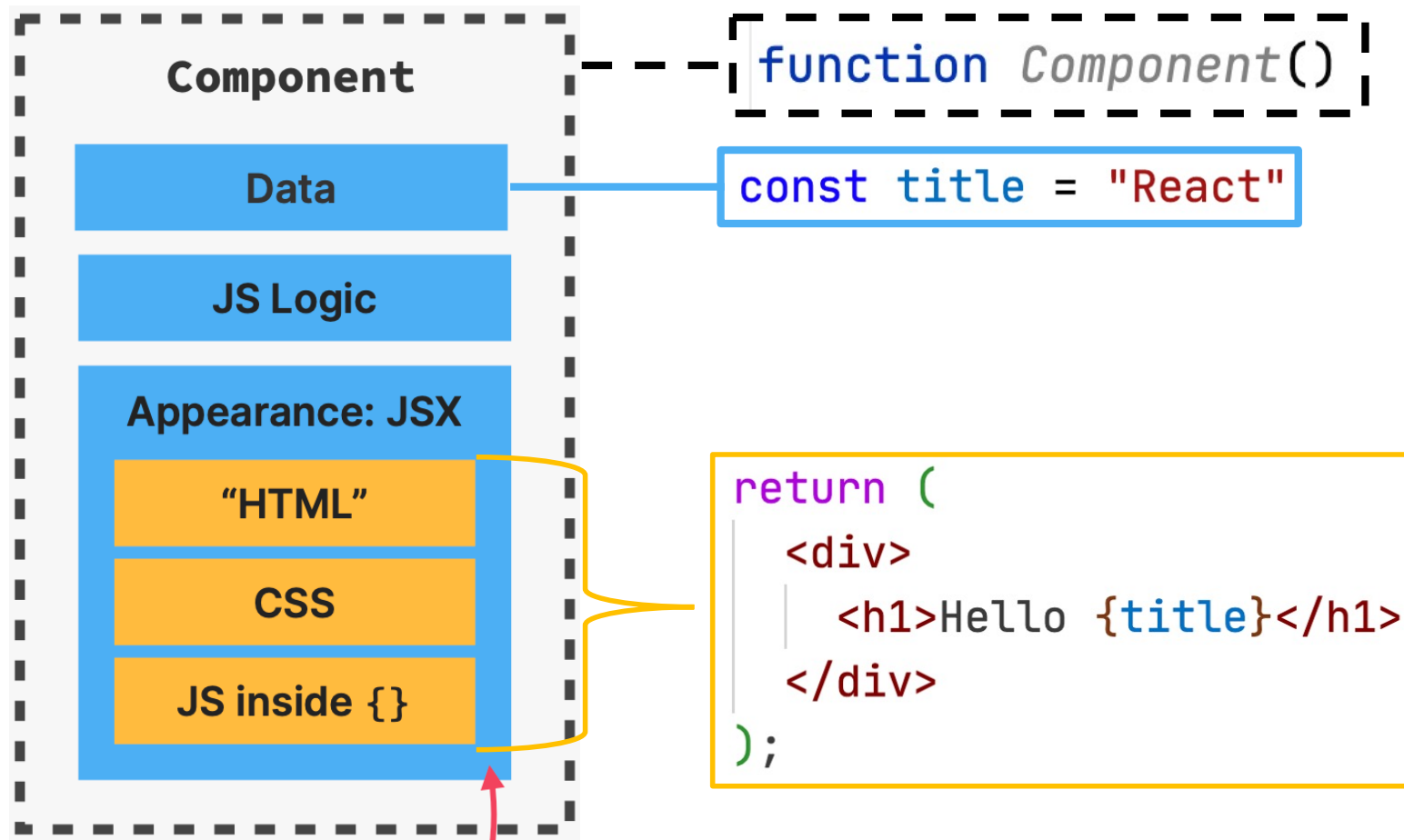
Exporter la fonction (module)

Un **composant** doit commencer par une **majuscule**, sinon il n'est pas traité comme un composant dans React.

Contenu dynamique

```
3  function App() {  
4  |    const title = "React"  
5  |    return (  
6  |        <div>  
7  | |    <h1>Hello {title}</h1>  
8  |    </div>  
9  |    );  
10 | }
```

1 composant = 1 fonction

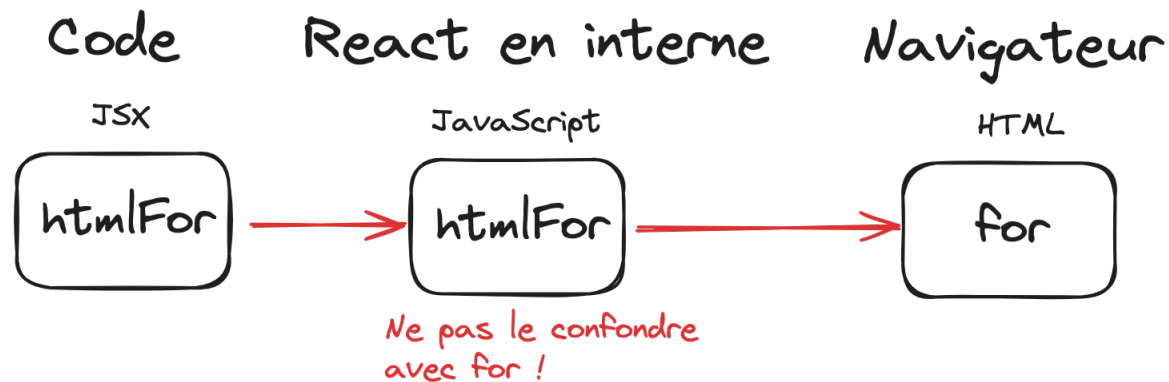


```

3  function App() {
4      const title = "React"
5      return (
6          <div>
7              <h1>Hello {title}</h1>
8              <label htmlFor="search">Search: </label>
9              <input id="search" type="text" />
10         </div>
11     );
12 }

```

JSX !!!!



React utilise la convention de dénomination **camelCase** pour les attributs.
Exemple : **className** au lieu de **class** et **onClick** au lieu de **onclick**

Composant `<App />`

```
import './App.css'

function App() { Show usages
  const welcome : {greeting: string, title: string} = {
    greeting: 'Hey',
    title: 'React'
  };
  return (
    <h1> {welcome.greeting} {welcome.title}</h1>
  )
}

export default App Show usages
```

Hey React

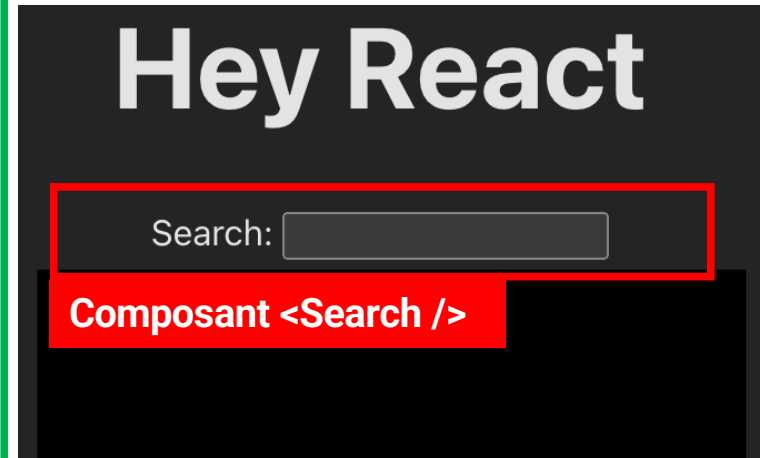
```
function App() { Show usages
  const welcome : {greeting: string, title: string} = {
    greeting: 'Hey',
    title: 'React'
  };
  return (
    <div>
      <h1> {welcome.greeting} {welcome.title}</h1>
      <Search/> ← Utilisation du composant « Search »
    </div>
  )
}

const Search = () => { ← Définition du composant « Search »
  return (
    <div>
      <label htmlFor="search">Search: </label>
      <input id="search" type="text"/>
    </div>
  );
};

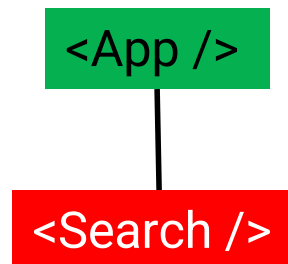
export default App Show usages
```

1 composant = 1 fonction

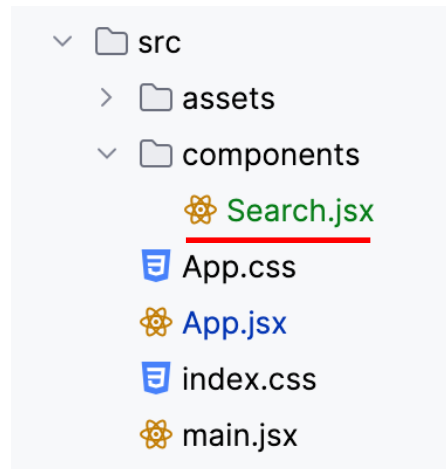
Composant <App />



Arbre de rendu



Fichier : App.jsx



```
1  const Search = () => { Show usages new *  
2      return (  
3          <div>  
4              <label htmlFor="search">Search: </label>  
5              <input id="search" type="text"/>  
6          </div>  
7      );  
8  };  
9  export default Search; Show usages new *
```

1 composant = 1 fonction = 1 fichier

src
 assets
 components
 Search.jsx
 App.css
 App.jsx
 index.css
 main.jsx

```
1 import './App.css'
2 import Search from './components/Search'
3 function App() { Show usages ⓘ HeH-Gianni
4   const welcome : {greeting: string, title: string} = {
5     greeting: 'Hey',
6     title: 'React'
7   };
8   return (
9     <div>
10       <h1> {welcome.greeting} {welcome.title}</h1>
11       <Search/>
12     </div>
13   )
14 }
15 export default App Show usages ⓘ HeH-Gianni
```

Données JavaScript

Object JavaScript

```
1  const book = {  
2      title: 'React',  
3      url: 'https://reactjs.org/',  
4      author: 'Jordan Walke',  
5      num_comments: 3,  
6      points: 4,  
7      objectID: 0  
8  }
```

Tableau

```
19  const numbers = [1, 2, 3, 4];  
20  const shopping = ["bread", "milk", "cheese", "hummus", "noodles"];
```

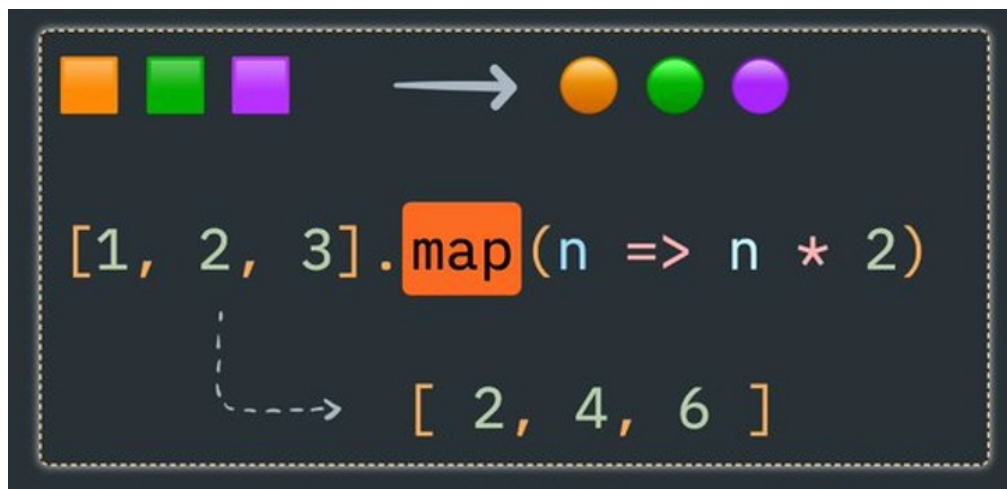
Tableau d'objets JavaScript

```
1  ∨ const books = [  
2  ∨  {  
3      title: "React",  
4      url: "https://reactjs.org/",  
5      author: "Jordan Walke",  
6      num_comments: 3,  
7      points: 4,  
8      objectID: 0,  
9  },  
10 ∨  {  
11     title: "Redux",  
12     url: "https://redux.js.org/",  
13     author: "Dan Abramov, Andrew Clark",  
14     num_comments: 2,  
15     points: 5,  
16     objectID: 1,  
17  },  
18 ];
```

La méthode ***map()*** (« carte » en français) retourne un **nouveau tableau** composé des éléments du tableau initial auxquels une fonction est appliquée.

```
const fruits = ["pomme", "banane", "poire", "kiwi"];  
//const fruitsMaj = [];  
  
// for(let fruit of fruits){  
// fruitsMaj.push(fruit.toUpperCase());  
// }  
  
const fruitsMaj = fruits.map((fruit) => fruit.toUpperCase());  
  
console.log(fruitsMaj);  
// ["POMME", "BANANE", "POIRE", "KIWI"]
```

Synthèse de la méthode (fonction) *map()* :



tab.map(callback)

=> Une fonction de rappel (*callback*) est une fonction passée dans une autre fonction en tant qu'argument, qui est ensuite invoquée à l'intérieur de la fonction externe pour accomplir une sorte de routine ou d'action.

Liste d'éléments ()

```
<hr />
<ul>{books.map((item) => {
  return <li key={item.objectID}> {item.title} </li>
})}</ul>
</div>
```

Une « clé » (*key*), est un **attribut spécial** que vous devez inclure quand vous créez une liste d'éléments.

Les **key** (clés) aident React à **identifier quels éléments d'une liste ont changé, ont été ajoutés ou supprimés**. Vous devez donner une clé à chaque élément dans un tableau, afin d'apporter aux éléments une identité stable.

Vous pouvez aussi **utiliser l'id associée à votre donnée** dans votre base de données.

```
const List = () => { Show usages new *  
  return (  
    <ul>  
      {books.map((book : {...} ) => (  
        <li key={book.objectID}>  
          <span><a href={book.url}>{ book.title}</a></span>  
          <span> {book.author}</span>  
          <span> ({book.num_comments})</span>  
          <span> [{book.points}]</span>  
        </li>  
      ))}  
    </ul>  
  );  
}  
export default List; Show usages new *
```

Hey React

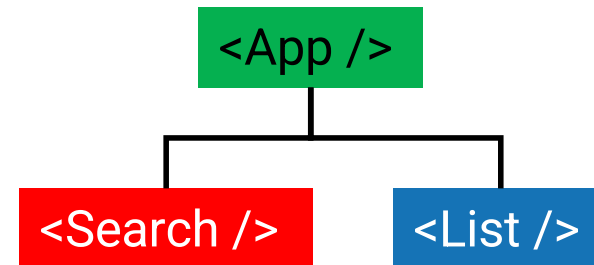
Search:

- [React](#) Jordan Walke (3) [4]
- [Redux](#) Dan Abramov, Andrew Clark (2) [5]

Composant <List />

```
import Search from './components/Search'
import List from './components/List.jsx';
function App() { Show usages ⓘ HeH-Gianni *
  const welcome : {greeting: string, title: string} = {
    greeting: 'Hey',
    title: 'React'
  };
  return (
    <>
      <h1> {welcome.greeting} {welcome.title}</h1>
      <Search/>
      <hr />
      <List />
    </>
  )
}
export default App Show usages ⓘ HeH-Gianni
```

Arbre de rendu



```
function App() {  
  return (  
    <p>Hello World!</p>  
    <p>Let's learn React!</p>  
  );  
};
```

Invalide => Une fonction ne peut que renvoyer une seule valeur !!

```
function App() {  
  return (  
    <React.Fragment>  
      <p>Hello World!</p>  
      <p>Let's learn React!</p>  
    </React.Fragment>  
  );  
}
```

Valide

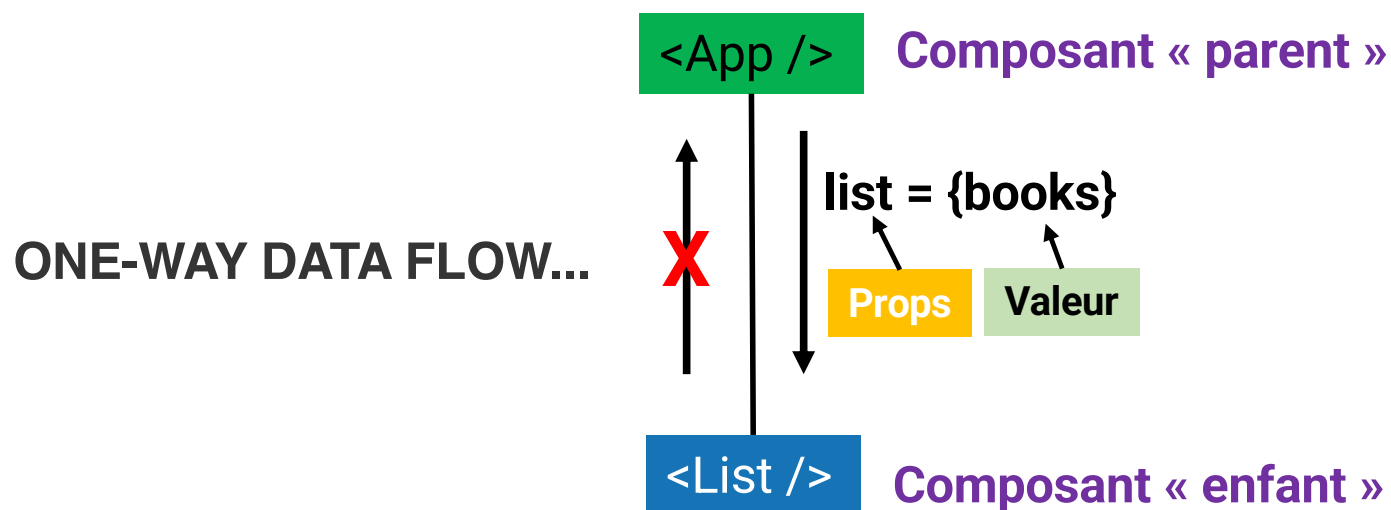
```
function App() {  
  return (  
    <div>  
      <p>Hello World!</p>  
      <p>Let's learn React!</p>  
    </div>  
  );  
}
```

Valide

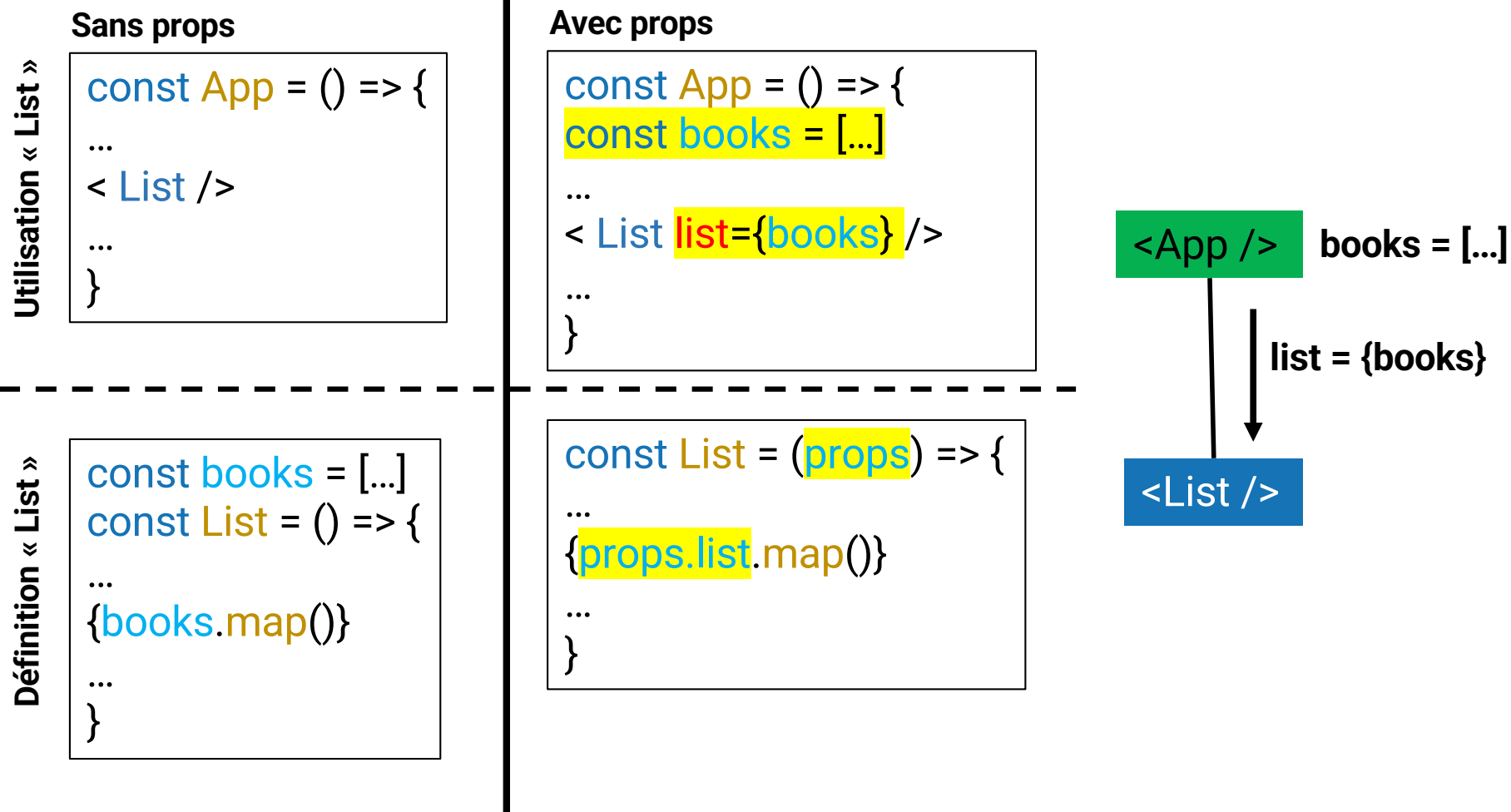
```
function App() {  
  return (  
    <>  
      <p>Hello World!</p>  
      <p>Let's learn React!</p>  
    </>  
  );  
}
```

Valide

Les **props (properties)** sont utilisés **pour transmettre des informations** au sein de la hiérarchie des composants (Parent -> Enfant).



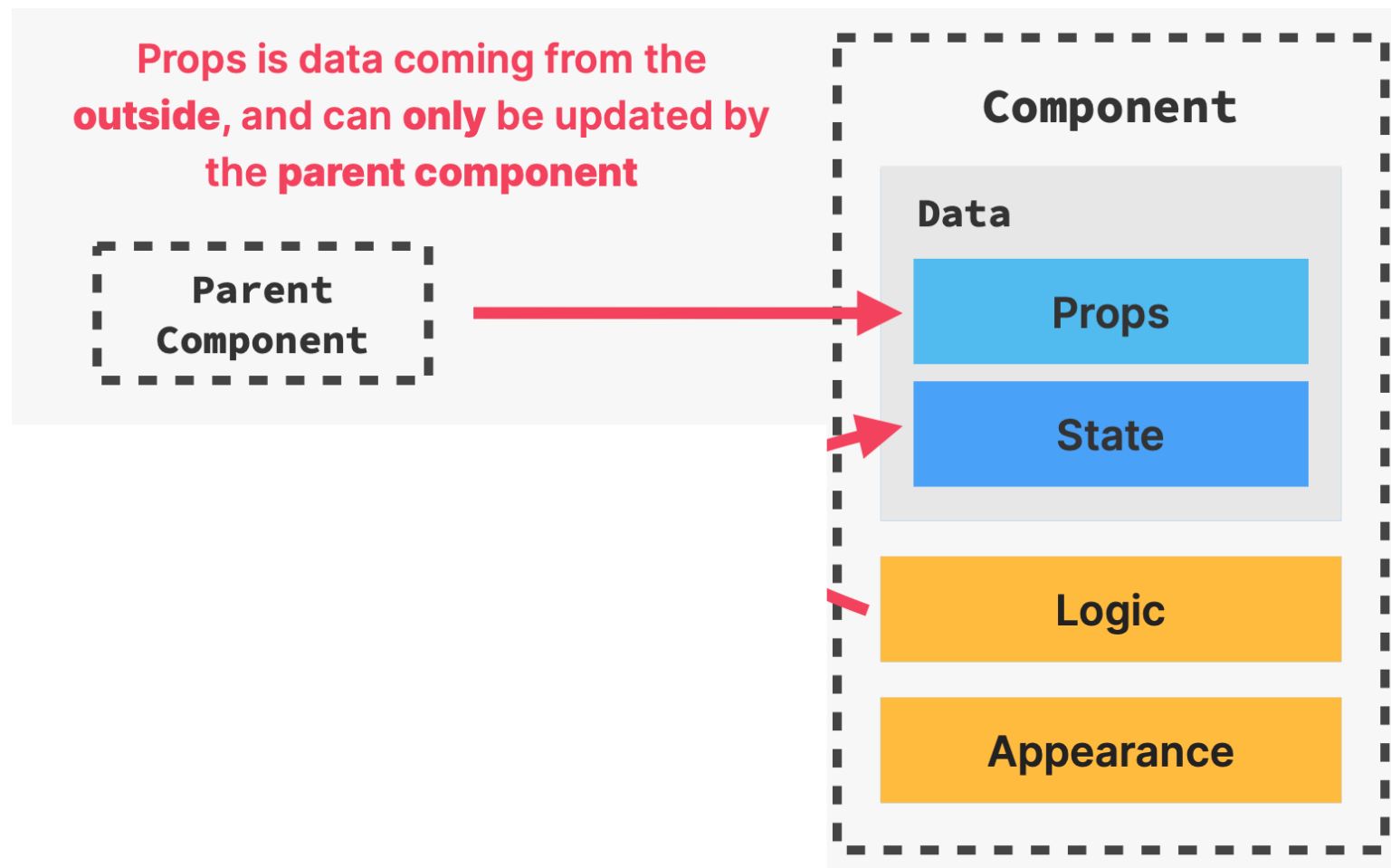
Les informations (props) ne peuvent être transmises que d'un composant parent à un composant enfant et non l'inverse.



```
function App() { Show usages ⓘ HeH-Gianni *  
  const welcome : {greeting: string, title: string} = {  
    greeting: 'Hey',  
    title: 'React'  
  };  
  const books : [{title: string, url: string, ...} = [ 2 elements... ];  
  return (  
    <>  
      <h1> {welcome.greeting} {welcome.title}</h1>  
      <Search/>  
      <hr />  
      <List list={books}/>  
    </>  
  )  
}  
export default App Show usages ⓘ HeH-Gianni
```

```
const List = (props) => { Show usages ⓘ HeH-Gianni *  
  return (  
    <ul>  
      {props.list.map((book) => (  
        <li key={book.objectID}>  
          <span><a href={book.url}>{ book.title}</a></span>  
          <span> {book.author}</span>  
          <span> ({book.num_comments})</span>  
          <span> [{book.points}]</span>  
        </li>  
      )  
    )  
  }  
    </ul>  
  );  
}  
export default List; Show usages ⓘ HeH-Gianni
```

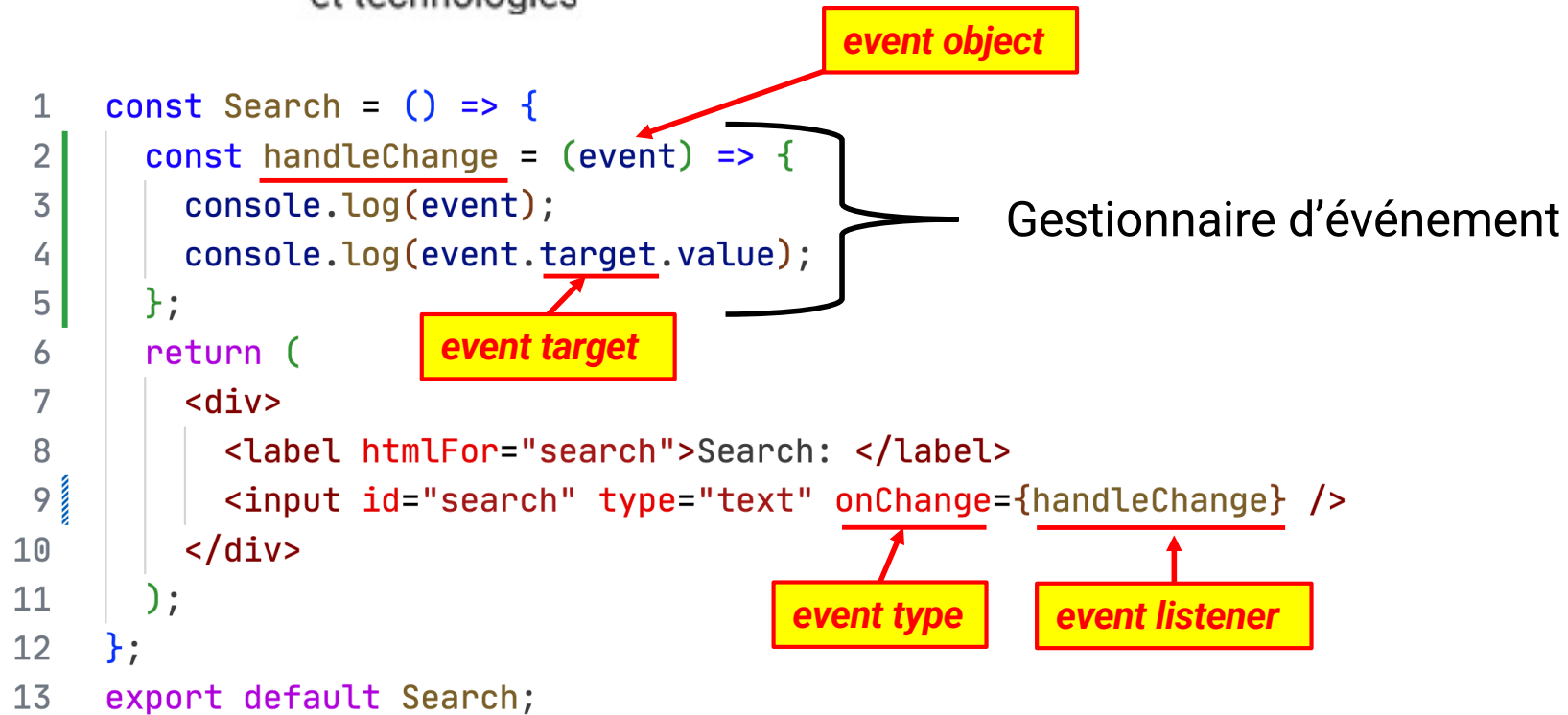

- Le nom du paramètre (props) est libre (par convention, props).
- Cet argument props automatiquement fourni contiendra toujours un objet (React passe un objet comme valeur pour cet argument)
- les propriétés de cet objet seront les "attributs" que vous avez ajoutés à votre composant dans le code JSX où le composant est utilisé.



Partie 4: Événements, state et affichage conditionnel

Rappel :

- **Objet événement (*event object*)**
Cet objet est associé à un événement particulier et contient des détails sur cet événement. Les objets d'événement sont transmis en tant qu'argument à la fonction de gestion d'événement.
- **Cible de l'événement (*event target*)**
C'est *l'objet* sur lequel l'événement s'est produit ou avec lequel l'événement est associé (bouton, champ texte, ...)
- **Gestionnaire d'événement ou écouteur d'événement (event handler, or *event listener*)**
Cette *fonction* gère ou répond à un événement.
- **Type d'événement (*event type*)**
C'est une *chaîne de caractères* qui indique le type d'événement (click, load, ...)



Remarque :

- `handleChange()` : **invoker** la fonction => récupérer la valeur de retour
- `handleChange` : retourner la **référence** de la fonction

```
1  const Search = () => {  
2    let searchTerm = '';  
3    const handleChange = (event) => {  
4      searchTerm = event.target.value;  
5    };  
6    return (  
7      <div>  
8        <label htmlFor="search">Search: </label>  
9        <input id="search" type="text" onChange={handleChange} />  
10  
11        <p>Recherche de <strong>{searchTerm}</strong></p>  
12      </div>  
13    );  
14  };  
15  export default Search;
```

Les valeurs saisies n'apparaissent pas, car le composant n'est pas réaffiché (re-render) !!!

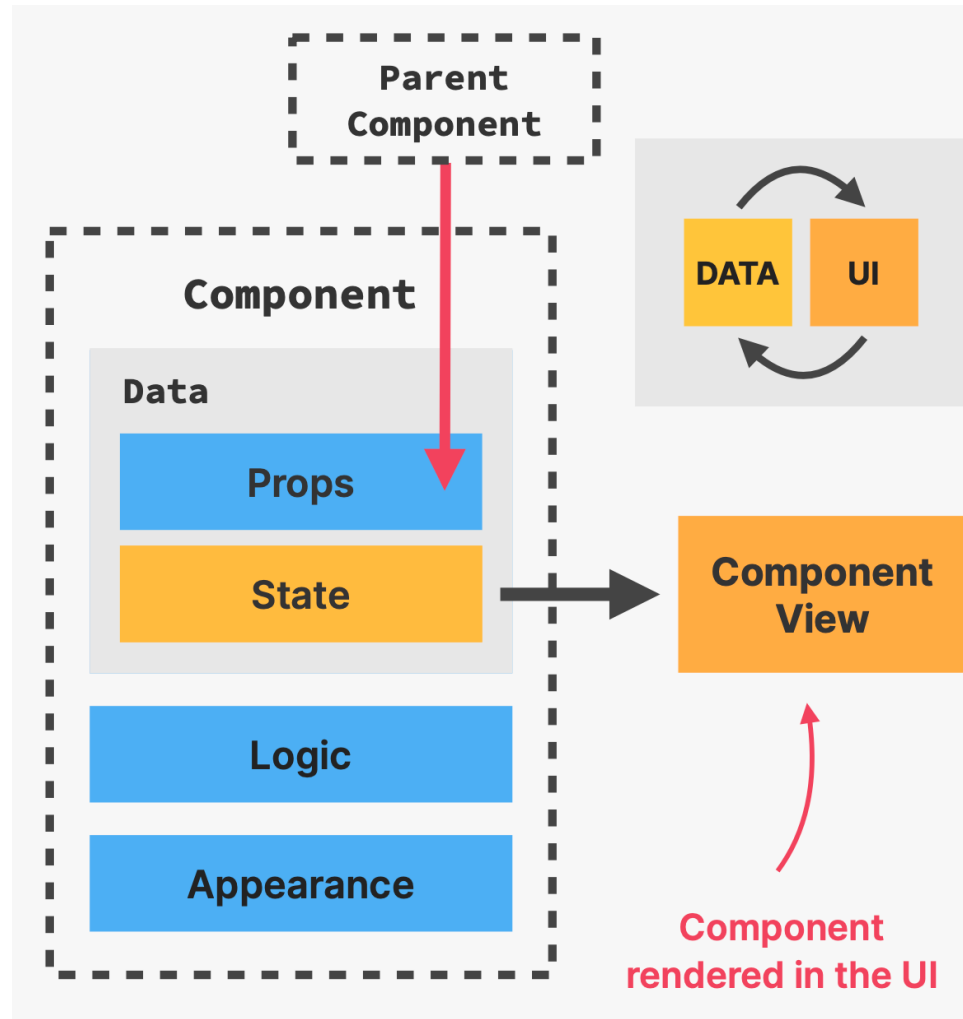
Pour que le composant se redessine lorsque la valeur de la variable « *searchTerm* » change => celle-ci doit être définie comme une **variable d'état**.

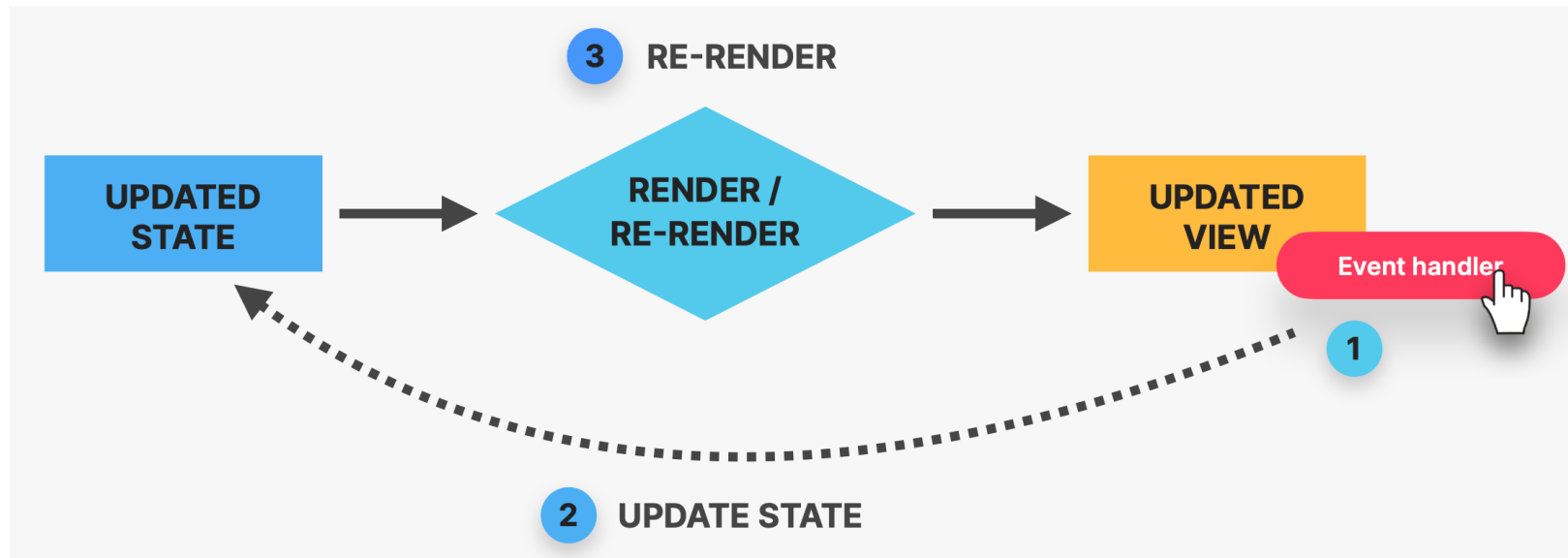
L'état est une donnée qui, lorsqu'elle est modifiée, doit **forcer React à réévaluer** un composant et à mettre à jour l'interface utilisateur si nécessaire.

Hooks :

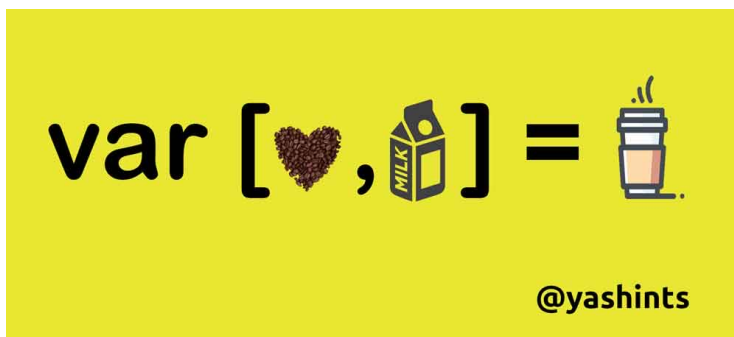
Ce sont des **fonctions spéciales** qui ne peuvent être utilisées qu'à **l'intérieur des composants** React.

Hook *useState()* permet à un composant de définir et de gérer un état lié à ce composant.





Affectation par décomposition (destructuring) - Tableaux



<https://yashints.dev/blog/2018/09/12/destructuring-in-js/>

```
const toto = ["un", "deux", "trois"];

// sans utiliser la décomposition
const un = toto[0];
const deux = toto[1];
const trois = toto[2];

// en utilisant la décomposition
const [un, deux, trois] = toto;

console.log(` ${un} et ${deux} et ${trois} `);
```

```
const [searchTerm, setSearchTerm] = useState("");
```

Affectation de décomposition

Valeur de l'état initiale

`useState("")` -> fonction qui retourne un tableau à 2 éléments.

Le **premier élément** du tableau (`searchTerm`) est toujours la **valeur** de l'état actuel.

Le **second élément** du tableau (`setSearchTerm`) est une **fonction** que vous pouvez appeler pour **mettre à jour l'état**. => ça permet d'informer React qu'un état a changé.

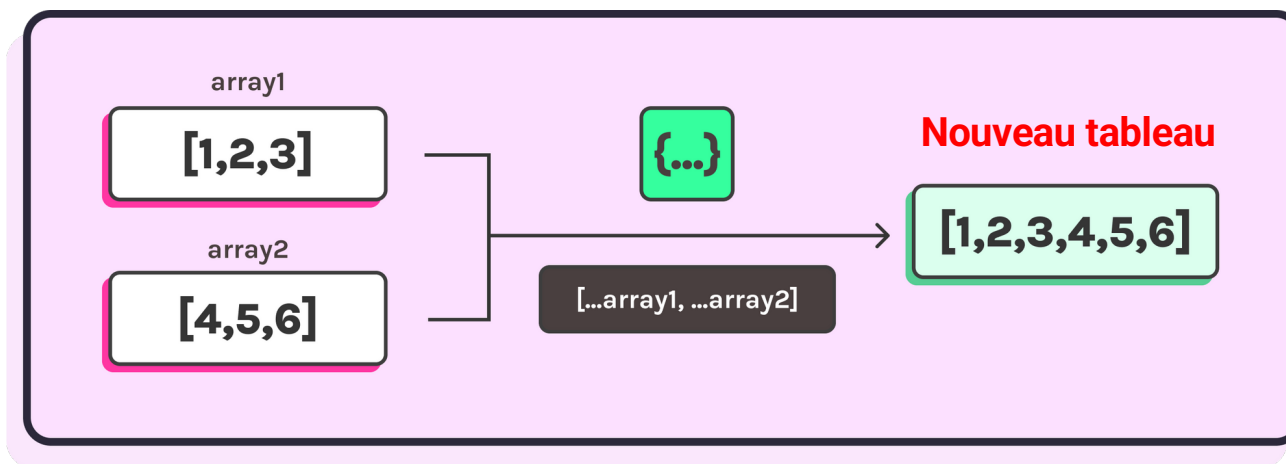
```
1 | import { useState } from 'react'; ← Importer la fonction « useState »
2 | const Search = () => {
3 |   const [searchTerm, setSearchTerm] = useState('');
4 |   const handleChange = (event) => {
5 |     setSearchTerm(event.target.value);
6 |   };
7 |   return (
8 |     <div>
9 |       <label htmlFor="search">Search: </label>
10 |      <input id="search" type="text" onChange={handleChange} />
11 |
12 |      <p>Recherche de <strong>{searchTerm}</strong></p>
13 |    </div>
14 |  );
15 | };
16 | export default Search;
```

```
const book : {...} = {
  title: "Pro Git",
  url: "https://git-scm.com/book/en/v2",
  author: "Scott Chacon and Ben Straub",
  num_comments: 2,
  points: 5,
  objectID: 3
};

const List = () => { Show usages  ⓘ HeH-Gianni *
  const [books_state : [{title: string, url: string, ...} , setbooks_state] = useState(books);
  const addBook = () : void => { Show usages  ⓘ new *
    books.push(book);
    setbooks_state(books);
  }
  return (
    <>
      <ul>
        {books_state.map((book : {...}) => (
          <li key={book.objectID}>
            <span><a href={book.url}>{book.title}</a></span>
            <span> {book.author}</span>
            <span> ({book.num_comments})</span>
            <span> [{book.points}]</span>
          </li>
        ))}
      </ul>
      <button onClick={addBook}>Add</button>
    </>
  );
};
```

← Pas de changement au niveau de l'affichage !!! => la référence elle-même n'a pas été mise à jour (le contenu a été mis à jour).

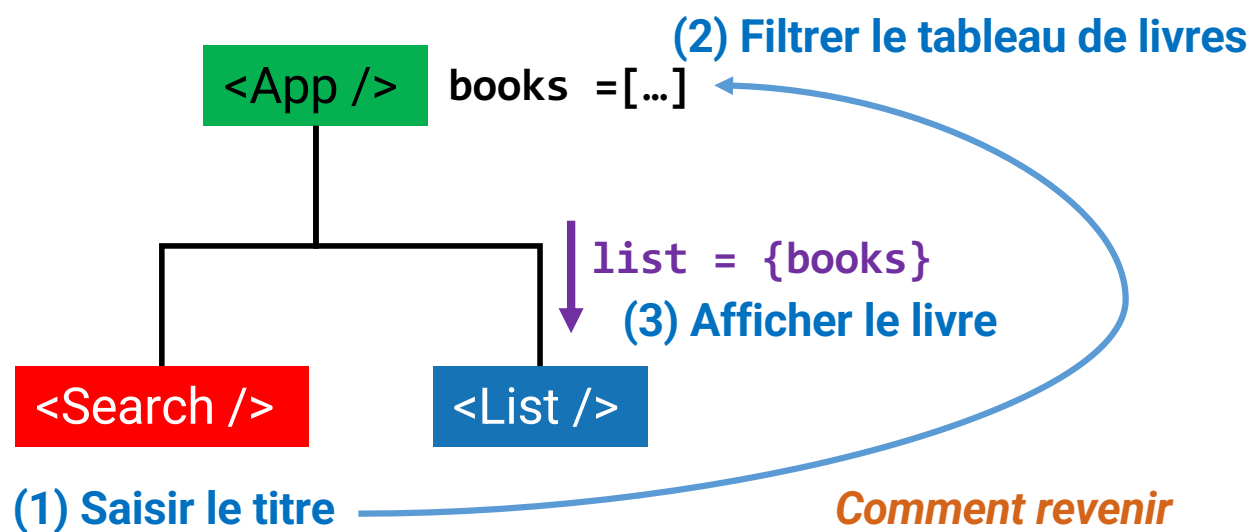
Opérateur spread (étalement)



```
const book : {...} = {
  title: "Pro Git",
  url: "https://git-scm.com/book/en/v2",
  author: "Scott Chacon and Ben Straub",
  num_comments: 2,
  points: 5,
  objectID: 3
};

const List = () => { Show usages  ⓘ HeH-Gianni *
  const [books_state : [{title: string, url: string, ... } , setbooks_state] = useState(books);
  const addBook = () : void => { Show usages  ⓘ new *
    //books.push(book);
    //setbooks_state(books);
    //console.log(books_state);
    setbooks_state( value: (prev : [{title: string, url: string, ... } ] => [...prev, book]));
  }

  return (
    <>
    <ul>
      {books_state.map((book : {...} ) => (
        <li key={book.objectID}>
          <span><a href={book.url}>{book.title}</a></span>
          <span> {book.author}</span>
          <span> ({book.num_comments})</span>
          <span> [{book.points}]</span>
        </li>
      ))}
    </ul>
    <button onClick={addBook}>Add</button>
  )
}
```



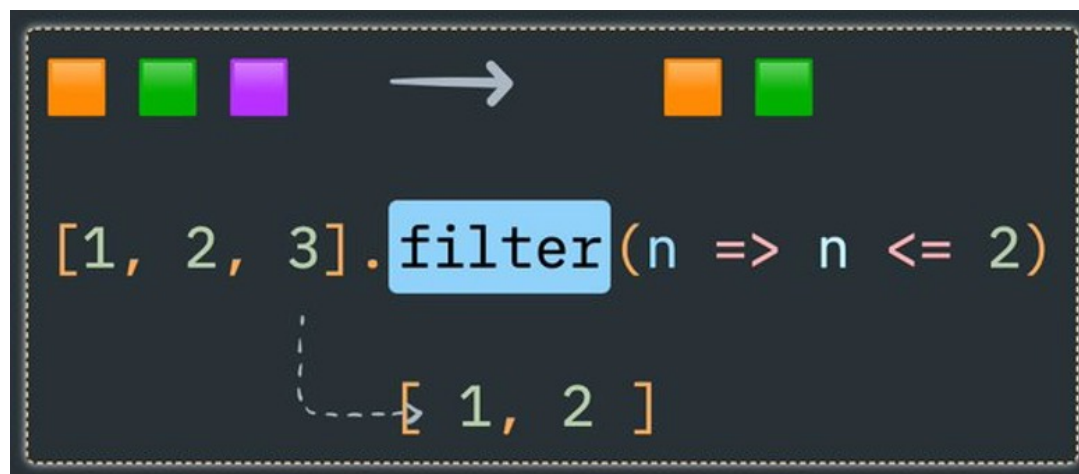
*Comment revenir
vers le composant
parent ?*

**=> En passant la référence
d'une fonction dans les props**

Filtrer le tableau de livres

La méthode ***filter()*** retourne un nouveau tableau contenant tous les éléments du tableau initial qui remplissent la condition définie dans la fonction passée en paramètre (callback).

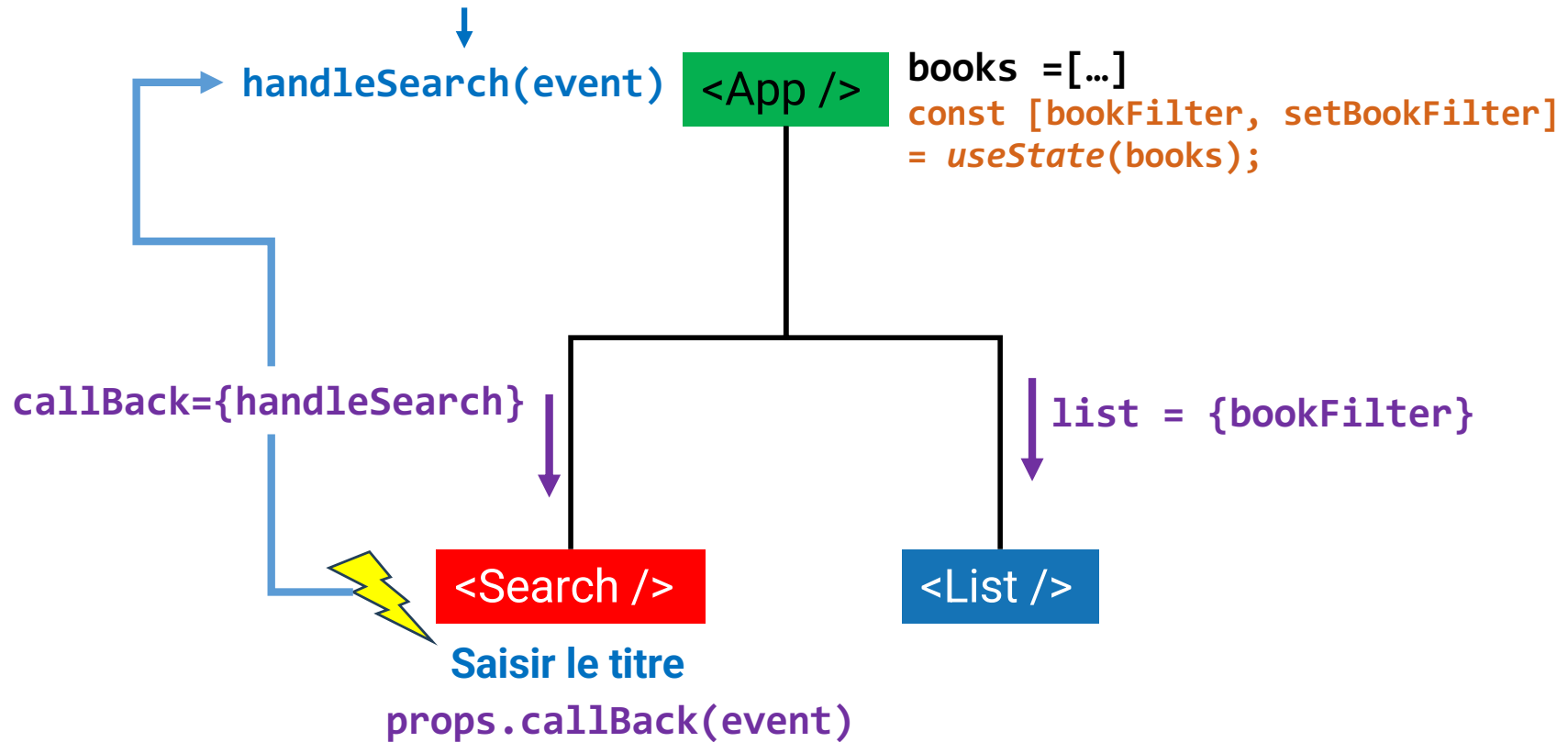
```
const fruits = ['pomme', 'banane', 'kiwi', 'raisin', 'poire', 'mandarine'];  
  
const resultat = fruits.filter(fruit => fruit.length > 5);  
  
console.log(resultat); // [ 'banane', 'raisin', 'mandarine' ]
```



Props

State

Fonction qui va filtrer le tableau de livres



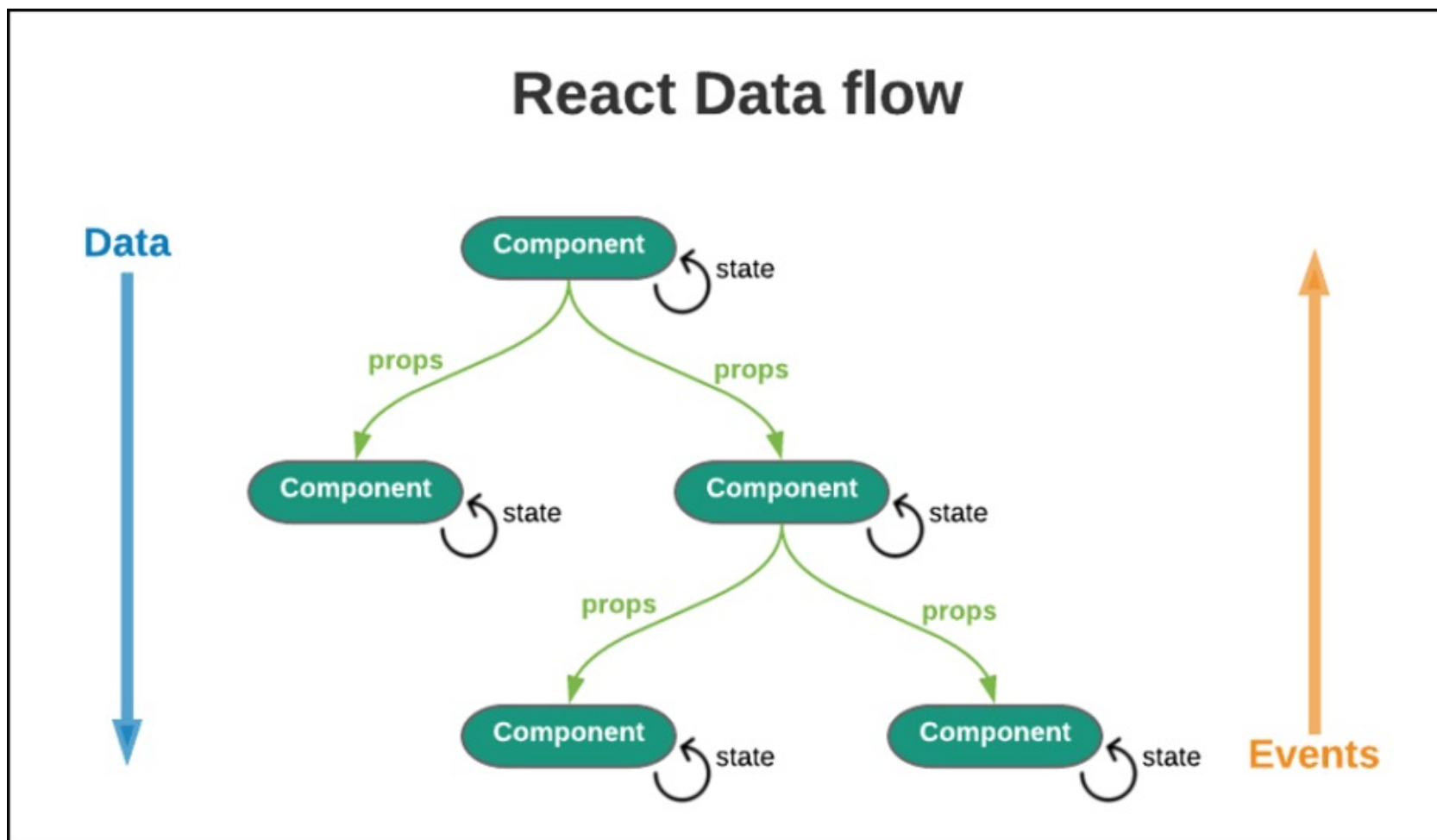
```
const books : [{title: string, url: string, ...} ] = [ 2 elements... ];

const [bookFilter : [{title: string, url: string, ...}], setBookFilter] = useState(books);

const handleSearch = (event) : void => { Show usages new *
  let searchTerm = event.target.value;
  searchTerm = searchTerm.toLowerCase();
  let returnedBook : ({...})[] = books.filter((book : {...}) => {
    return book.title.toLowerCase().includes(searchTerm);
  })
  setBookFilter(returnedBook);
}

return (
  <>
    <h1> {welcome.greeting} {welcome.title}</h1>
    <Search callback={handleSearch}/>
    <hr/>
    <List list={bookFilter}/>
  </>
)
```

```
const Search = (props) => { Show usages ⓘ HeH-Gianni *  
  const [searchTerm : string , setSearchTerm] = useState( initialState: '' );  
  
  const handleChange = (event) : void => { Show usages ⓘ HeH-Gianni *  
    setSearchTerm(event.target.value);  
    props.callBack(event);  
  }  
  return (  
    <div>  
      <label htmlFor="search">Search: </label>  
      <input id="search" type="text" onChange={handleChange}/>  
      <p>Recherche de <strong>{searchTerm}</strong></p>  
    </div>  
  );  
};
```



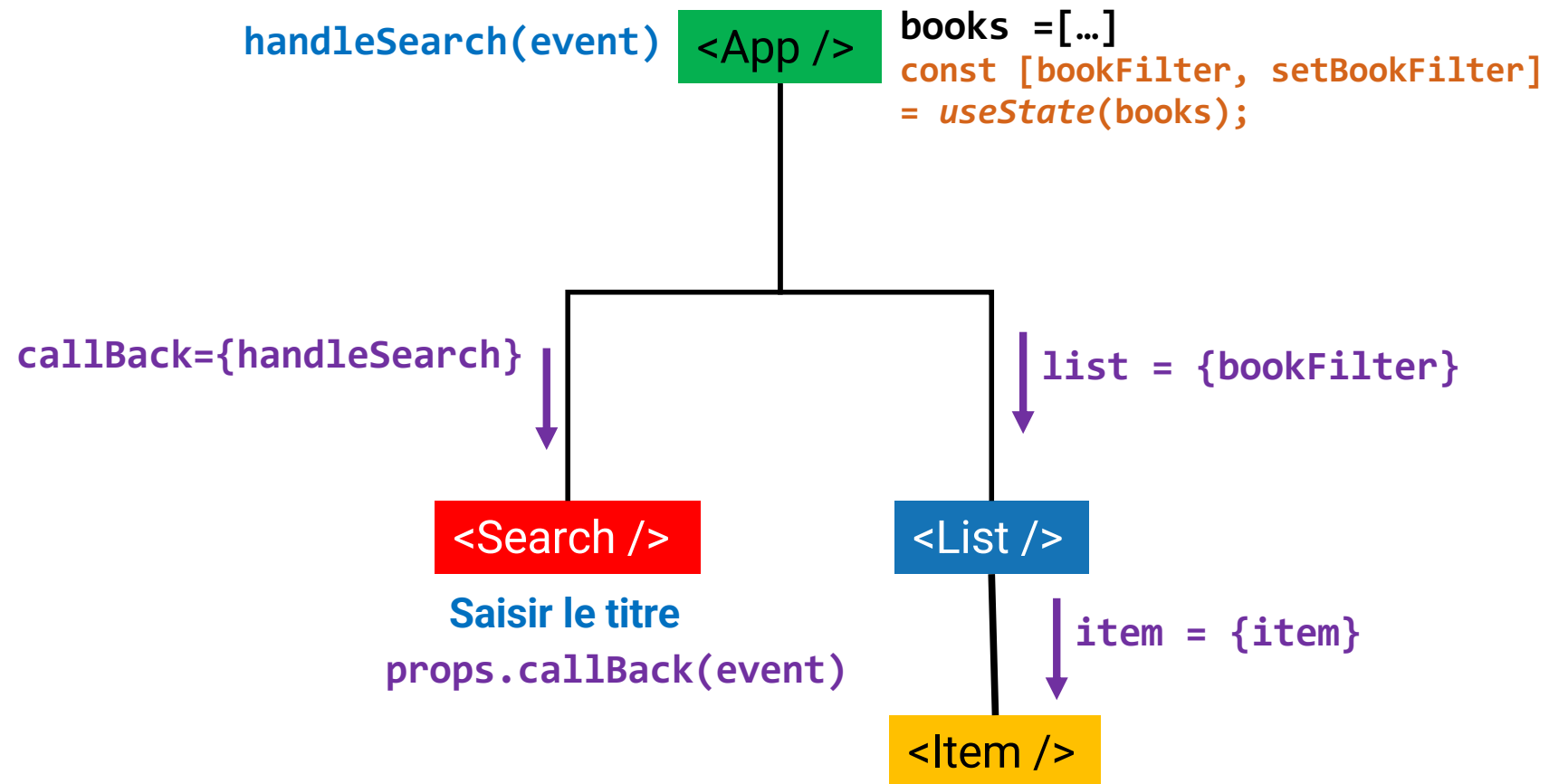
Hey React

Search:

- [React](#) Jordan Walke (3) [4]
- [Redux](#) Dan Abramov, Andrew Clark (2) [5]

Composant `<Item />`

Composant `<List />`




```

1 | import Item from "./Item"
2 | const List = (props) => {
3 |   return (
4 |     <ul>
5 |       {props.list.map((item) => {
6 |         |   return <Item key={item.objectID} item={item} />
7 |       })}
8 |     </ul>)
9 |   };
-----
1 | const Item = (props) => {
2 |   return (
3 |     <li>
4 |       <span>
5 |         <a href={props.item.url}> {props.item.title}</a>
6 |       </span>
7 |       <span>{props.item.author}</span>
8 |       <span>{props.item.num_comments}</span>
9 |       <span>{props.item.points}</span>
10 |     </li>
11 |   );
12 | }

```

<List />

↓ **item = {item}**

<Item />

Affectation par décomposition (destructuring) - Objet littéral

```
1  const contact = {  
2      nom: 'Tricarico',  
3      prenom: 'Gianni',  
4      mail: 'gianni.tricarico@heh.be'  
5  };  
6  //déstructuration de l'objet 'contact'  
7  const {nom, prenom, mail} = contact;  
8  // équivalent au code suivant  
9  // const nom = contact.nom;  
10 // const prenom = contact.prenom;  
11 // const mail = contact.mail;  
12  
13 const contactString = `Nom = ${nom} - prénom = ${prenom} - mail = ${mail}`;  
14 console.log(contactString);
```

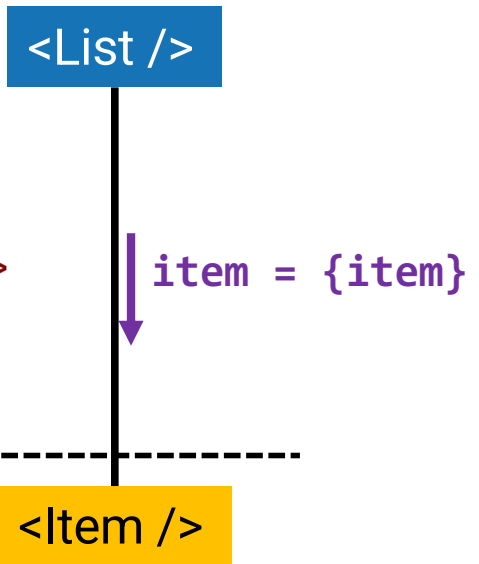
Affectation par décomposition (destructuring) - Objet littéral

```
1  const contact = {  
2      nom: 'Tricarico',  
3      prenom: 'Gianni',  
4      mail: 'gianni.tricarico@heh.be'  
5  };  
6  
7  function afficherContact({nom, prenom, mail}){  
8      const contactString = `Nom = ${nom} - prénom = ${prenom} - mail = ${mail}`;  
9      console.log(contactString);  
10 };  
11  
12 afficherContact(contact);
```

```

2  | const List = ({ list }) => {
3  |   return (
4  |     <ul>
5  |       {list.map((item) => {
6  |         return <Item key={item.objectID} item={item} />
7  |       })}
8  |     </ul>)
9  |   };
1 | const Item = ({ item }) => {
2 |   return (
3 |     <li>
4 |       <span>
5 |         <a href={item.url}> {item.title}</a>
6 |       </span>
7 |       <span>{item.author}</span>
8 |       <span>{item.num_comments}</span>
9 |       <span>{item.points}</span>
10 |     </li>
11 |   );
12 | }

```



```

const props : {item: {...}} = {
  item : {
    title: "Redux",
    url: "https://redux.org",
    author: "Dan Abramov, Andrew Clark",
    num_comments: 2,
    points: 5,
    objectID: 2
  }
}

```

```
const contact = {  
  nom: 'Tricarico',  
  prenom: 'Gianni',  
  gsm: '1234',  
  mail: 'gianni.tricarico@heh.be',  
  adresse: {  
    rue: 'Av. Maistriau, 8',  
    ville: 'Mons' }  
};  
  
const {gsm, mail, adresse:{ville}} = contact;  
  
console.log(gsm);  
console.log(ville);
```

```
1  const Item = ({ item: { url, title, author, num_comments, points } }) => {  
2    return (  
3      <li>  
4        <span>  
5          <a href={url}> {title}</a>  
6        </span>  
7        <span>{author}</span>  
8        <span>{num_comments}</span>  
9        <span>{points}</span>  
10     </li>  
11   );  
12 };
```

```
1  import Item from "./Item";
2  const List = ({ list }) => {
3    return (
4      <ul>
5        {list.map((item) => {
6          return (
7            <Item
8              key={item.objectID}
9              title={item.title}
10             url={item.url}
11             author={item.author}
12             num_comments={item.num_comments}
13             points={item.points}
14           />
15         )};
16       )}
17     </ul>
```

```
1  const Item = ({ url, title, author, num_comments, points } ) => {  
2      return (  
3          <li>  
4              <span>  
5                  <a href={url}> {title}</a>  
6              </span>  
7              <span>{author}</span>  
8              <span>{num_comments}</span>  
9              <span>{points}</span>  
10         </li>  
11     );  
12 };
```



```
var profil = { prenom: "Sarah", profilComplet: false };  
var profilMisAJour = { nom: "Dupont", profilComplet: true };  
  
var clone = { ...profil };  
// Object { prenom: 'Sarah', profilComplet: false }  
  
var fusion = { ...profil, ...profilMisAJour };  
// Object { prenom: 'Sarah', nom: 'Dupont', profilComplet: true };
```

```
const contact = {  
  nom: 'Tricarico',  
  prenom: 'Gianni',  
  gsm: '1234',  
  mail: 'gianni.tricarico@heh.be'  
};  
  
const {gsm, mail, ...identite} = contact;  
  
console.log(gsm);  
console.log(mail);  
  
console.log(identite);  
//{nom: 'Tricarico',prenom: 'Gianni'}
```

Equivalent à **...identite**

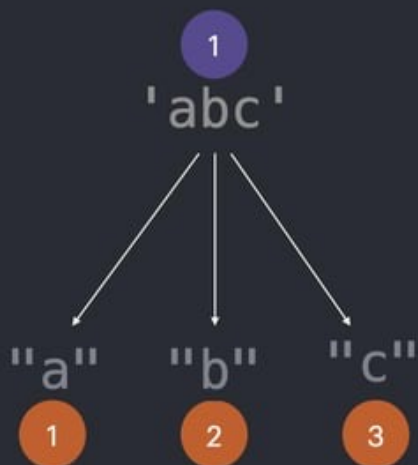
```
const identite = {  
  nom: 'Tricarico',  
  prenom: 'Gianni'  
};  
  
console.log(identite);
```

Spread

Expands iterables into individual elements

```
function spreadEx(el1, el2, el3) {  
  console.log(el1, el2, el3)  
}
```

```
spreadEx(...'abc') // "a", "b", "c"
```



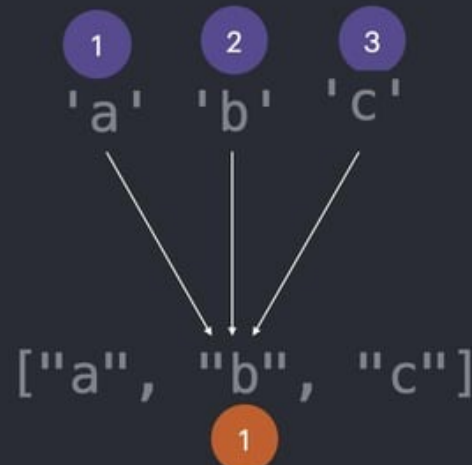
@ZorefCode

Rest

Collects multiple elements into one

```
function restEx(...elems) {  
  console.log(elems)  
}
```

```
restEx('a', 'b', 'c') // ["a", "b", "c"]
```

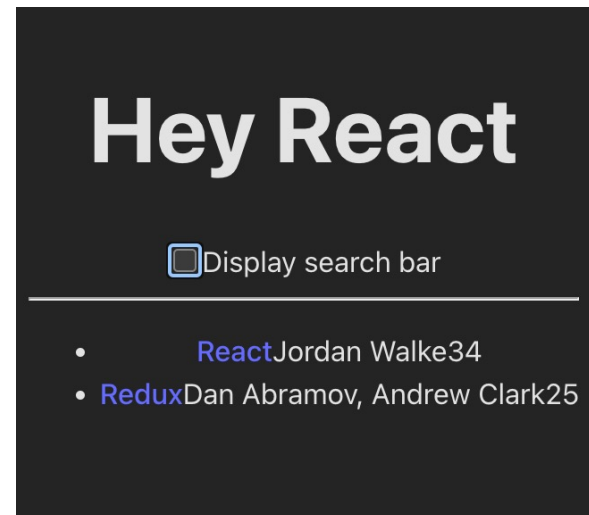
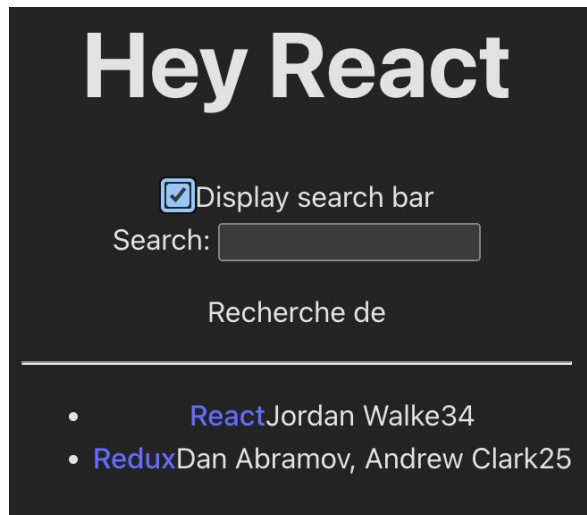


```
2  const List = ({ list }) => {  
3      return (  
4          <ul>  
5              {list.map((item) => {  
6                  return (  
7                      <Item  
8                          key={item.objectID}{...item}  
9                      />  
10                 )};  
11             }  
12         </ul>
```

```
1  import Item from "./Item";
2  const List = ({ list }) => {
3      return (
4          <ul>
5              {list.map(({objectID, ...item}) => {
6                  return (
7                      <Item
8                          key={objectID}{...item}
9                      />
10                 );
11             })}
12          </ul>
13      );
```

Opérateur Rest
`{objectID,...item} = item`

Opérateur Spread
`item = {...item}`



Le contenu conditionnel désigne simplement tout type de contenu qui ne doit s'afficher que dans certaines circonstances.

Les 2 techniques pour un affichage conditionnel

- L'opérateur logique &&
- L'opérateur ternaire

a	b	a && b
true	true	true
true	false	false
false	true	false
false	false	false

&& équivalent à une multiplication

$$1 \cdot X = X$$

$$0 \cdot X = 0$$

(x peut prendre 0 ou 1)

a	b	a b
true	true	true
true	false	true
false	true	true
false	false	false

|| équivalent à une addition logique

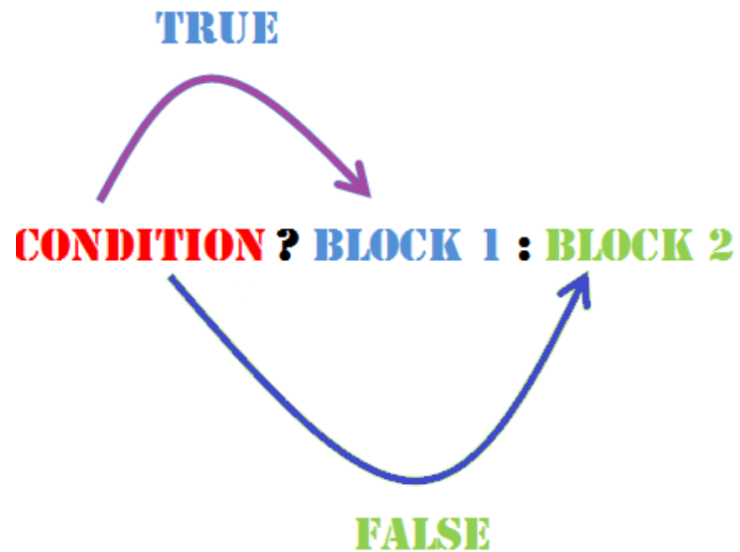
$$1 + X = 1$$

$$0 + X = X$$

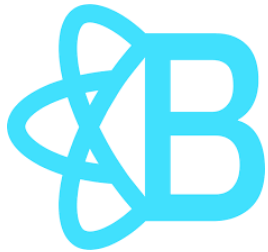
(x peut prendre 0 ou 1)


```
37 | const [isShow, setIsShow] = useState(false);
38 | const handleCheckedChange = (event) => {
39 |   setIsShow(event.target.checked);
40 | };
41 | return (
42 |   <div>
43 |     <h1>
44 |       {welcome.greeting} {welcome.title}
45 |     </h1>
46 |     <div>
47 |       <input
48 |         type="checkbox"
49 |         name="searchBar"
50 |         onChange={handleCheckedChange}
51 |       />
52 |       <label htmlFor="searchBar">Display search bar</label>
53 |     </div>
54 |     {isShow && <Search onSearch={handleSearch} />}
55 |     <hr />
56 |     <List list={bookFilter} />
57 |   </div>
58 | );
59 |
```

You, il y a 3 semaines • Premier composant React



```
{isShow?<Search onSearch={handleSearch} />:null}
```



React Bootstrap

<https://react-bootstrap.netlify.app/>

Why React-Bootstrap?

Theming

Color modes

Getting help

RTL

Server-side Rendering

Layout >

Forms >

Components ▾

- Accordion
- Alerts
- Badges
- Breadcrumbs
- Button group
- Buttons**
- Cards
- Carousels

RESULT

Primary Secondary Success Warning Danger Info Light Dark [Link](#)

LIVE EDITOR

```
import Button from 'react-bootstrap/Button';

function TypesExample() {
  return (
    <>
      <Button variant="primary">Primary</Button>{' '}
      <Button variant="secondary">Secondary</Button>{' '}
      <Button variant="success">Success</Button>{' '}
      <Button variant="warning">Warning</Button>{' '}
      <Button variant="danger">Danger</Button>{' '}
      <Button variant="info">Info</Button>{' '}
      <Button variant="light">Light</Button>{' '}
      <Button variant="dark">Dark</Button>
      <Button variant="link">Link</Button>
    </>
  );
}
```

Installation de la bibliothèque

```
npm install react-bootstrap bootstrap
```

Importer le composant

```
import Button from 'react-bootstrap/Button';
```

Importer la feuille de style

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

Partie 5: web service

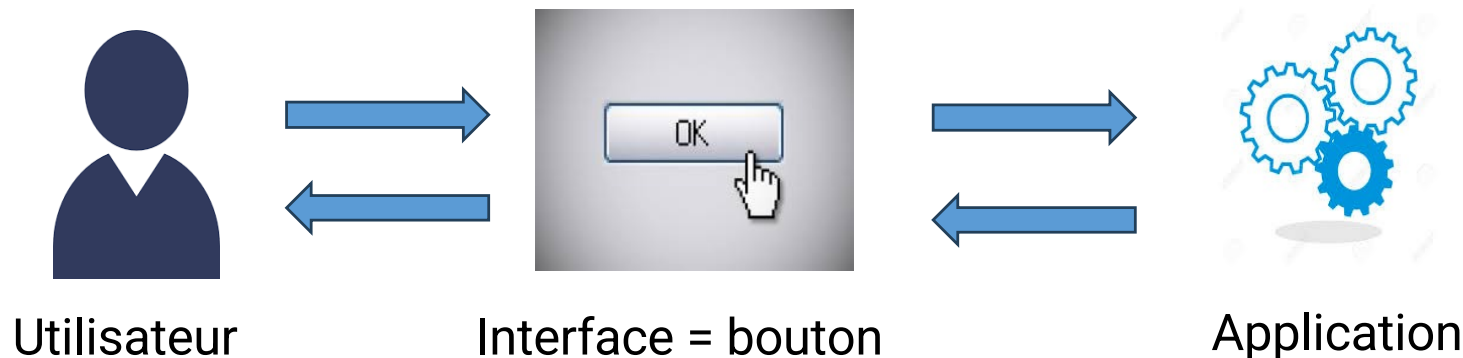


WEB SERVICE

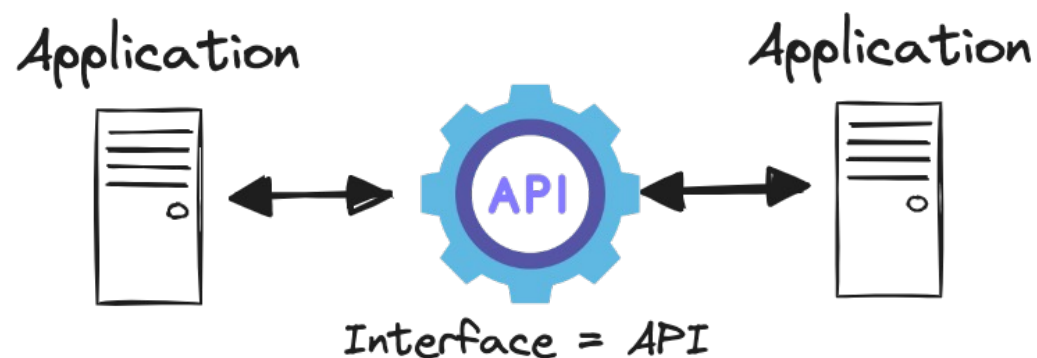
API = **A**pplication **P**rogramming **I**nterface = Interface applicative de programmation

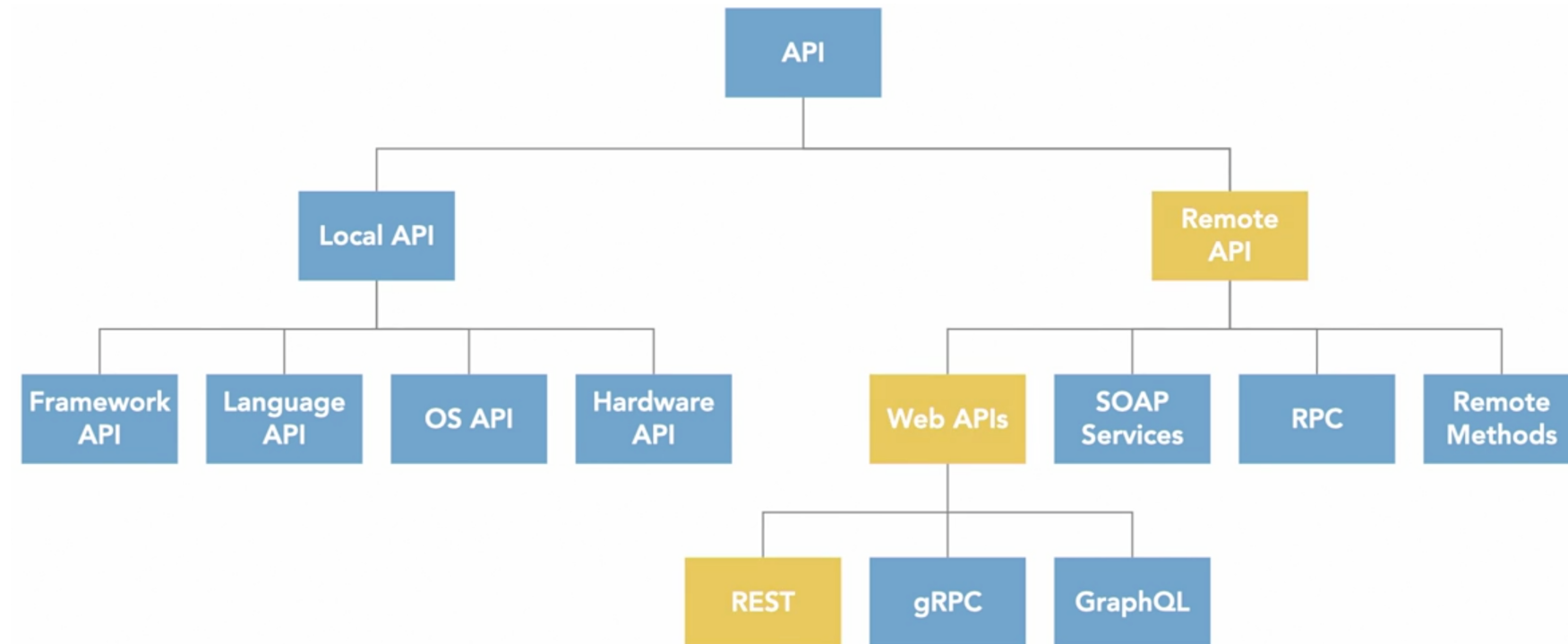
Interface : Une interface définit la frontière de communication entre deux entités, comme des éléments de logiciel, des composants de matériel **informatique**, ou un utilisateur. Elle se réfère généralement à une image abstraite qu'une entité fournit d'elle-même à l'extérieur. ([Wikipédia](#))

GUI = Graphical User Interface

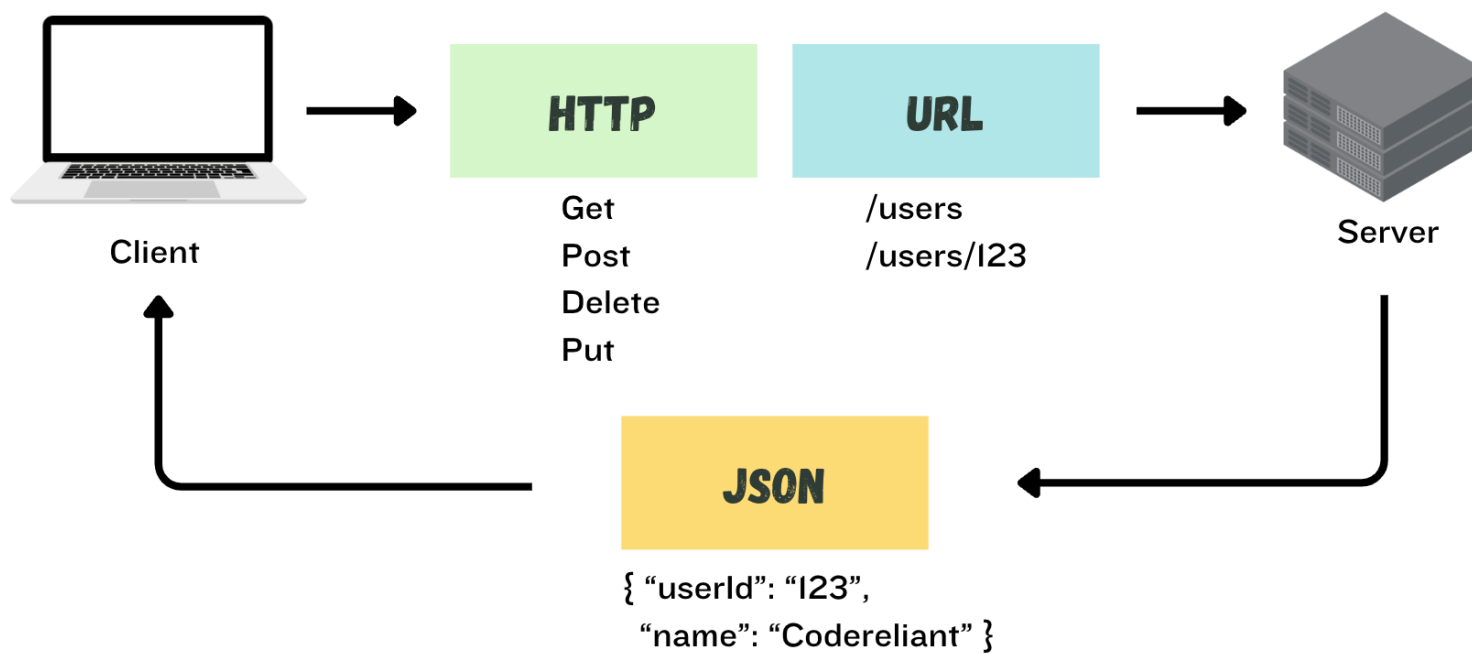


On parle d'API à partir du moment où une entité informatique cherche à agir avec ou sur un système tiers, et que cette interaction se fait de manière normalisée en respectant les contraintes d'accès définies par le système tiers. On dit que le système tiers « expose une API. »

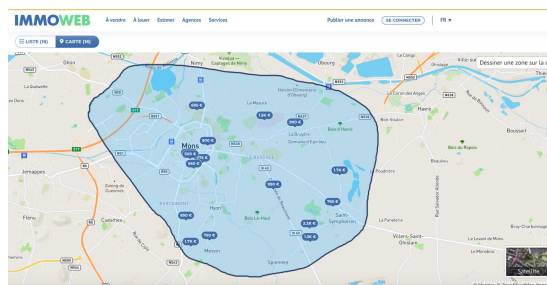




*Remote **P**rocedure **C**all = RPC*



API REST – Exemple



Immoweb.be (biens immobiliers)



lacampagne.mobicoop.fr
(covoiturage)

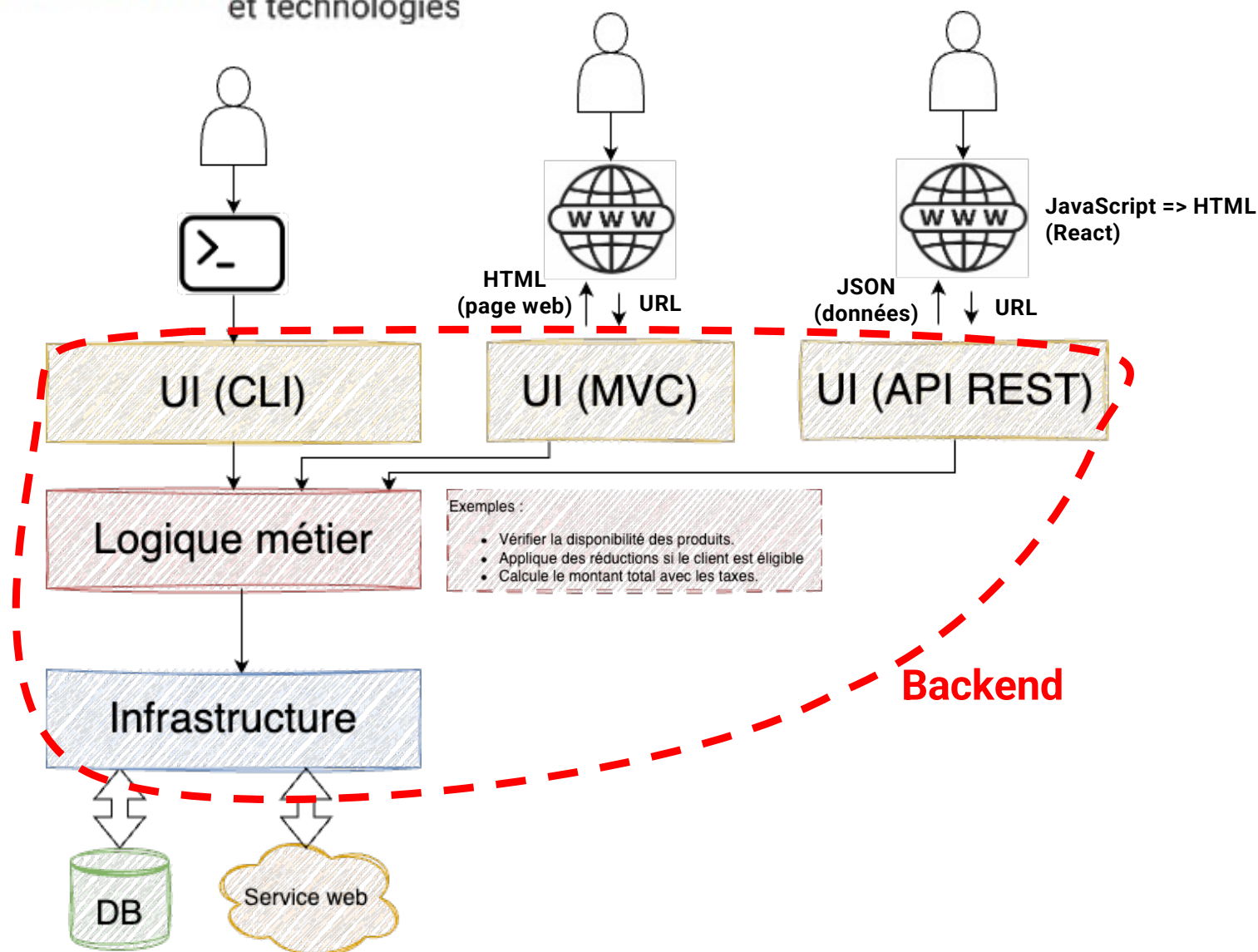


Interface
= API



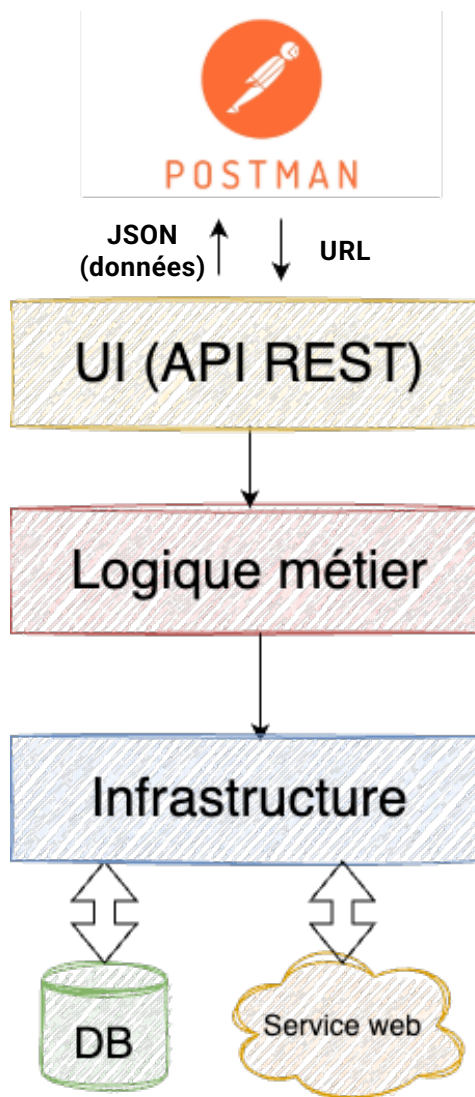
OpenStreetMap.org

API REST – Exemple



API REST – Exemple

FreeTestAPI



API REST est basée sur une architecture client-serveur => Les clients de l'API utilisent des appels **HTTP** pour demander une ressource (une méthode GET) ou envoyer des données au serveur (une méthode POST), ou l'une des autres **méthodes HTTP** prises en charge par l'API.

HTTP (HyperText Transfer Protocol)

Protocole destiné, initialement, à transférer des documents hypertextes (documents contenant des liens vers d'autres documents).

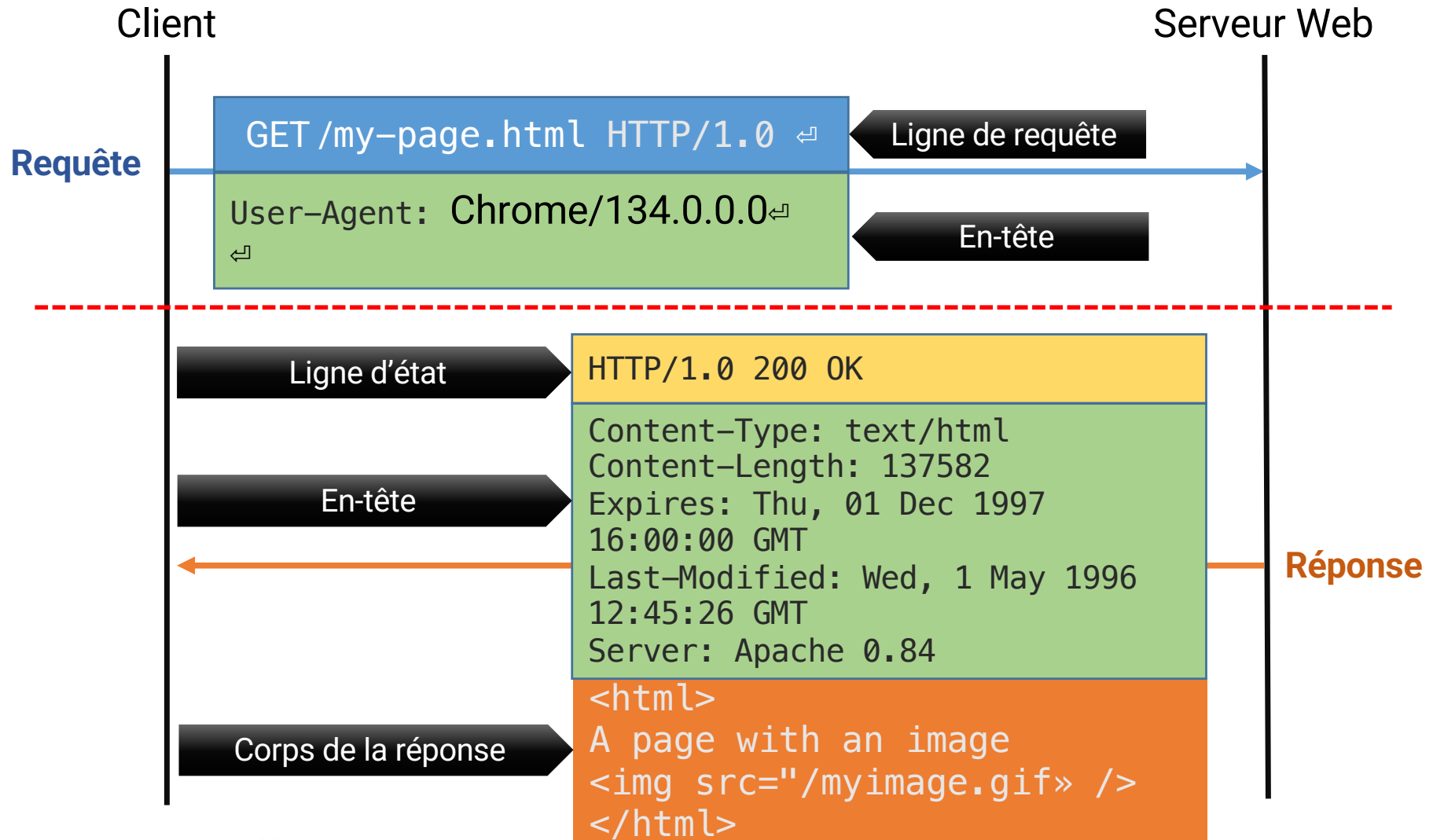
A l'origine, les documents contenaient uniquement du texte.

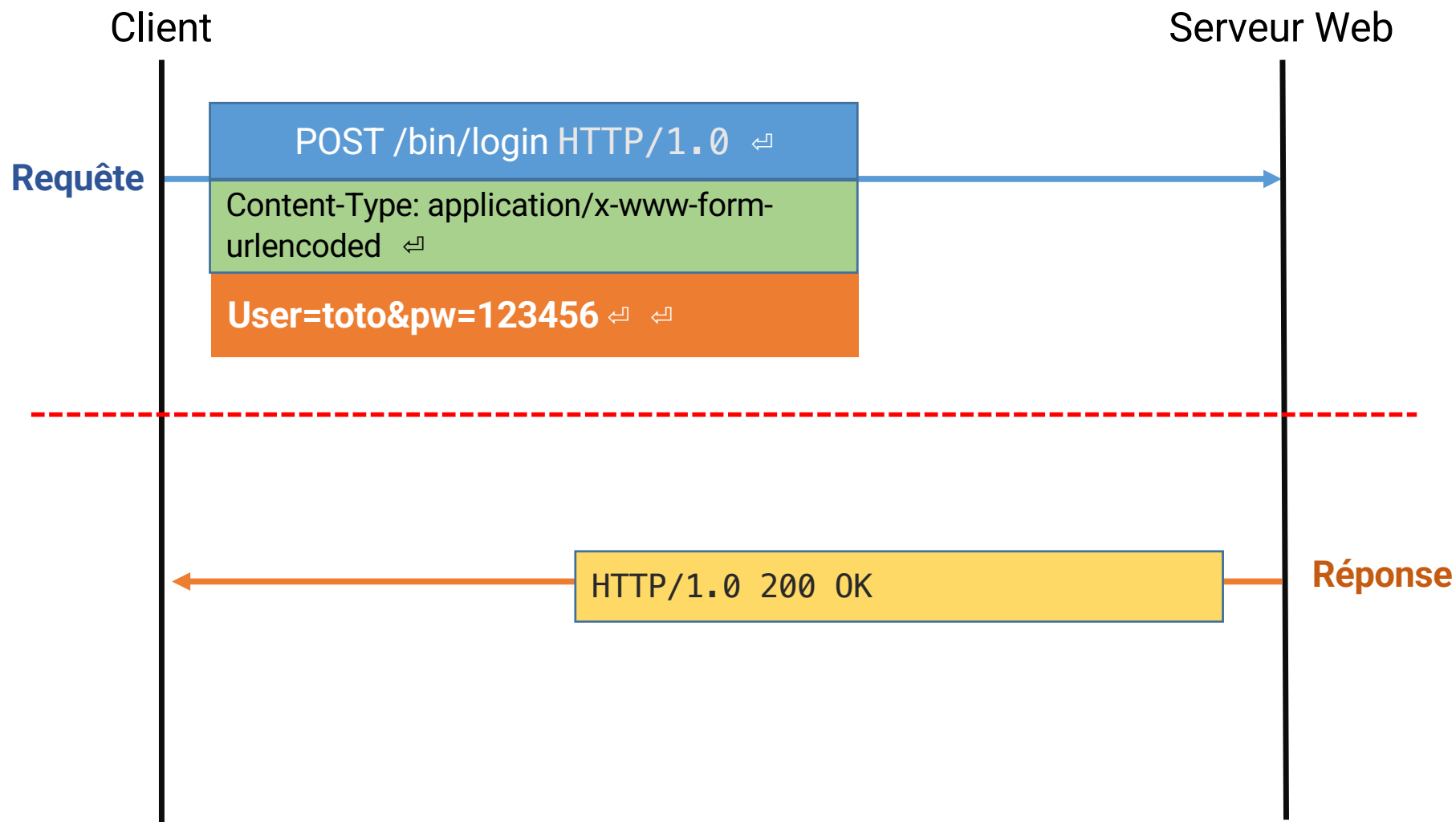
Protocole = Format des données + Règles

Document hypertexte

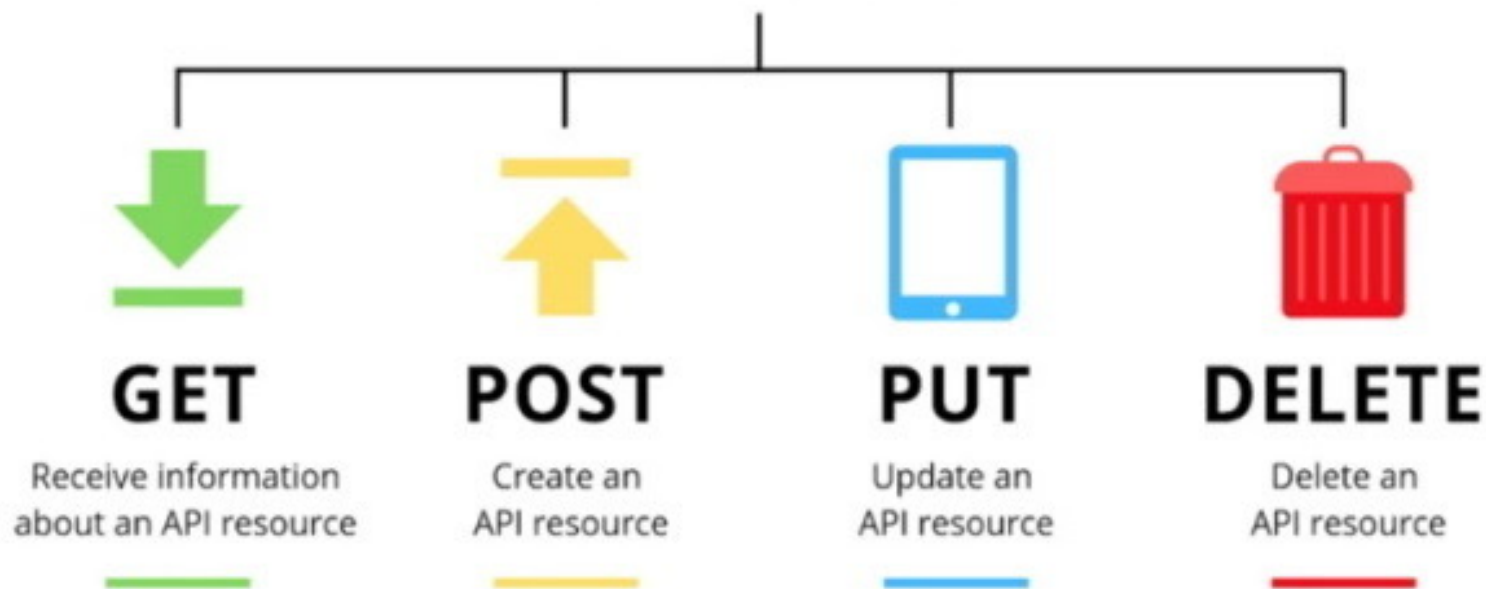
C'est un document ou un ensemble de documents informatiques qui permet de passer d'une information à l'autre grâce à un système de renvois appelés hyperliens, ou liens hypertextes. (Wikipédia)

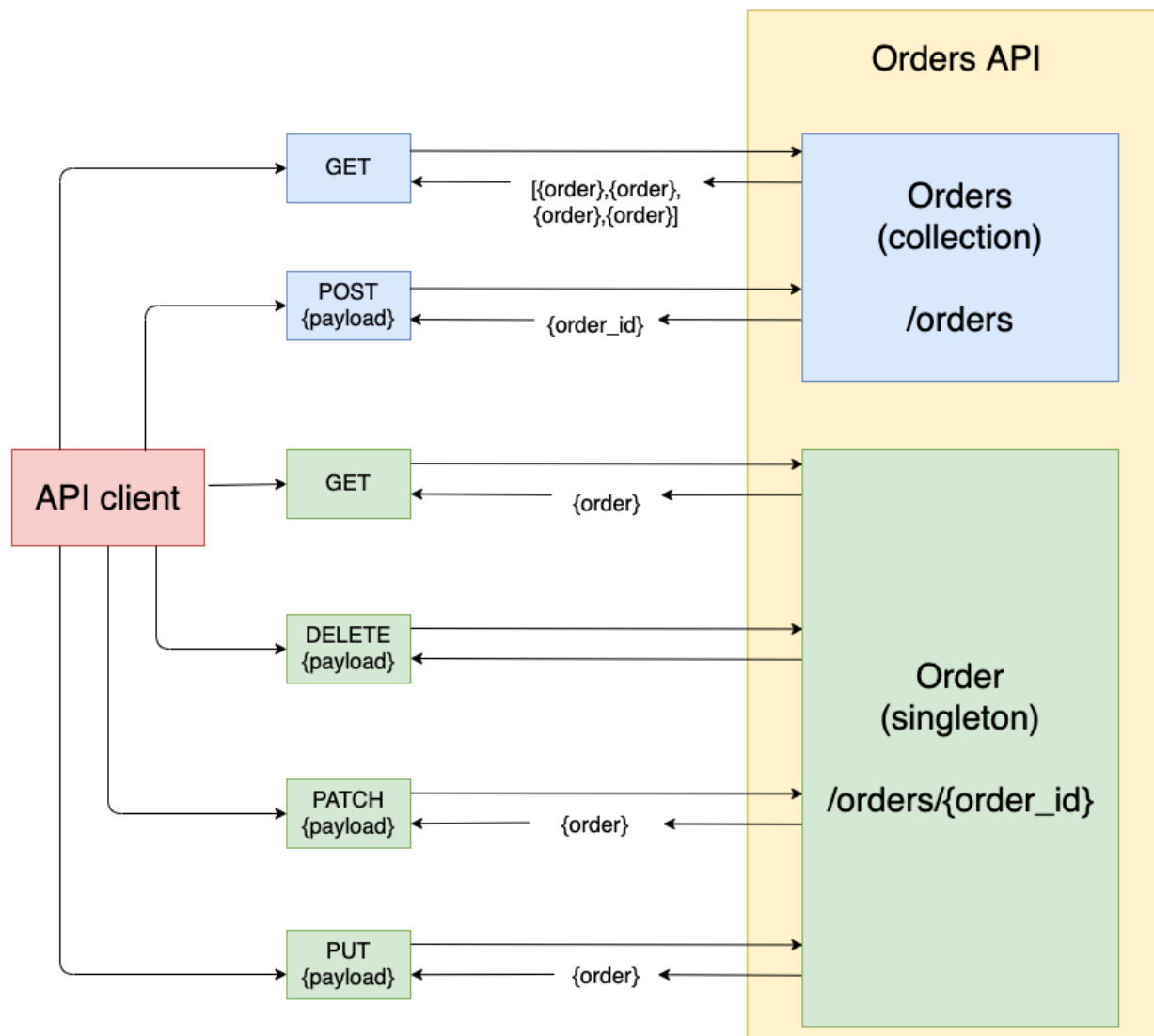




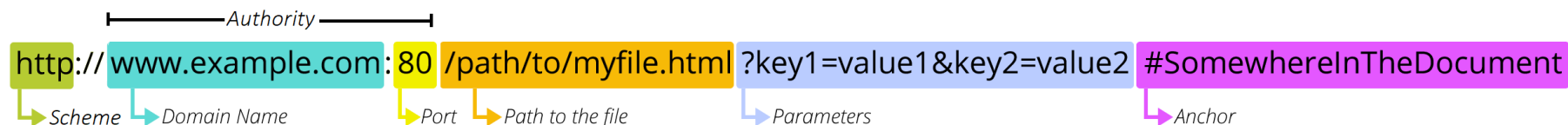


REST API Methods



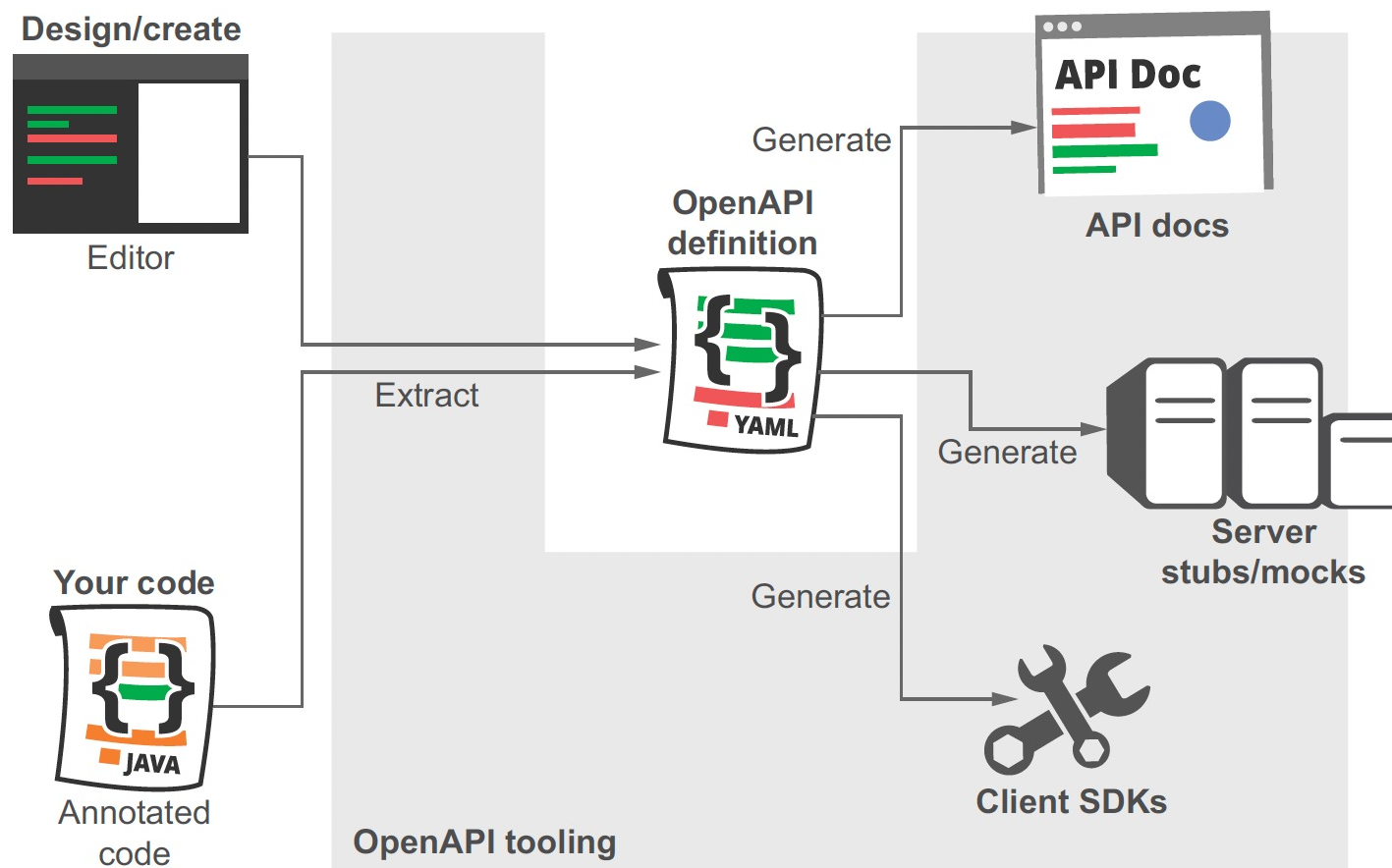


Le localisateur de ressources uniformes (URL) est la forme la plus courante d'identificateur de ressources. Les URL décrivent l'emplacement spécifique d'une ressource sur un serveur particulier.





« La spécification OpenAPI 3.0 (OAS) définit une interface standard indépendante du langage pour les API RESTful, qui permet à la fois aux humains et aux ordinateurs de découvrir et de comprendre les fonctionnalités du service sans accès au code source, à la documentation ou via l'inspection du trafic réseau. »



POST /product
^ 🔒

This route allow only admin and seller to add new product

Parameters
Try it out

Name	Description
Accept-Language (header)	<i>Example : en_MX</i> en_MX

Request body required
multipart/form-data

```

name
string

description
string

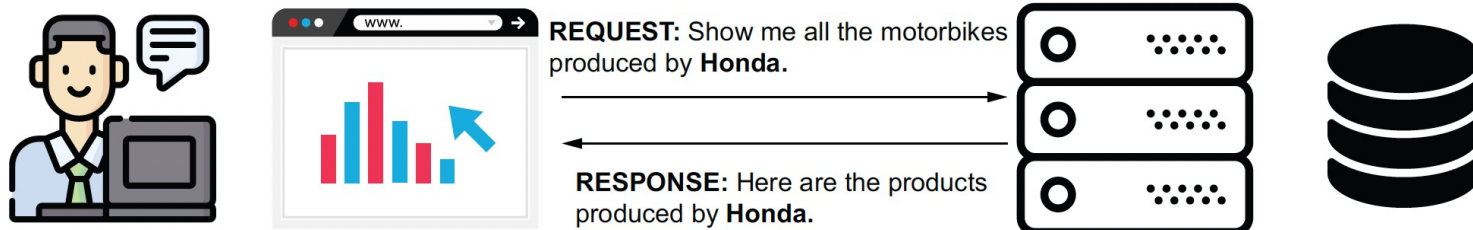
category
string

price
integer

priceDiscount
integer
    
```

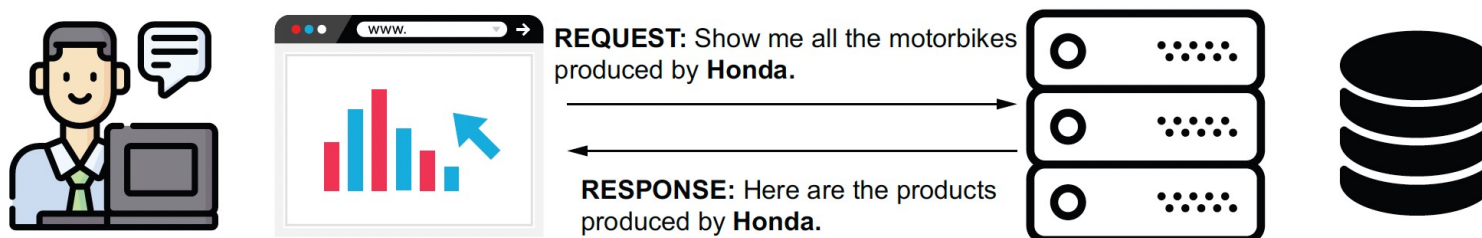
Paramètre de requête (query)

`http://example.com/products?brand=honda`



Paramètre/variable de chemin (path)

`http://example.com/products/honda`



Product

GET

/product

This route allow you to get all products

Parameters

Cancel

Name	Description
Accept-Language (header)	en_MX
filter (query)	This will filter all products and select only products that contain the word you insert and search in all product fields about this word flawless
select (query)	Select only fields you want. name, image
limit (query)	Limit the number of products from for example 20 product to 5 products. 5
sort (query)	Sorting products according to specified field for example the name field, and the number before the field name indicates the order of items: descending (-1) or ascending (1)

<http://localhost:3000/api/product?filter=flawless&limit=5>

DELETE `/product/{productId}`

This route allow logged in seller/admin to delete product using it's ID

Parameters Try it out

Name	Description
Accept-Language <i>(header)</i>	Example : en_MX en_MX
productId <i>(path)</i>	Product ID productId

Authentication
=> chemin d'accès protégé

POST **/product** 

This route allow only admin and seller to add new product

Parameters Try it out

Name	Description
Accept-Language	Example : en_MX
(header)	en_MX

Request body required multipart/form-data

name

string

description

string

category

string

price

integer

priceDiscount

integer

JWT
JSON WEB TOKEN



Il permet l'échange sécurisé de jetons entre plusieurs parties. Cette sécurité de l'échange se traduit par la vérification de **l'intégrité** et de **l'authenticité** des données.

HEADER
ALGORITHM
& TOKEN TYPE

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

← Algorithme de signature

PAYLOAD
DATA

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "admin": true  
}
```

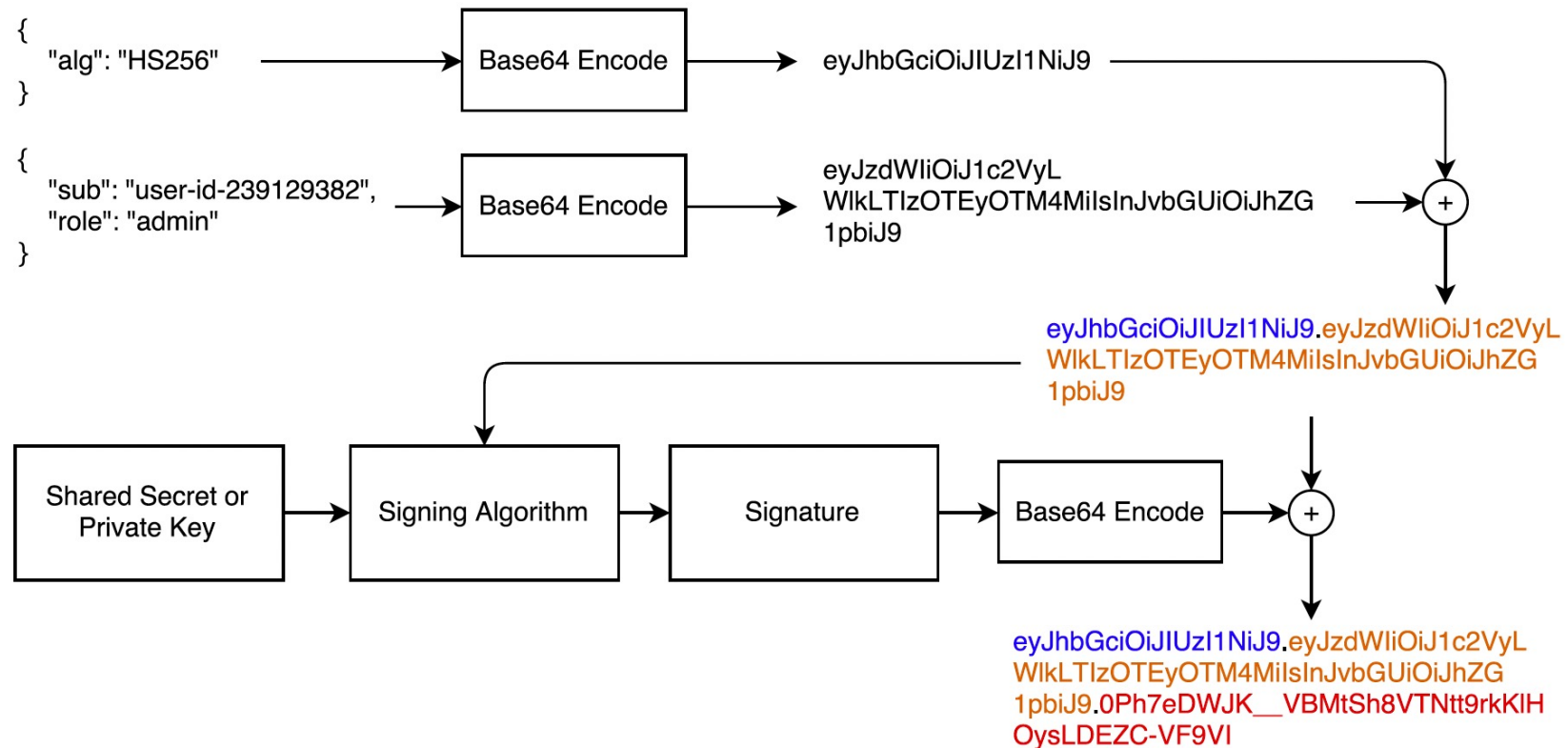
SIGNATURE
VERIFICATION

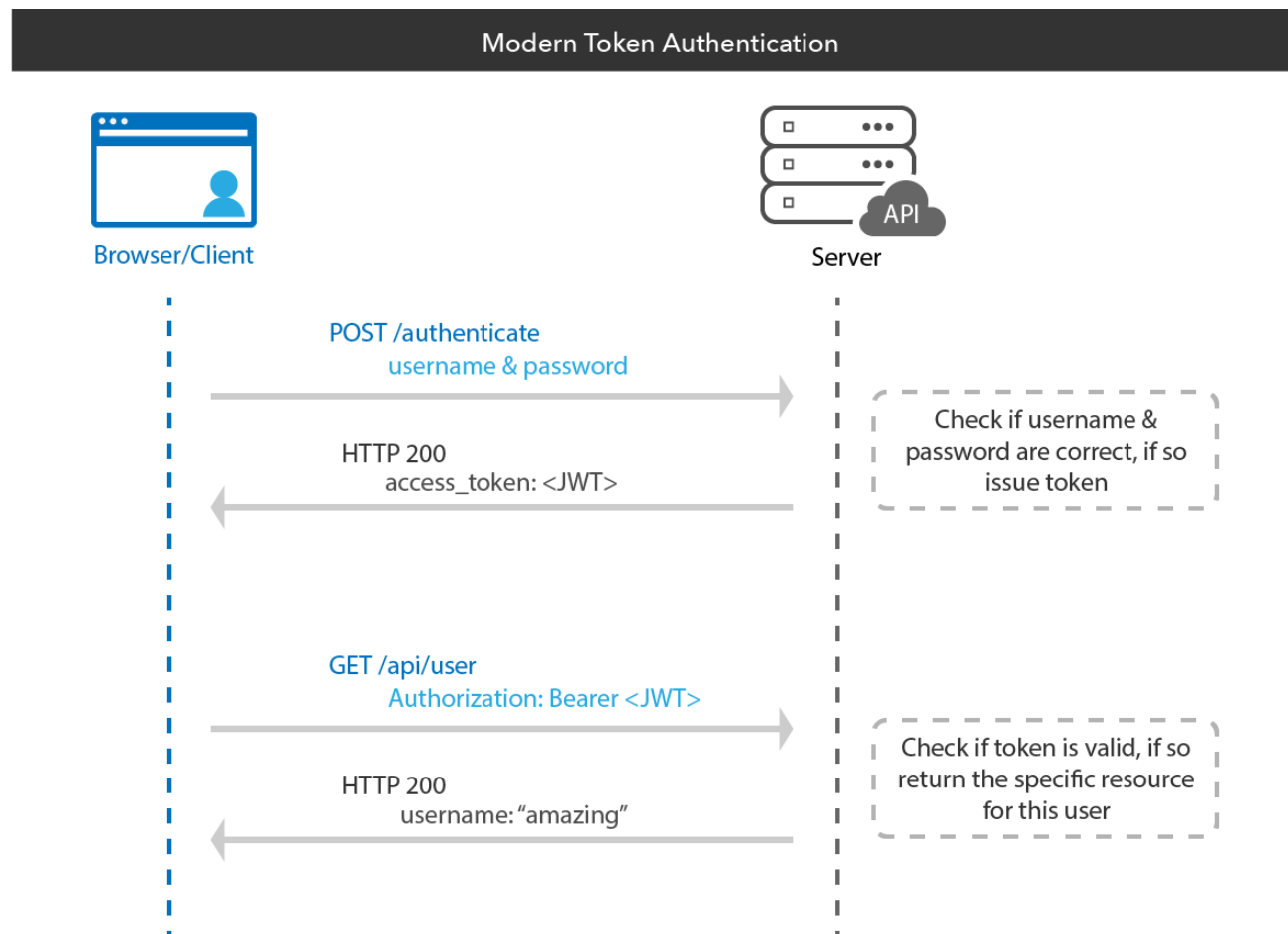
```
HMACSHA256(  
  base64UrlEncode(header) + "." +  
  base64UrlEncode(payload),secretKey)
```

NORDICAPIS.COM

<https://jwt.io/>

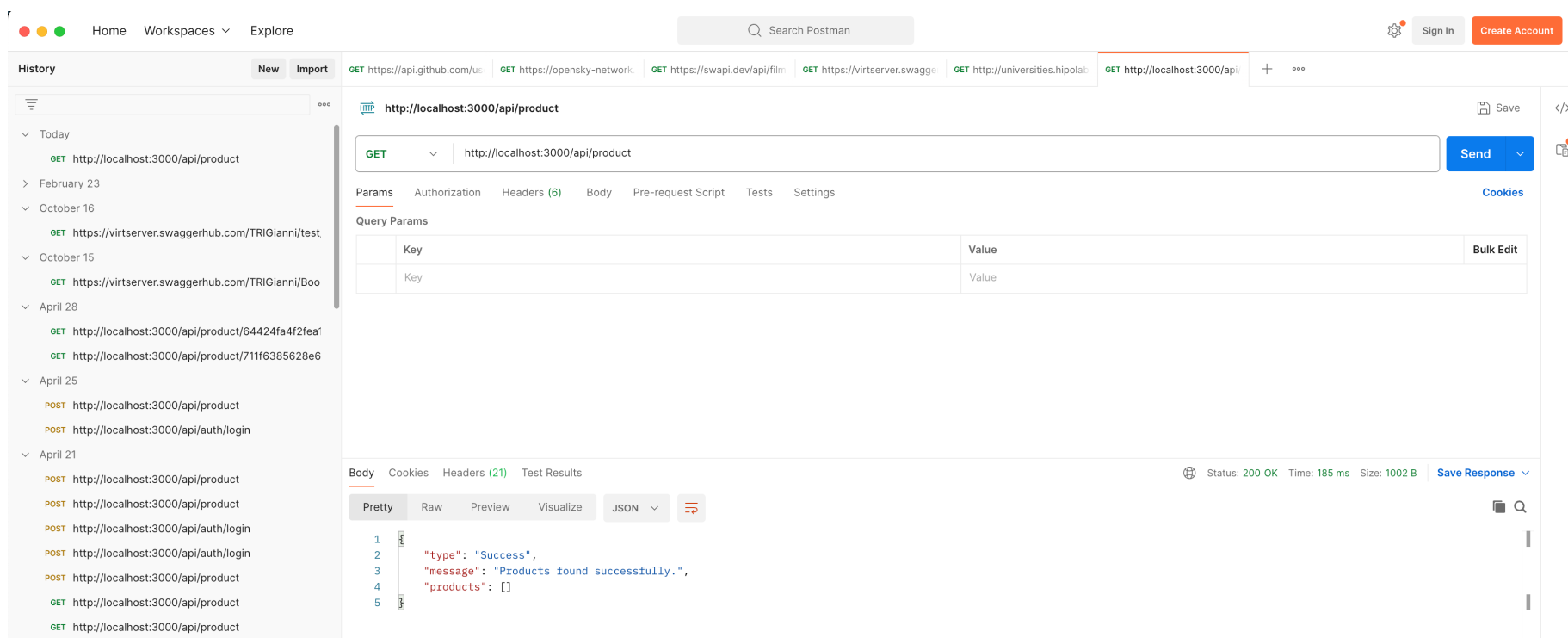
Génération du token



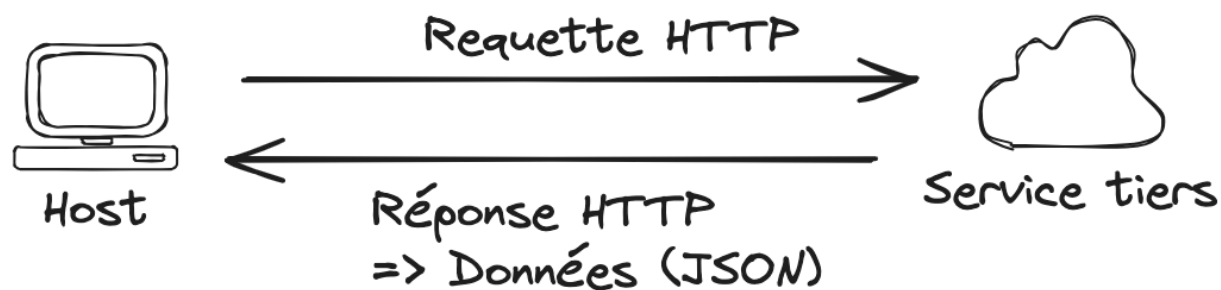




Postman est une application permettant de tester des API.



Axios : bibliothèque qui effectue des requêtes asynchrones vers des API distantes.



Installer la bibliothèque

```
npm install axios
```

Importer axios

```
import axios from 'axios';
```

```
7 function App() {  
8   const [pics, setPics] = useState([]);  
9   const [isHide, setIsHide] = useState(false);  
10  const getData = () => {  
11    axios  
12    .get("https://picsum.photos/v2/list?page=5&limit=5")  
13    .then((response) => {  
14      setPics(response.data);  
15      console.log(response.data);  
16    })  
17    .catch((error) => {  
18      console.log(error.message);  
19    });  
20  };  
21  getData();  
22  const handleChange = (event) => {  
23    let searchText = event.target.value;  
24    let picsFiltered = pics.filter((item) => {  
25      return item.author.includes(searchText);  
    });  
  }  
}
```

Définition de la fonction de
récupération de données (Axios)

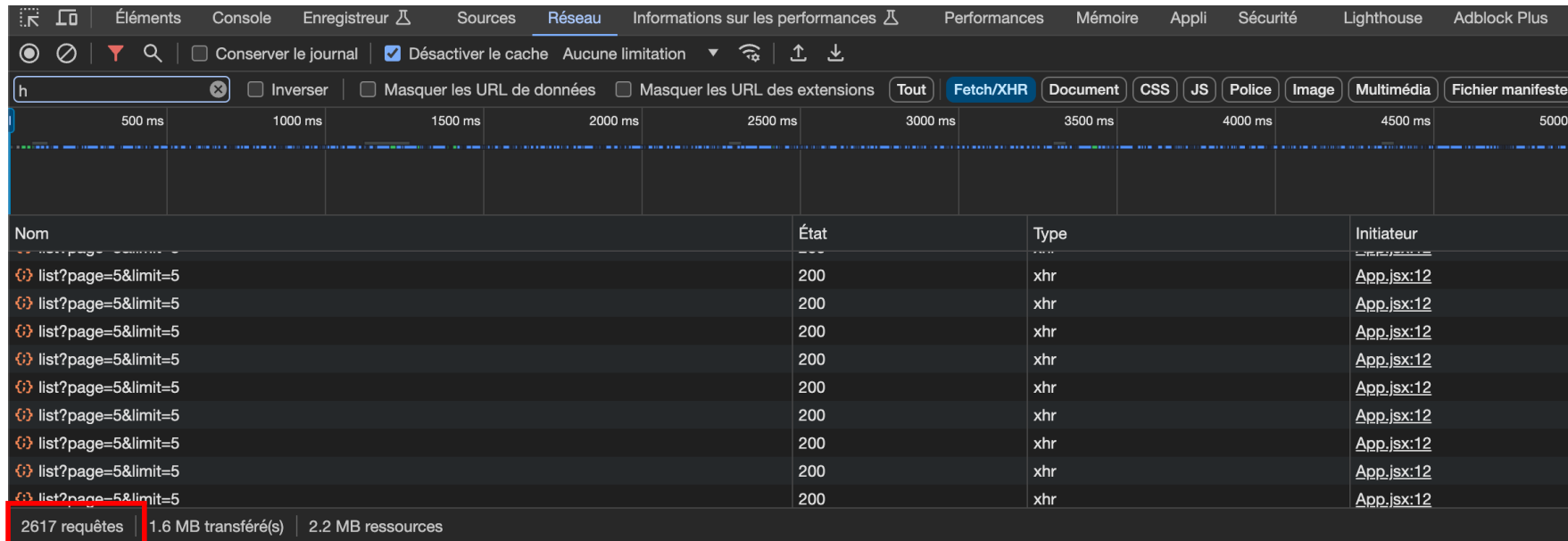
Invocation de la méthode

Les effets de bord sont des actions ou des processus qui se produisent en plus du "processus principal".

Rappel : Chaque fois que l'état d'un composant change, React réévalue ce composant.

=> "Réévaluer" signifie simplement que la fonction du composant est exécutée à nouveau automatiquement par React.

Étant donné que l'envoi de la requête HTTP ne fait pas partie du processus principal (rendu de l'interface utilisateur) exécuté par la fonction du composant, il est considéré comme un "effet de bord".



XHR => XMLHttpRequest (requête AJAX)

Effet de bord : Première exécution (rendu) du composant App => exécution de la fonction « getData » => Axios change l'état de la variable « pics » => réévaluation du composant App => exécution de la fonction « getData » => Axios change l'état de la variable « pics » => etc...
=> Boucle infinie !!!!!

Voici une liste non exhaustive d'exemples d'effets de bord:

- Envoi d'une requête HTTP (comme indiqué précédemment)
- Stockage de données dans la mémoire du navigateur ou récupération de données de cette mémoire (par exemple, via l'objet localStorage intégré)
- Réglage de temporisations (via setTimeout()) ou d'intervalles (via setInterval())
- Enregistrement de données dans la console via console.log()

Hook useEffect()

```
useEffect( () => {...}, [dependencies] );
```

() => {...} Une fonction « effet de bord » qui doit être exécutée APRÈS chaque évaluation de composant SI les dépendances spécifiées ont changé.

[dependencies]

Dépendances d'effet de bord sont les valeurs d'état, les props et les fonctions qui peuvent être exécutées à l'intérieur de la fonction d'effet de bord.

```
useEffect( () => {...}, [] );
```

Un **tableau vide** permet à la fonction d'effet de ne s'exécuter qu'une seule fois (après la première invocation du composant)

Contrairement à useState() ne renvoie pas de valeur.

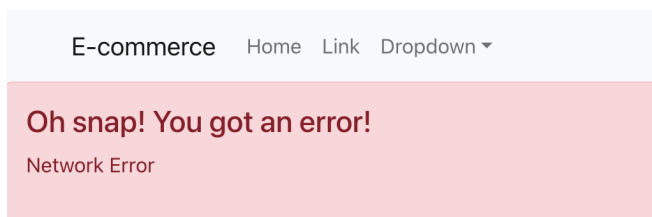
```
const [bookFilter : any[], setBookFilter] = useState( initialState: [] );  
useEffect( effect: () : void => {  
  axios.get( url: "https://freetestapi.com/api/v1/books?limit=100" )  
    .then( (response : AxiosResponse<any> ) : void => {  
      console.log(response);  
      setBookFilter(response.data);  
    }) Promise<void>  
    .catch( (error) : void => {  
      console.log(error);  
    })  
  }, deps: [] )
```

Lors de la récupération des données, vous devez gérer 2 situations :

1. **Connexion lente : => Afficher l'état de chargement des données à l'aide d'un spinner**



2. **Serveur du back-end ne répond plus : Afficher un message d'erreur**



```
6  function App() {
42  |   const [books, setBooks] = useState([]);
43  |   const [isLoading, setIsLoading] = useState(true);
44  |   const [httpError, setHttpError] = useState();
45  |   useEffect(() => {
46  |       axios
47  |       |   .get("https://freetestapi.com/api/v1/books?limit=100")
48  |       |   .then((response) => {
49  |           |   setBooks(response.data);
50  |           |   setIsLoading(false);
51  |       |   })
52  |       |   .catch((error) => {
53  |           |   setIsLoading(false);
54  |           |   setHttpError(error.message);
55  |       |   });
56  |   }, []);
57  |
58  |   if (isLoading) {
59  |       return (
60  |         <h1>Loading ...</h1>
61  |       )
62  |   }

```

} spinner

```
63 | if (httpError) {  
64 |     return (  
65 |         <h1>{httpError} </h1>  
66 |     )  
67 | }  
68 |
```


Hey React

<Search />

Search:

Recherche de

- [To Kill a Mockingbird](#) Harper Lee () []
- [1984](#) George Orwell () []
- [Pride and Prejudice](#) Jane Austen () []
- [The Great Gatsby](#) F. Scott Fitzgerald () []
- [Moby-Dick](#) Herman Melville () []
- [The Lord of the Rings](#) J.R.R. Tolkien () []

Nouvelle fonctionnalité :

L'utilisateur doit pouvoir rechercher un livre en utilisant le formulaire intégré au composant `<Search />`.

```
function App() { Show usages ⓘ HeH-Gianni *  
  const welcome : {greeting: string, title: string} = {  
    greeting: 'Hey',  
    title: 'React'  
  };  
  
  const [bookFilter : any[] , setBookFilter] = useState( initialState: []);  
  const [url : string , setUrl] = useState( initialState: 'https://freetestapi.com/api/v1/books?limit=100');  
  const [searchTerm : string , setSearchTerm] = useState( initialState: '' );  
  
  useEffect( effect: () : void => {  
    axios.get(url) Promise<AxiosResponse<...>>  
      .then((response : AxiosResponse<any> ) : void => {  
        console.log(response);  
        setBookFilter(response.data);  
      }) Promise<void>  
      .catch((error) : void => {  
        console.log(error);  
      })  
  }, deps: [url])
```

```
const handleChange = (event) :void => { Show usages new *  
  setSearchTerm(event.target.value);  
}  
const handleSearchSubmit = (event) :void => { Show usages new *  
  setUrl( value: 'https://freetestapi.com/api/v1/books?search=' + searchTerm);  
  event.preventDefault();  
}  
  
return (  
  <>  
    <h1> {welcome.greeting} {welcome.title}</h1>  
    <Search submit={handleSearchSubmit} callback={handleChange} term={searchTerm} />  
    <hr/>  
    <List list={bookFilter}/>  
  </>  
)
```

preventDefault() : Cela permet d'éviter le comportement natif du formulaire HTML qui conduirait à un rechargement du navigateur.

```
const Search = (props) => { Show usages ⓘ HeH-Gianni *  
  return (  
    <div>  
      <form onSubmit={props.submit}>  
        <label htmlFor="search">Search: </label>  
        <input id="search" type="text" onChange={props.callback}/>  
        <button type="submit">Submit</button>  
      </form>  
      <p>Recherche de <strong>{props.term}</strong></p>  
    </div>  
  );  
};  
export default Search; Show usages ⓘ HeH-Gianni
```

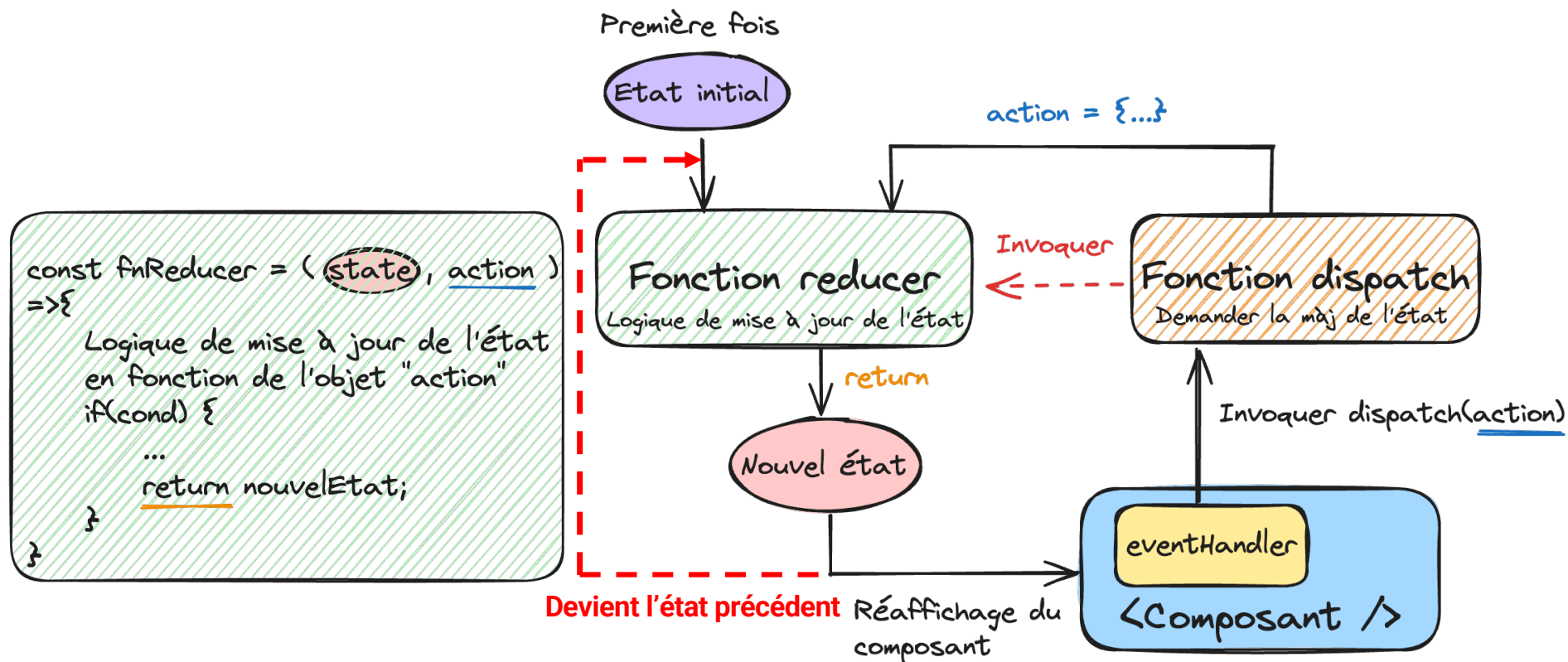
Partie 6: Gestion de l'état

L'utilisation de `useReducer` au lieu de `useState` est judicieuse dès que plusieurs valeurs d'état dépendent les unes des autres ou sont liées à un domaine.

Par exemple,

- `books`, `isLoading` et `httpError` sont tous liés à la collecte de données.
- Gestion d'un panier, `items` (ajout, suppression, modifier) et le total du panier.

Gestion de l'état - Principe



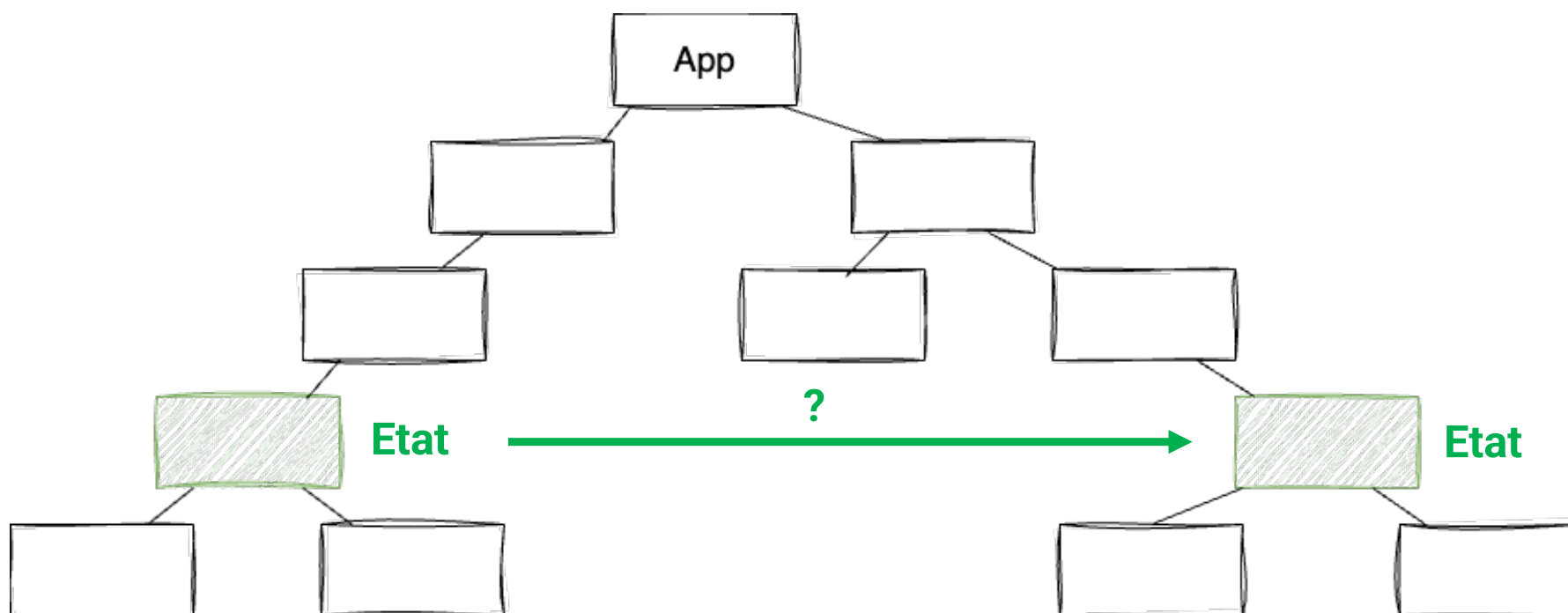
³ [state , ⁴ dispatch] = useReducer (² fnReducer , ¹ initialState)

function App() { Show usages ⓘ HeH-Gianni *

```
useEffect( effect: () :void => {  
  axios.get(url) Promise<AxiosResponse<...>>  
    .then((response : AxiosResponse<any> ) :void => {  
      console.log(response);  
      dispatchBook({type: "SET_BOOKS", payload: response.data})  
    }) Promise<void>  
    .catch((error) :void => {  
      console.log(error);  
    })  
}, deps: [url])  
  
const bookReducer = (state, action) => { Show usages new *  
  switch (action.type) {  
    case 'SET_BOOKS' :  
      return action.payload;  
  }  
}  
  
const [books, dispatchBook] = useReducer(bookReducer, initialState: []);
```

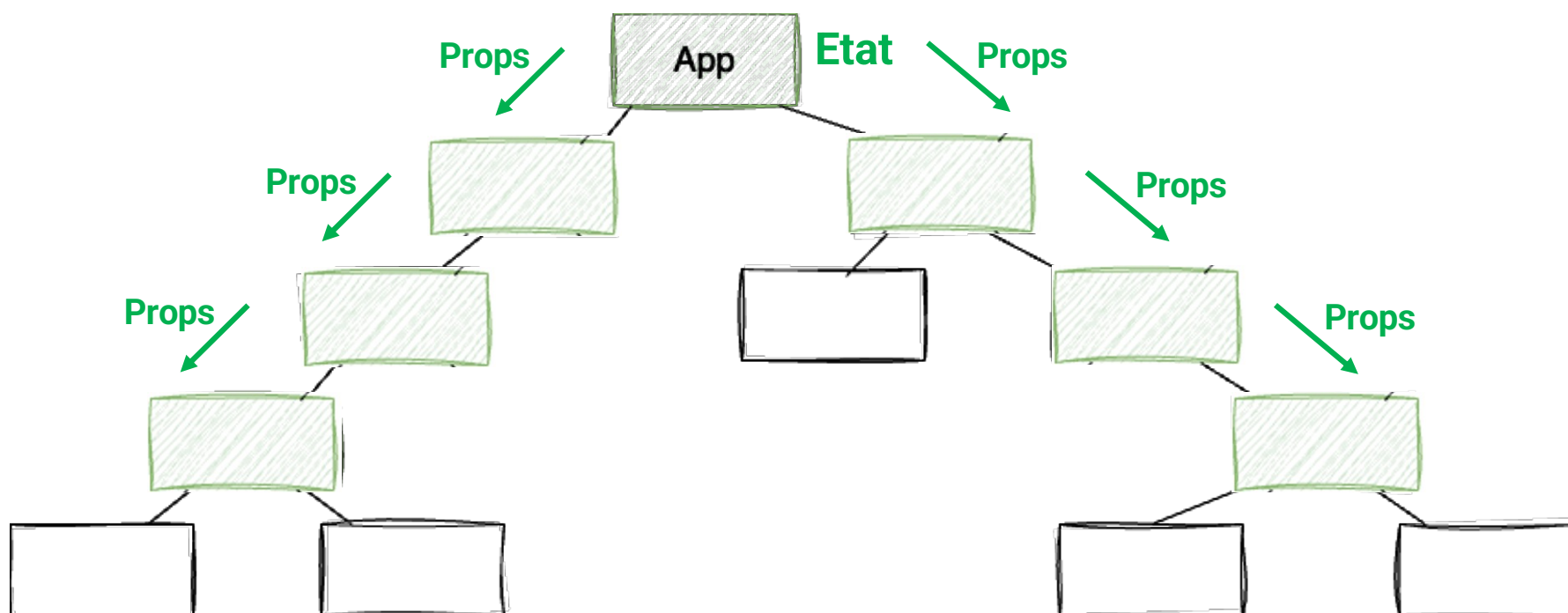

Gestion d'un panier

```
if (action.type === "REMOVE") {  
  const existingCartItemIndex = state.items.findIndex(  
    (item) => item.id === action.id  
  );  
  const existingCartItem = state.items[existingCartItemIndex];  
  
  let updatedItems = state.items.filter((item) => item.id !== action.id);  
  let updatedTotalAmount =  
    state.totalAmount - existingCartItem.price * existingCartItem.amount;  
  return {  
    items: updatedItems,  
    totalAmount: updatedTotalAmount,  
  };  
}
```



Solutions :

1. *faire remonter l'état*



Le **prop drilling** signifie qu'une valeur d'état est transmise à plusieurs composants par l'intermédiaire de props.

Le **prop drilling** (forage de props) présente quelques faiblesses:

- Les composants qui font partie du prop drilling ne sont plus vraiment **réutilisables** car tout composant qui veut les utiliser doit fournir une valeur pour le prop d'état transféré.
- Il entraîne également l'écriture d'une grande quantité de code de **surcharge** (le code qui accepte les props et les transmet)
- Le **remaniement** des composants est plus complexe car des props d'état doivent être ajoutés ou supprimés.

Pour ces raisons, le **prop drilling** n'est acceptable que si tous les composants impliqués ne sont utilisés que dans cette partie spécifique de l'application React, et que la probabilité de les réutiliser ou de les remanier est faible.

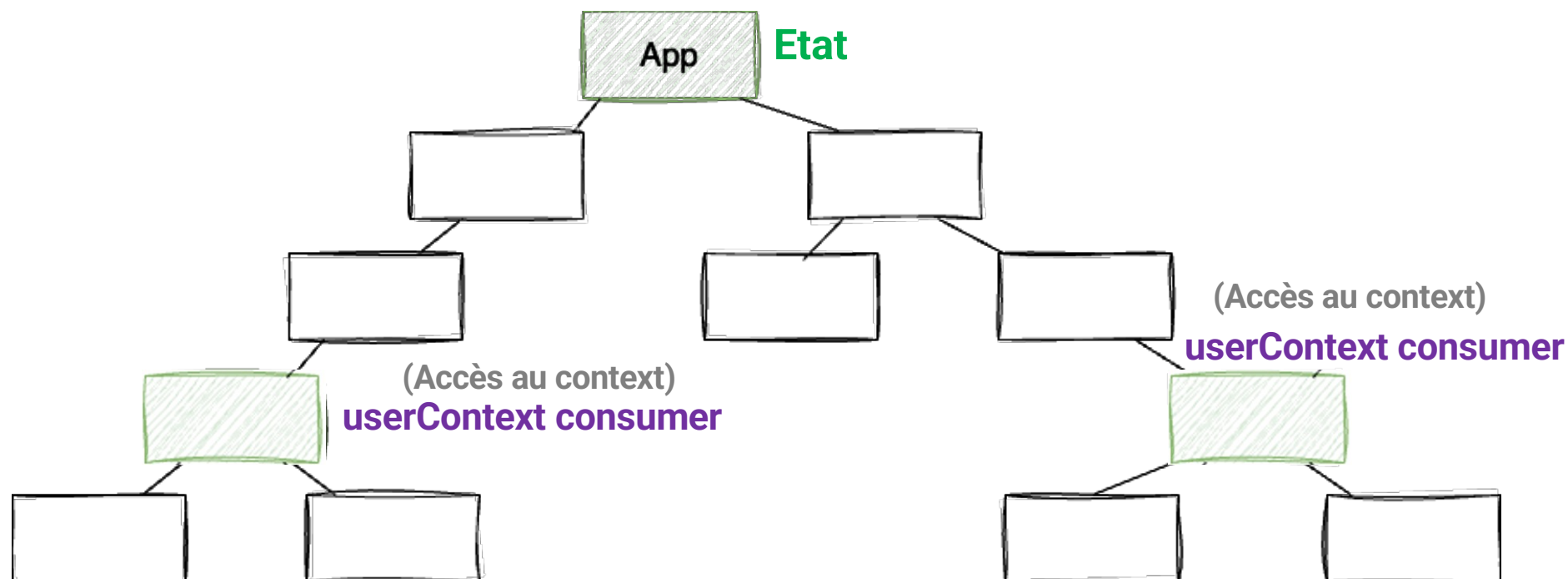
Solutions :

1. *faire remonter l'état*

2. *Context API*

(Maintient les données et fonctions)

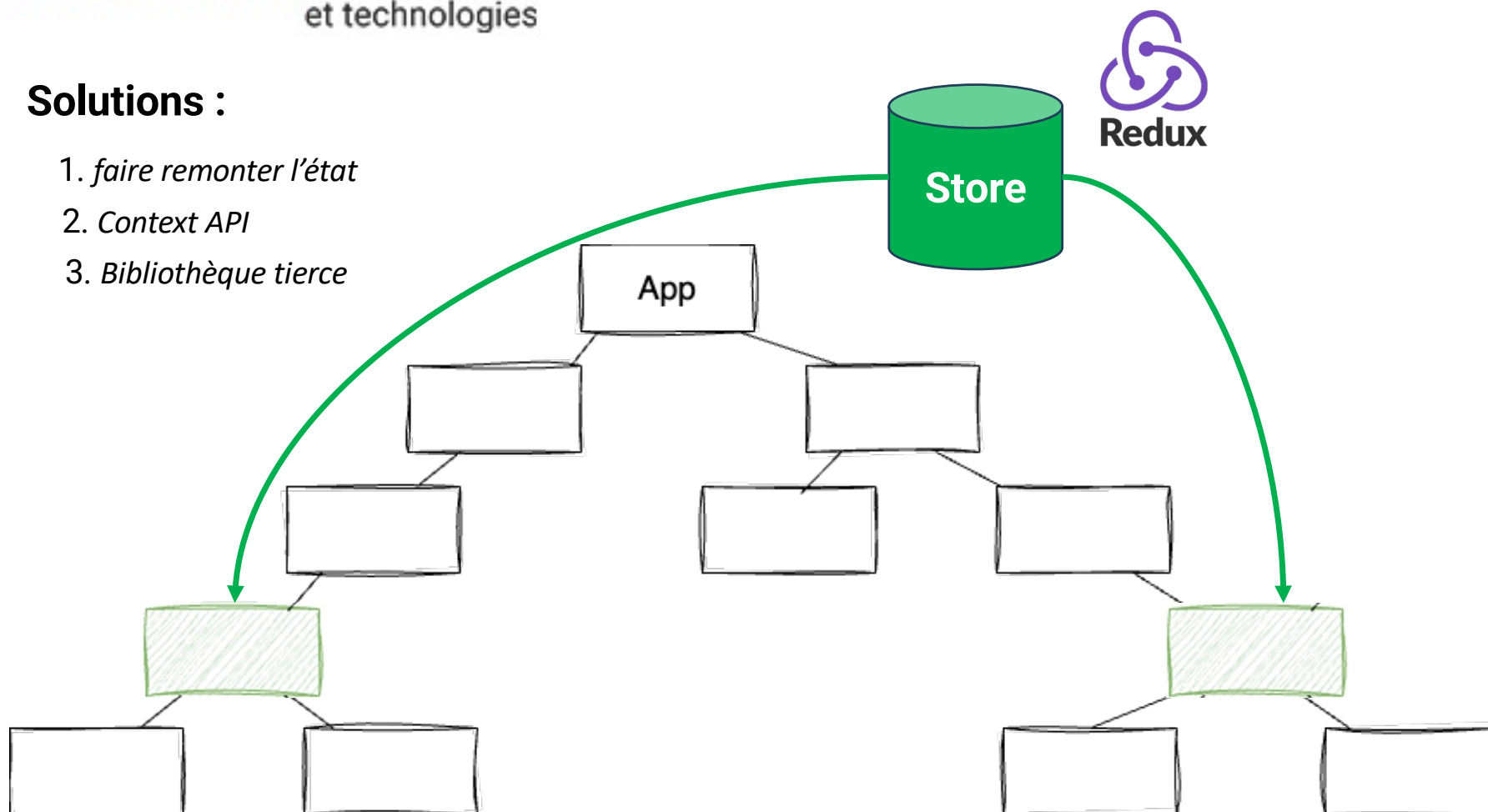
userContext Provider



Le **Contexte API** offre un moyen de **partager des données** (tableaux, objets, fonctions) entre des composants.

Solutions :

1. *faire remonter l'état*
2. *Context API*
3. *Bibliothèque tierce*



Hey React

50

<WishList /> ☐ Display search bar

Wish list

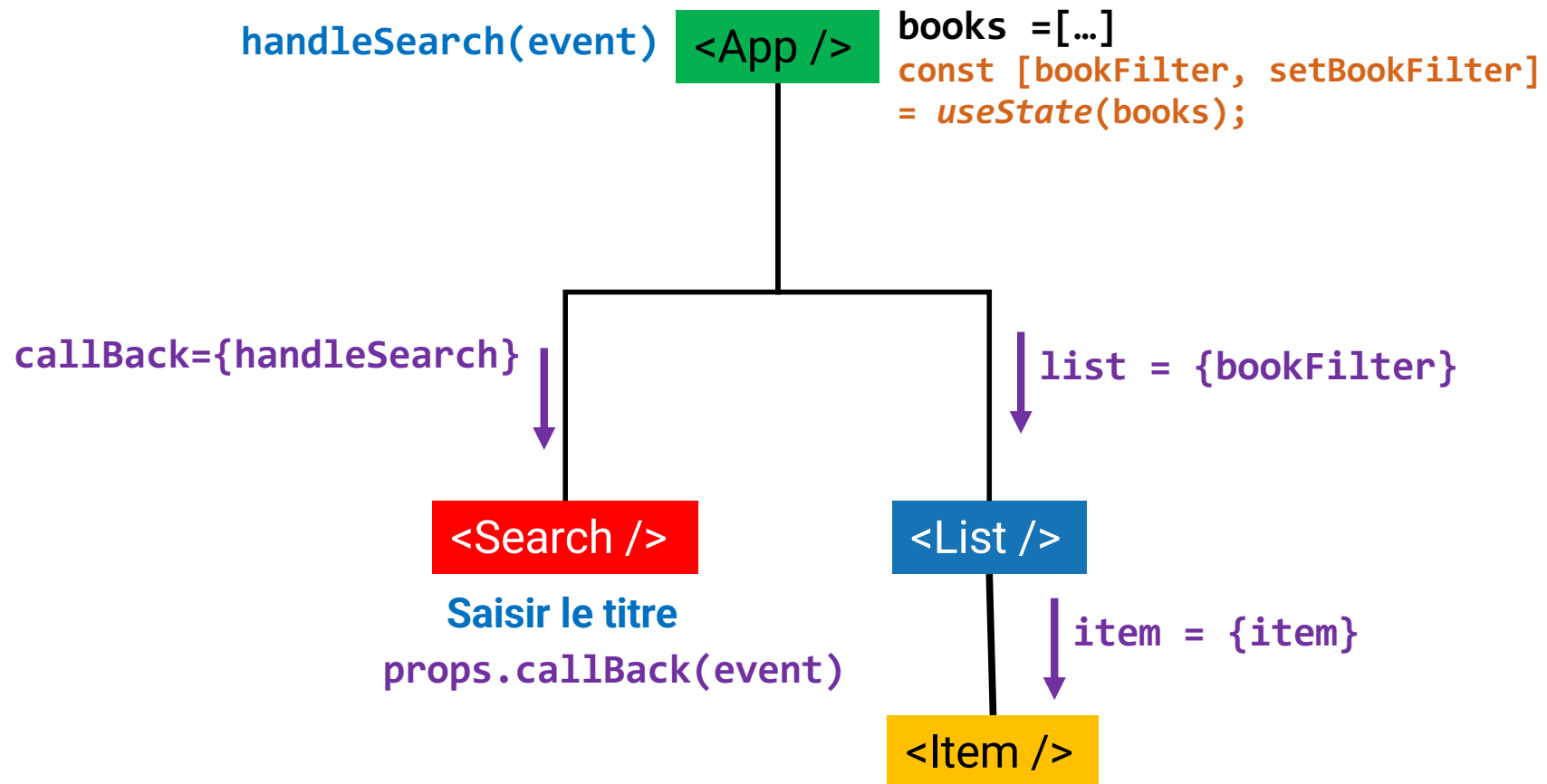
- To Kill a Mockingbird
- 1984
- Pride and Prejudice
- The Great Gatsby
- War and Peace

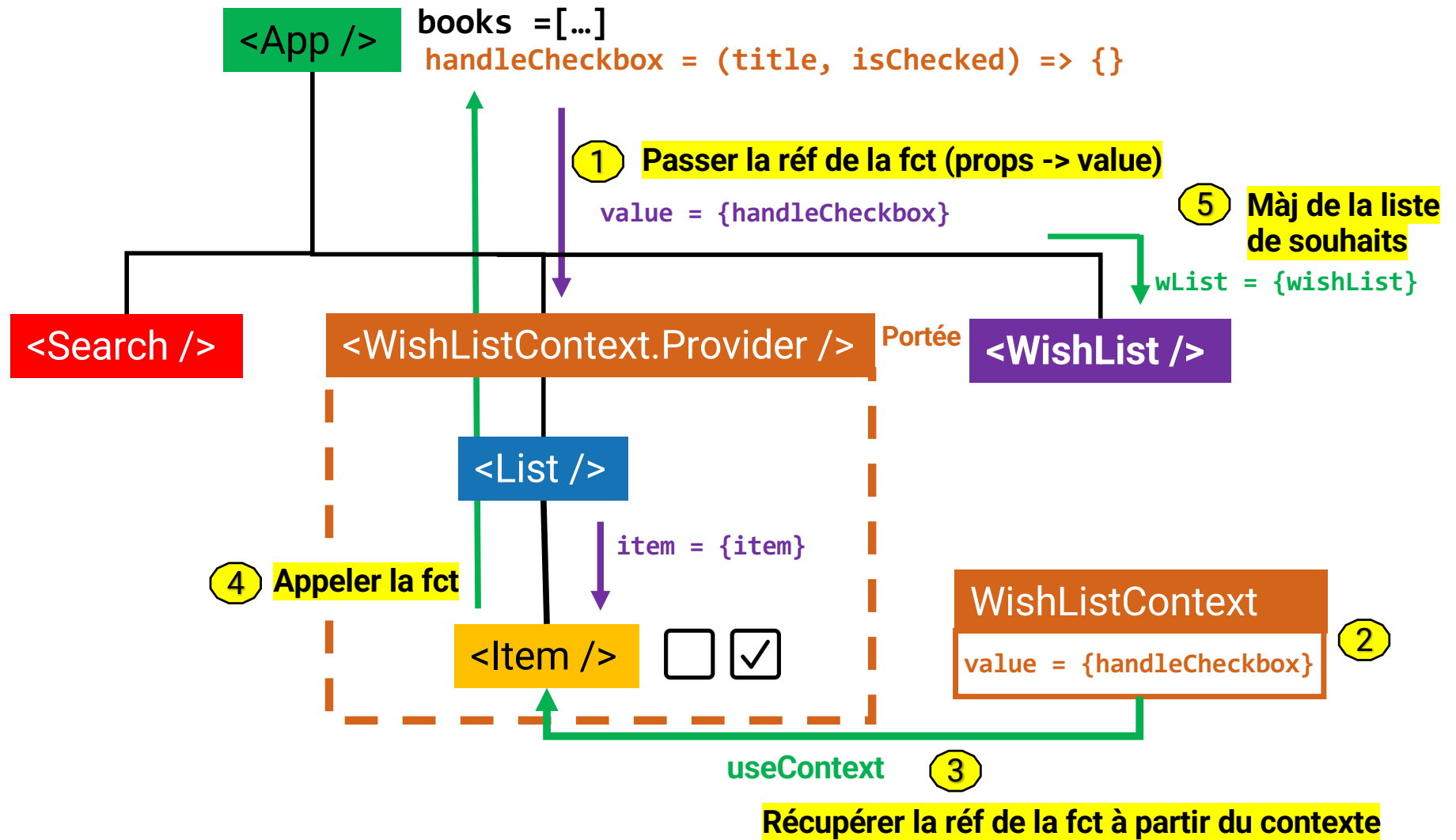
- To Kill a Mockingbird Harper Lee ☒
- 1984 George Orwell ☒
- Pride and Prejudice Jane Austen ☒
- The Great Gatsby F. Scott Fitzgerald ☒
- Moby-Dick Herman Melville ☐
- The Lord of the Rings J.R.R. Tolkien ☐
- The Catcher in the Rye J.D. Salinger ☐
- The Hobbit J.R.R. Tolkien ☐
- One Hundred Years of Solitude Gabriel Garcia Marquez ☐
- War and Peace Leo Tolstoy ☒

Nouvelle fonctionnalité :

L'utilisateur sélectionne les livres qu'il souhaite lire. À chaque fois qu'il en coche un, celui-ci s'ajoute à sa liste de souhaits.

Rappel





ex1 > src > Contexts >  WishListContext.jsx > ...

```
1  import { createContext } from "react";  
2  
3  const WishListContext = createContext(null);  
4  
5  export default WishListContext;
```

Créer un objet contexte nommé WishListContext.

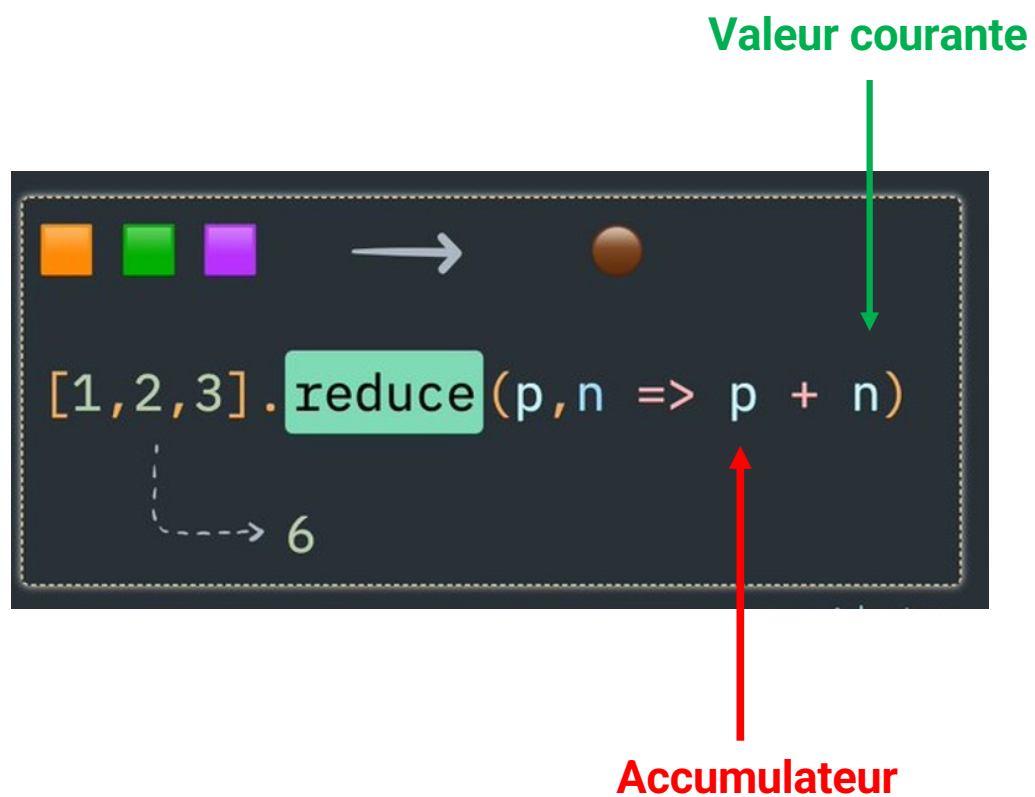
```
9  function App() {  
57  }  
58  const handleCheckbox = (title, isChecked) => {  
59    if(isChecked){  
60      wlist = [...wishList];  
61      wlist.push(title);  
62      setWishList(wlist);  
63    }  
64    else {  
65      wlist = [...wishList];  
66      let res = wlist.filter((item)=>{  
67        return item!=title;  
68      });  
69      setWishList(res);  
70    }  
71  };
```

Chaque objet **Contexte** est livré avec un composant React **Provider** qui permet aux composants consommateurs de s'abonner aux mises à jour du contexte.

```
9  function App() {
78    </h1>
79    <div>{books.length}</div>
80    <div>
81      <input
82        type="checkbox"
83        name="searchBar"
84        onChange={handleCheckedChange}
85      />
86      <label htmlFor="searchBar">Display search bar</label>
87    </div>
88    {isShow && <Search onSearch={handleSearch} />}
89    <hr />
90    <WishList wList={wishList} />
91    <hr />
92    <WishListContext.Provider value={handleCheckbox}>
93      <List list={books} />
94    </WishListContext.Provider>
95  </div>
96  };
97 }
```

① Passer la réf de la fct (props -> value)

```
1 | import { useContext } from "react";
2 | import WishListContext from "../Contexts/WishListContext";
3 | const Item = ({ url, title, author, num_comments, points } ) => {
4 |
5 |   const onChange = useContext(WishListContext); ③ Récupérer la réf de la fct à partir
6 |                                                     du contexte
7 |   const checkChange = (event) => {
8 |     onChange(title, event.target.checked); ④ Appeler la fct
9 |   }
10 |   return (
11 |     <li>
12 |       <span>
13 |         <a href={url}> {title}</a>
14 |       </span>
15 |       <span>{author}</span>
16 |       <span>{num_comments}</span>
17 |       <span>{points}</span>
18 |       <input type="checkbox" onChange={checkChange}/>
19 |     </li>
20 |   );
21 | };
```



La méthode **reduce()** réduit le tableau en appliquant **une fonction accumulatrice** sur deux éléments pour n'en retourner qu'un seul.

```
function somme(...param) {  
  let total = 0;  
  for (const p of param) {  
    total += p;  
  }  
  return total;  
}  
console.log(somme(1, 2, 3));  
// expected output: 6  
console.log(somme(1, 2, 3, 4));  
// expected output: 10
```

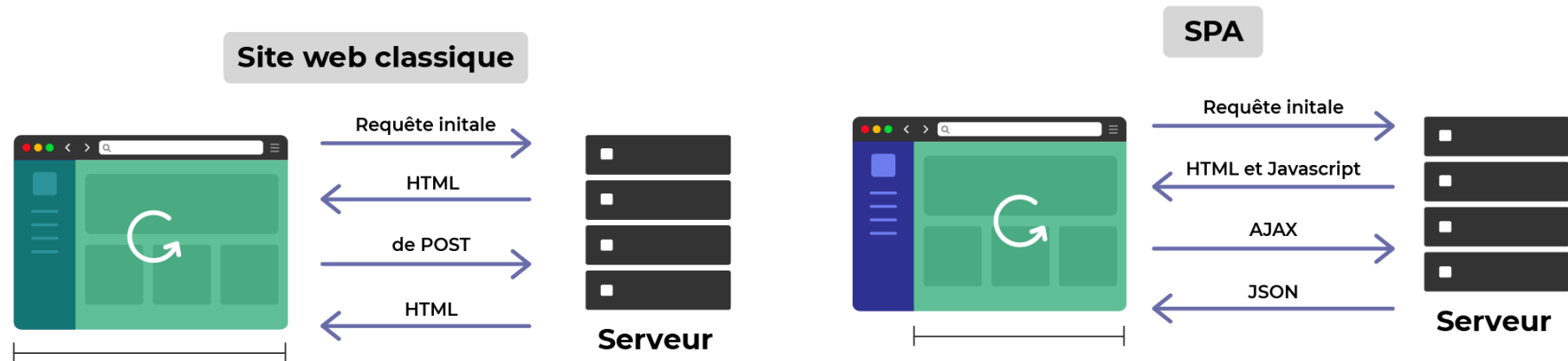
```
reduce( (accumulator, currentValue) =>  
  accumulator + currentValue, initialValue);
```

```
function sommeReduce(...reste){  
  return reste.reduce((acc,currentValue)=>acc+currentValue,0);  
}  
console.log('Somme avec méthode reduce : ' + sommeReduce(1, 2, 3));  
console.log('Somme avec méthode reduce : ' + sommeReduce(1, 2, 3,4));
```

Partie 7: Routage



« Une application monopage ou SPA (« Single-page application » en anglais) est une implémentation d'application web qui ne charge qu'un seul document web, puis met à jour le contenu du corps de ce document via des API JavaScript telles que [XMLHttpRequest](#) et [Fetch](#) lorsqu'un contenu différent doit être affiché. » Mozilla



Via le package **React Router**, votre application React peut détecter les changements de chemin d'URL et afficher des **composants** différents pour des chemins différents.

Installation :

```
npm i react-router
```

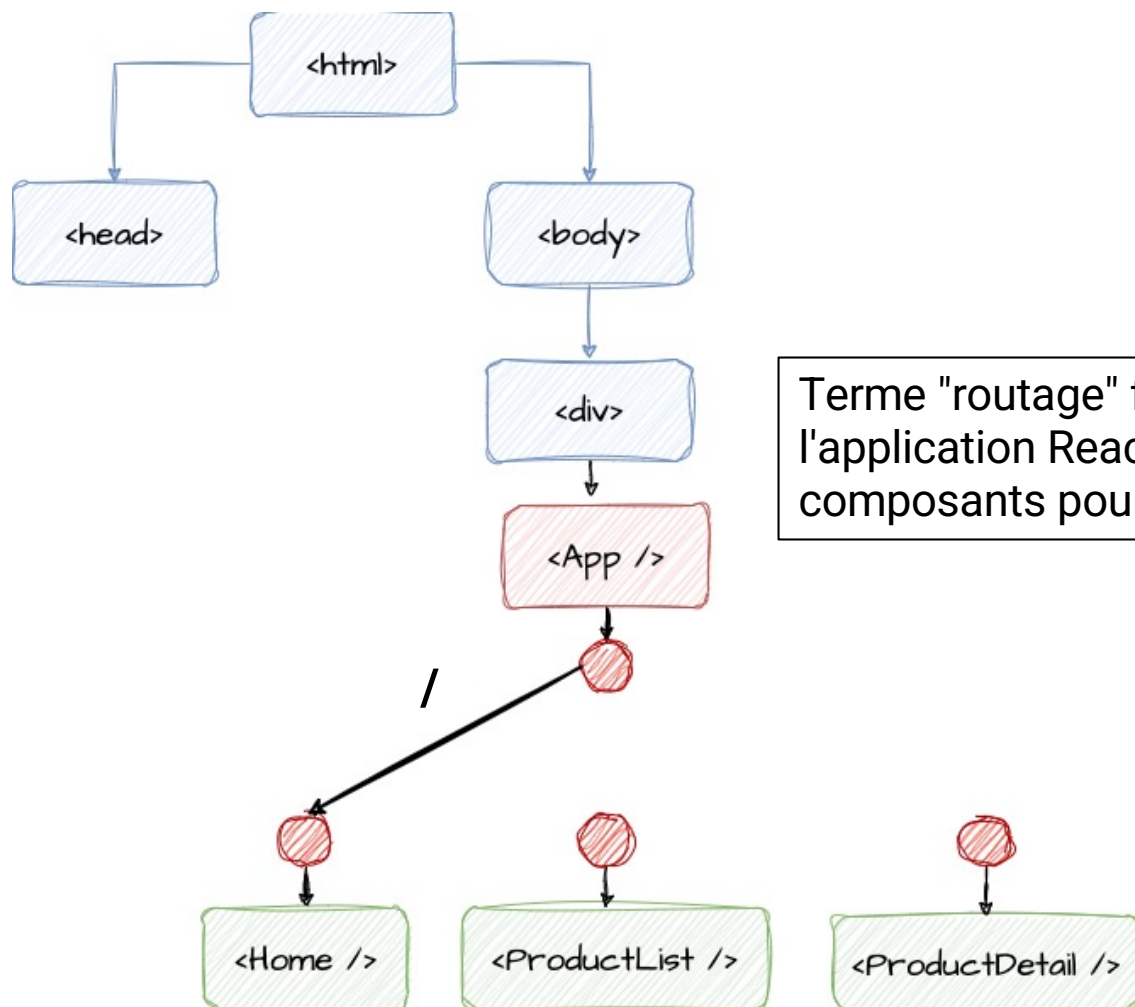
Path URL ↔ Composant React

/ => composant **<Home />** est chargé.

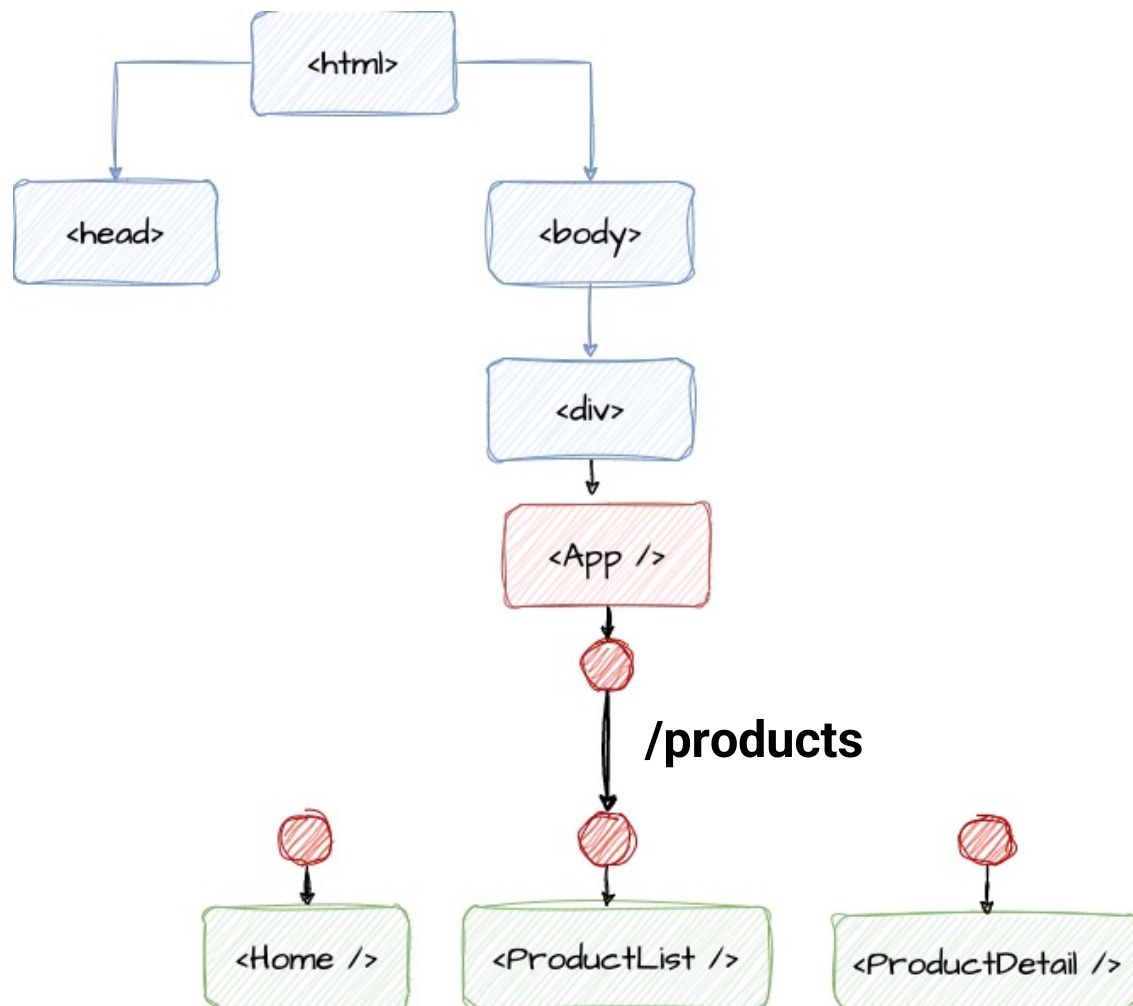
/products => composant **<ProductList />** est chargé.

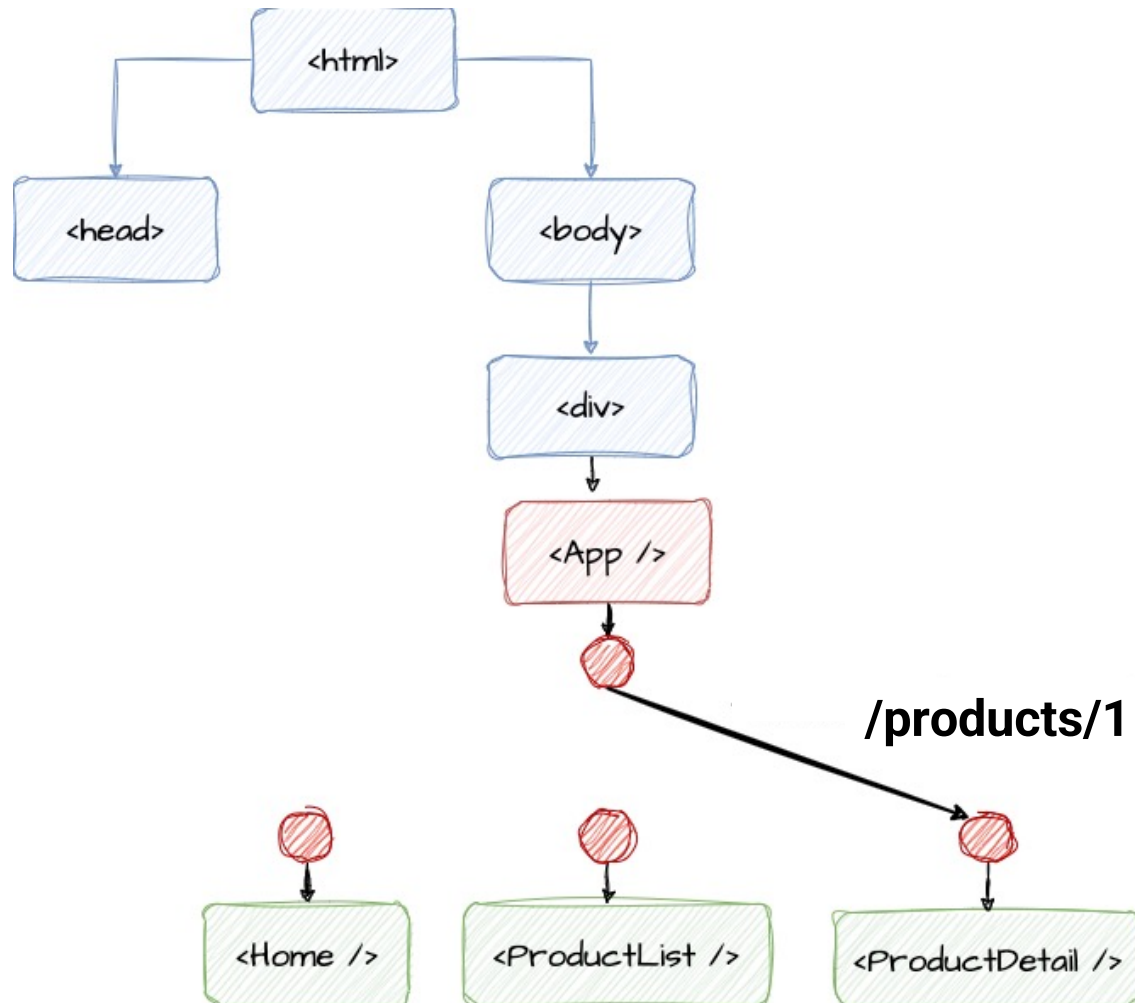
/products/1 => composant **<ProductDetail />** est chargé.

/about => composant **<AboutUs />** est chargé.



Terme "routage" fait référence à la capacité de l'application React à charger différents composants pour différents chemins d'URL.





Etapas de configuration du routage :

1. Créez différents composants pour vos différentes pages (par exemple, les composants **Dashboard** et **Orders**).
2. Utilisez les composants **BrowserRouter**, **Routes** et **Route** de la bibliothèque *React Router* pour activer le routage et définir les routes qui doivent être prises en charge par l'application React.

BrowserRouter, qui active toutes les fonctionnalités liées au routage (par exemple, il met en place un système qui détecte et analyse les changements d'URL).

Ce composant ne doit être utilisé **qu'une seule fois** dans l'ensemble de l'application (généralement dans votre composant racine, tel que le composant App).

Routes, qui contient vos définitions de routes. Vous pouvez utiliser ce composant plusieurs fois pour définir plusieurs groupes de routes indépendants.

Route, qui sert à définir une route individuelle. Le paramètre *path* est utilisé pour définir le chemin d'accès à l'URL qui doit activer cette route. Une fois activé, le contenu défini par la propriété *element* est rendu.

```
1  import { BrowserRouter, Routes, Route } from "react-router-dom";
2  import Dashboard from "../routes/Dashboard";
3  import Orders from "../routes/Orders";
4  function App() {
5    return (
6      <BrowserRouter>
7        <Routes>
8          <Route path="/" element={<Dashboard />} />
9          <Route path="/orders" element={<Orders />} />
10         </Routes>
11       </BrowserRouter>
12     );
13   }
14   export default App;
```

pages

└─ Dashboard.jsx
└─ Orders.jsx

Le composant **Link** est donc le composant par défaut à utiliser pour ajouter des liens aux sites web React (liens internes).

```
9      <BrowserRouter>
10      |   <nav>
11      |     <ul>
12      |       <li>
13      |         <Link to="/">Home</Link>
14      |       </li>
15      |       <li>
16      |         <Link to="/orders">All Orders</Link>
17      |       </li>
18      |     </ul>
19      |   </nav>
20      |   <p>
21      |     {counter} - <button onClick={incCounterHandler}>Increase</button>
22      |   </p>
23      |   <Routes>
24      |     <Route path="/" element={<Dashboard />} />
25      |     <Route path="/orders" element={<Orders />} />
26      |   </Routes>
27      </BrowserRouter>
```

Définition d'une route dynamique

```
<Route path="/orders/:id" element={ <OrderDetail /> } />
```

Chaque fois qu'un segment d'un chemin commence par deux points, *React Router* le traite comme un segment dynamique.

Cela signifie qu'il sera remplacé par une valeur différente dans le chemin d'accès à l'URL.

Pour le chemin d'accès **/orders/:id**, les chemins **/orders/o1**, **/orders/o2** et **/orders/abc** correspondraient tous et activeraient donc la route.

Extraire les données encodées dans le chemin d'URL à l'intérieur du composant.

Comme le composant est créé lors de la définition de l'itinéraire dans le composant **App**, vous ne pouvez pas passer de valeurs prop dynamiques, spécifiques à l'ID.

Le hook `useParams()` peut être utilisé pour accéder aux paramètres de la route actuellement activée.

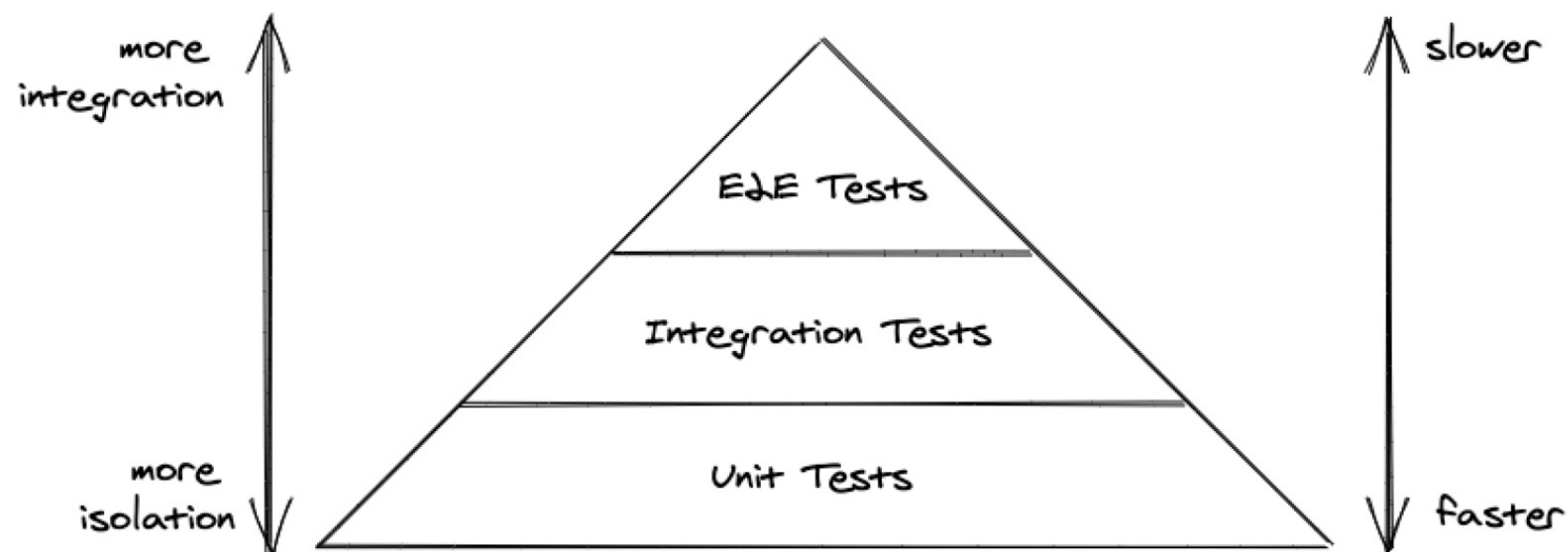
```
1  import { useParams } from "react-router-dom";
2  import Details from "../components/orders/Details";
3  import { getOrderById } from "../data/orders";
4  function OrderDetail() {
5      const params = useParams();
6      const orderId = params.id; // in this example, orderId is "o1" or "o2" etc.
7      const order = getOrderById(orderId);
8      return <Details order={order} />;
9  }
10 export default OrderDetail;
```

```
1  function OrdersList() {  
2    return (  
3      <ul className={classes.list}>  
4        {orders.map((order) => (  
5          <li key={order.id}>  
6            <Link to={`/orders/${order.id}`}>  
7              <OrderItem order={order} />  
8            </Link>  
9          </li>  
10         ) ) }  
11       </ul>  
12     );  
13  }
```

Partie 8: Tests



Tests en React – Pyramide des tests





Vitest est un framework de tests unitaires

Pour l'installer :

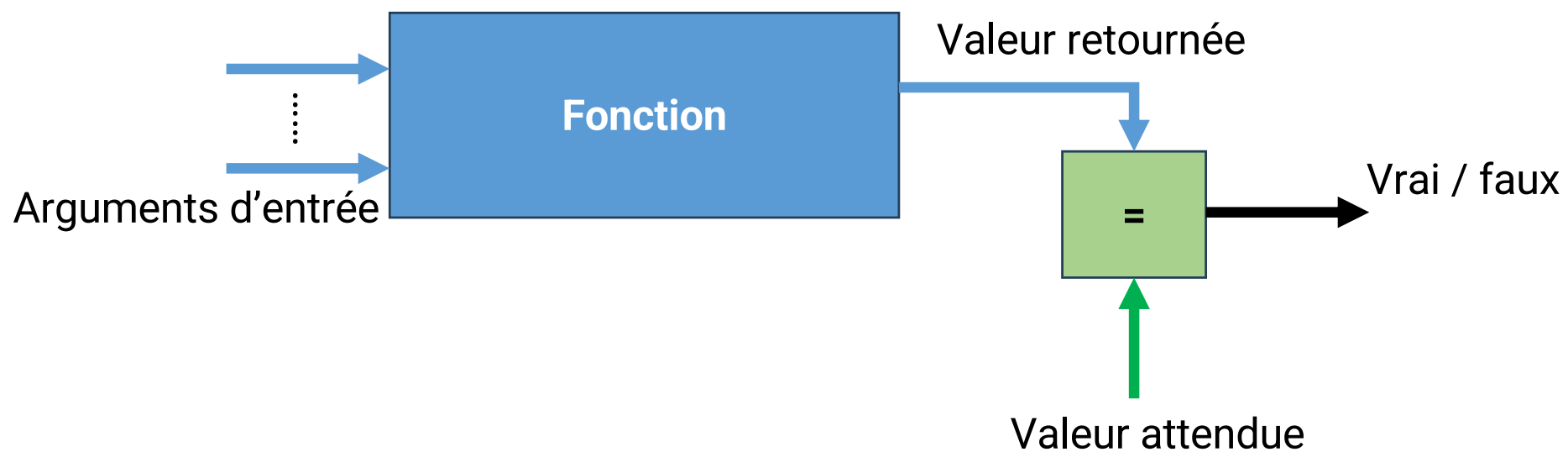
```
npm install vitest --save-dev
```

package.json

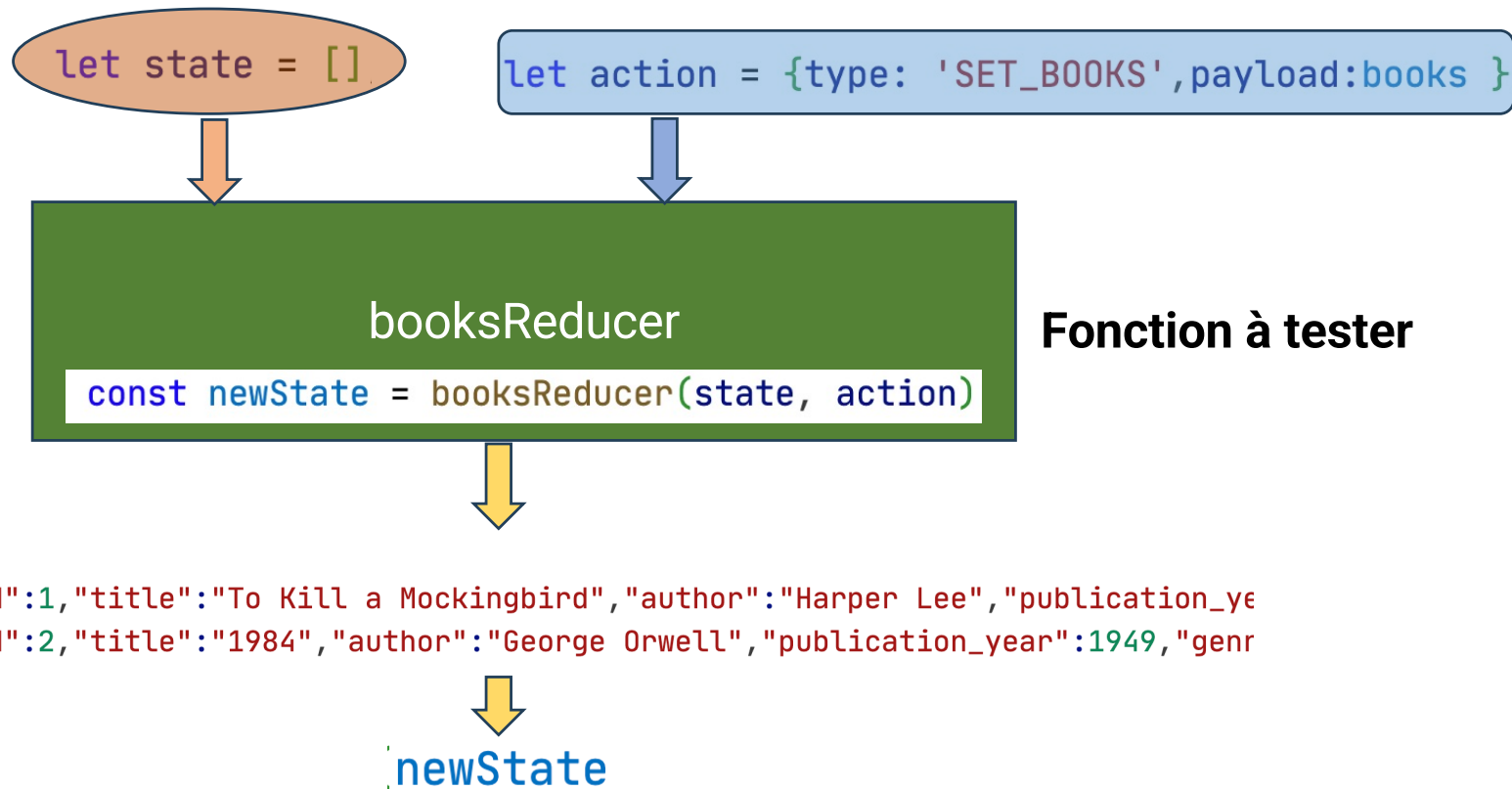
```
{  
  ...  
  "scripts": {  
    "dev": "vite",  
    "build": "vite build",  
    "test": "vitest",  
    "preview": "vite preview"  
  },  
  ...  
}
```

← Ajouter cette ligne

Tester une fonction




```
const books=[  
  {"id":1,"title":"To Kill a Mockingbird","author":"Harper Lee",  
  {"id":2,"title":"1984","author":"George Orwell","publication_y  
]
```



Il faut exporter la fonction reducer (booksReducer)

```
src > App.jsx > App
21  function App() {
97  }
98  export default App;
99
100 export {booksReducer};
```

src > App.test.jsx > ...

You, il y a 21 heures | 1 author (You)

```
1 import {booksReducer} from './App';
2 import { describe, it, expect } from 'vitest';
3 describe('Suites de test : booksReducer', () => {
4   it('Assign books', () => {
5     const books = [
6       { "id": 1, "title": "To Kill a Mockingbird", "author": "Harper Lee",
7       { "id": 2, "title": "1984", "author": "George Orwell", "publication_y
8     ]
9     let action = { type: 'SET_BOOKS', payload: books };
10    let state = [];
11
12    const newState = booksReducer(state, action);
13
14    const expectedState = [
15      { "id": 1, "title": "To Kill a Mockingbird", "author": "Harper Lee",
16      { "id": 2, "title": "1984", "author": "George Orwell", "publication_y
17    ]
18
19    expect(newState).toStrictEqual(expectedState);
20
21  })
22 }
```

Importer describe, it et expect

Définition d'une suite de tests

Définition d'un test

Définition d'une assertion

Lancer le test unitaire :

```
npm run test
```

```
SORTIE  TERMINAL  PORTS  GITLENS  CONSOLE DE DÉBOGAGE

(Use `node --trace-deprecation ...` to show where the warning v
✓ src/App.test.jsx (1)
  ✓ Suites de test : booksReducer (1)
    ✓ Assign books

Test Files  1 passed (1)
  Tests    1 passed (1)
  Start at 14:36:57
  Duration 1.50s (transform 75ms, setup 455ms, collect 111ms,
run 1.0s)

PASS Waiting for file changes...
press h to show help, press q to quit
```



React Testing Library (RTL) est utilisé pour le **rendu** des composants React, le **déclenchement d'événements** tels que les clics de souris et la sélection d'éléments HTML dans le DOM pour effectuer des assertions (render, screen et fireEvent).



Jest-dom est une bibliothèque qui étend Vitest afin de faciliter les **assertions** sur les éléments DOM. => **fonctions avancées de recherche dans le DOM** (toBeInTheDocument() et toHaveAttribute())



jsdom

HTML qui est rendu avec un composant React doit aboutir quelque part (par exemple dans le navigateur sans tête) pour le rendre **accessible pour les tests**.

```
npm install @testing-library/react @testing-library/jest-dom --save-dev
```

vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    setupFiles: ['./tests/setup.js'],
  },
});
```

Contenu du fichier : *tests/setup.js*

```
import { afterEach } from 'vitest';
import { cleanup } from '@testing-library/react';
//import matchers from '@testing-library/jest-dom/matchers';
import * as matchers from "@testing-library/jest-dom/matchers";
import { expect } from 'vitest';

// extends Vitest's expect method with methods from react-testing-library
expect.extend(matchers);

// runs a cleanup after each test case (e.g. clearing jsdom)
afterEach(() => {
  cleanup();
});
```

```
npm install jsdom --save-dev
```

vite.config.js

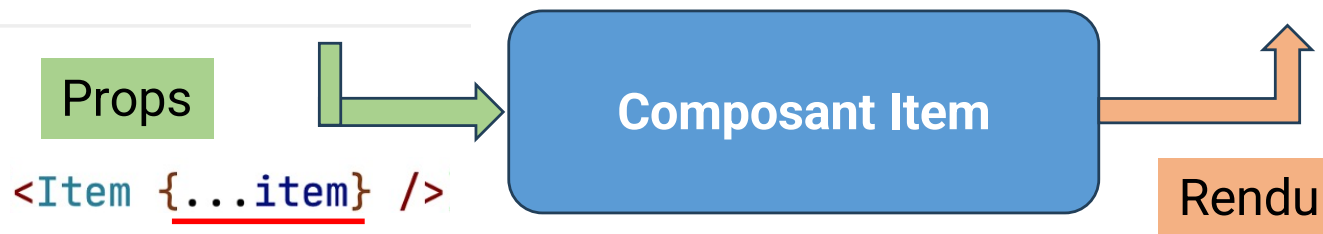
```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
  },
});
```



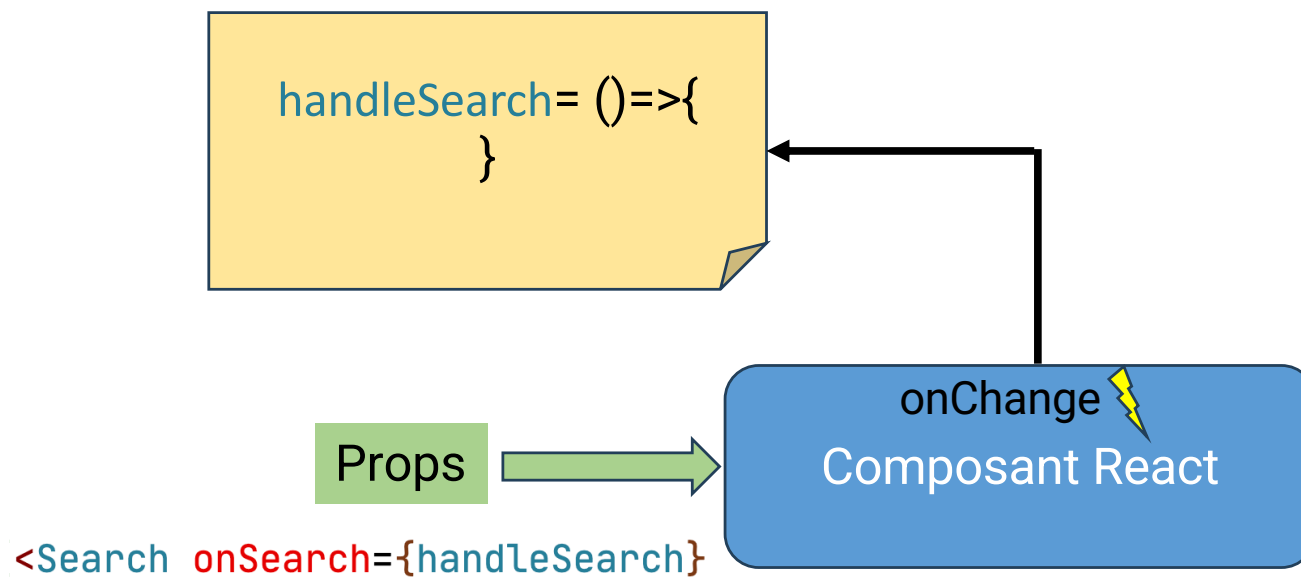
```
let item = {
  id: 1,
  title: "To Kill a Mockingbird",
  author: "Harper Lee",
  publication_year: 1960,
  genre: ["Fiction", "Classic"],
  description:
    "A classic novel depicting rac:
  cover_image: "https://fakeimg.pl,
```

```
<div>
  <li>
    <span>
      <a
        href="https://fakeimg.pl/667x1000/cc6600"
      >
        To Kill a Mockingbird
      </a>
    </span>
    <span>
      Harper Lee
    </span>
    <input
      type="checkbox"
    />
  </li>
</div>
```



src > Components >  Item.test.jsx > ...

```
1  import { describe, it, expect } from "vitest";
2  import { render, screen } from "@testing-library/react";
3
4  import Item from "../Item";
5
6  describe("Item", () => {
7    it("renders all properties", () => {
8 >     let item = { ...
17     };
18
19     render(<Item {...item} />);
20     screen.debug();
21     expect(screen.getByText("To Kill a Mockingbird")).toBeInTheDocument();
22     expect(screen.getByText("To Kill a Mockingbird")).toHaveAttribute(
23       "href",
24       "https://fakeimg.pl/667x1000/cc6600"
25     );
26     expect(screen.getByRole('checkbox')).toBeInTheDocument();
27   });
28 });
```



src > Components > Search.jsx > ...

```
2  const Search = (props) => {
3    const [searchTerm, setSearchTerm] = useState('');
4    const handleChange = (event) => {
5      setSearchTerm(event.target.value);
6      props.onChange(event);
7    };
8    return (
9      <div>
10        <label htmlFor="search">Search: </label>
11        <input id="search" type="text" value="React" onChange={handleChange} />
12
13        <p>Recherche de <strong>{searchTerm}</strong></p>
14      </div>
15    );
16  };
17  export default Search;
```

src > Components >  Search.test.jsx > ...

```
1  import { describe, it, expect, vi } from "vitest";
2  import { render, screen, fireEvent } from "@testing-library/react";
3
4  import Search from "../Search";
5
6  describe("Reacted event input zone", () => {
7    it("Changing the text in the text area", () => {
8      const handleSearch = vi.fn(); ← Fonction fictive
9      render(<Search onSearch={handleSearch} />);
10     fireEvent.change(screen.getByDisplayValue('React'), {
11       |   target: { value: 'Redux' }, ← Nouvelle valeur
12       |   });
13     expect(handleSearch).toHaveBeenCalledTimes(1); ← Assertion
14   });
15 });
```

Simuler l'événement onChange